ARRAYS 5
Program 1 : CLASSES & GRADES 5
Program 2 : To find the smallest and largest element 6
Program 3 : Sorting a Array 7
Program 4 : Program to multiply two polynomials 8
Program 5: Program to add two polynomials 12
Program 6: Program for matrix operations like dertminant, singular, etc 15
Program 7: matrix operations like addition, multiplicaton, etc. on 19
Program 8 : to merge two 1-D arrays after sorting them 22
Program 9: to implement an array(insertion,deletion,reversing the entire array and finding a element in an array). 25
Dyanmic memory 28
File handling 40
Program 1 40
Program 2 40
Program 3 41
Program 4 41
Program 5 42
Program 7 43
Program 8 43
Program 9 44
Program 10 44
Program 11 45
Program 12 46
Program 13 46
Program 14 47
Program 15 47
Program 16 48
Program 17 49
LINKED LIST 50
Program 1: To add char to a list 50
Program 2: list to implement adding,deleting,listing,searching and inserting of names 51
Program 3 : Program to form a linked list of integers 57
Program 4: Create,insert a value after last node,remove first node,adda node after a key value,remove a node that has key value, Delete all nodes of a list 58
Program 5: create a list; add and delete nodes 68
program 6: Link List Code Examples For Ordered Lists 71
Program 8: Create a list. Add & delete components and display 80
Program 9: Program to illustrate traversing a list 84
Program 10(check) 85
POINTERS 86
Program 1:Demononstration 86
Program 2 : Demo - Arrays and pointers 86
Program 3: Demo 86

Program 4: Fututе values of series of monthly deposits 87
Program 5: Sorting(Reordering) of strings - Linear 89
Program 7: Addition of two matrices(size dynamic) 90
Program 8: Program to count the no of vowels, consonants, digits, blanks and other special characters 92
Program 9: Outputs and Errors in programs with explaination 93
Program 10: Pointers(multiple) and some others basic functions 97
Program 11: Pointers and arrays (address of values in that array) 99
Program 12: passing address thro functions 99
Program 13: static arrays and pointers(truncation) 100
Program 14:Structures and pointers 100
program 15: return and store pointer values 101
Program 16: pointers used to swap 2 array variables 101
Program 17: function returning pointers 102
Program 18: return muliple values from a function 102
Program 19: call by reference 103
program 20: Call by value 104
Program 21: Pointers to pointers 104
Program 22: Pointers of different data types 105
Program 23: Programs using pointers 106
Program 24: What would be the output 106
Program 25: array of pointers 107
Program 26: 3-d arrays and pointers 107
Program 27: finding the address of elements in arrays using pointers(*,&) 108
Program 28: Address in 2-d array 109
Program 29 :Pointers and arrays (displaying of values using different notations) . 110
Program 30: pointers throu functions 110
Program 31: Arrays thro pointers 111
Program 32: increment pointers 111
Program 33: increament Pointers through Function 112
Program 34: printing the value with pointer notation without using pointers 113
Program 35: Some More programs on pointers 113
Program 36: Arrays and pointers (eg: ptr[-i]) 122
Program 37: Arrays and pointers fundamental 123
Program 38: Lvalue Error 123
Program 39: pointers and func 123
Program 40: Incrementing pointers(arrays); 124
Program 41: Pointers(multiple) and arrays 124
Program 42: Address of array thru pointers 125
Program 43: 3'd arrays 125
RECURSION 127
Program 1: Tower of Hanoi 127
Program 2: Implement recursive/primitive recursive functions in C 133
Program 3 : Queues using recursion 140
Program 4:Queue Strings implimentaton 142
Program 5:Product of two nos using Recursion 144

Program 6: GCD using recursion 145
Program 7: Fibonocci Using Recursion 146
Program 8:Factorial Using Recursion 147
Program 11: Factorial Using Recursion(Alternate and better) 147
Program 12 : Reversing a string using recursion 148
SORTINGS AND SEARCHINGS 149
Shell Sort 149
Shell sort 150
Quick Sort 151
Quick Sort 152
Linear Search 154
Linear Search 154
Insertion Sort 155
Insertion Search 156
Insertion Search 157
Insertion Search 158
Bubble Sort 158
Bubble Sort 159
Binary Search 161
STACKS AND QUEUES 162
Stack 1 162
Queues 1 164
Queues 2 166
STRUCTURES 169
Program 1:Basics fo Union using int86 and REGS 169
Program 2: Struct and Unions interupts 169
Program 3: Interupts 170
program 4: Structures within Unions 170
Program 5:Unions 171
Program 6: Records for agents and inches 171
Program 7: Agent name and no 174
Program 8: Example on Struct 175
Program 9:Example2 assigning a Struct to another 175
Program 10: initializing struct 176
Program 11: Reading values to struct items 176
Program 12: Item initialization of struct 177
Program 13: Example 3 178
program 14: Use of enum 178
Program 15: CUSTOMER BILLING SYSTEM 179
Program 16: use of typedef and passing structures thro functions for CUSTOMER BILLING SYSTEM 181
Program 17: Example on argc 184
Program 18: Maintaining Student Info 184
TREES 186
program 1: Insert, preorder, inorder,postorder,search 186
program 2: Stack implementation of Linked List 190

Program 3: Program to maintain a heap 193
Program 4: Program which maintains a B-tree of order 5 196
Program 5: Program to maintain an AVL tree 205
Program 6: Program to reconstruct a binary search tree and traversals 214
program 7: Program to maintain a threaded binary tree 219
Program 8: Program to insert and delete a node from the binary search tree 227
Program 9: Program to implement a binary search tree. 232
Program 10: Program to build a binary search tree from an array 235
Program 11: build a binary search tree from arrays 236

ARRAYS

Program 1 : CLASSES & GRADES

```
#include<stdio.h>
#include<ctype.h>
#include<stdlib.h>

#define CLASSES 2
#define GRADES 3

int grade[CLASSES][GRADES];

void enter_grades(void);
int get_grade(int num);
void disp_grades(int g[][GRADES]);

void main(void)
{
        char ch, str[80];

        for(;;)
        {
          do
          {
            printf("(E)nter grades\n");
            printf("(R)eport grades\n");
            printf("(Q)uit\n");
            gets(str);
            ch = toupper(*str);
          } while(ch !='E' && ch !='R' && ch != 'Q');

          switch(ch)
          {
                case 'E':
                        enter_grades();
                        break;
                case 'R':
                        disp_grades(grade);
                        break;
                case 'Q' :
                        exit(0);
          }
        }
}
void enter_grades(void)
```

```
{
        int t, i;

        for (t = 0; t < CLASSES; t++)
        {
                printf("Class # %d : \n", t+1);
                for (i = 0; i < GRADES; ++i)
                grade[t][i] = get_grade(i);
        }
}

int get_grade(int num)
{
        char s[80];
        printf("Enter grade for student #%d:\n", num + 1);
        gets(s);
        return(atoi(s));
}

void disp_grades(int g[][GRADES])
{
  int t, i;
  for (t = 0; t < CLASSES; ++t)
  {
    printf("Class # %d:\n", t + 1);
    for (i = 0; i < GRADES; ++i)
        printf("Student #%d is is %d\n", i+1, g[t][i]);
  }
}
```

Program 2 : To find the smallest and largest element

```
#include<stdio.h>
#include<conio.h>

int tab[5];
int i, j;
int low,hig;

main()
 {

        clrscr();
        printf("Enter Five Nos\n");
        for (i = 0; i < 5; i++)
```

```
        {
                scanf("\n\t%d", &tab[i]);
        }
        hig = low = tab[0];

        for (j = 0; j < i; j++)
        {
                if (tab[j] < low)
                  low = tab[j];
                else
                if (tab[j] > hig)
                  hig = tab[j];
        }
      printf("The Lowest No keyed In is %d ", low);
        printf("\nThe Highest No keyed In is %d ", hig);
        getch();
}
```

Program 3 : Sorting a Array

```
#include<stdio.h>
#include<conio.h>
int i,j,k,l,m, tab[5], temp;
main()
        {
                clrscr();
                printf("Enter 5 Nos ");
                for (i = 0; i < 5; i++)
                {
                        scanf("%d", &tab[i]);
                }

                for (k = 0; k < 5; k++)
                   for (l = k+1; l < 5 ; l++)
                    if (tab[k] > tab[l])
                    {
                          temp = tab[k];
                          tab[k]   = tab[l];
                          tab[l]   = temp;
                     }
                printf("The Sorted Array Is ");
                for (l = 0; l < i ; l ++)
                {
```

```
                printf("\n%d", tab[l]);
        }
            getch();
}
```

Program 4 : Program to multiply two polynomials

```
#include <stdio.h>
#include <conio.h>

#define MAX 10

struct term
{
        int coeff ;
        int exp ;
} ;

struct poly
{
        struct term t [10] ;
        int noofterms ;
} ;

void initpoly ( struct poly *) ;
void polyappend ( struct poly *, int, int ) ;
struct poly polyadd ( struct poly, struct poly ) ;
struct poly polymul ( struct poly, struct poly ) ;
void display ( struct poly ) ;

void main( )
{
        struct poly p1, p2, p3 ;

        clrscr( ) ;

        initpoly ( &p1 ) ;
        initpoly ( &p2 ) ;
        initpoly ( &p3 ) ;

        polyappend ( &p1, 1, 4 ) ;
        polyappend ( &p1, 2, 3 ) ;
        polyappend ( &p1, 2, 2 ) ;
        polyappend ( &p1, 2, 1 ) ;
```

```
        polyappend ( &p2, 2, 3 ) ;
        polyappend ( &p2, 3, 2 ) ;
        polyappend ( &p2, 4, 1 ) ;

        p3 = polymul ( p1, p2 ) ;

        printf ( "\nFirst polynomial:\n" ) ;
        display ( p1 ) ;

        printf ( "\n\nSecond polynomial:\n" ) ;
        display ( p2 ) ;

        printf ( "\n\nResultant polynomial:\n" ) ;
        display ( p3 ) ;

        getch( ) ;
}

/* initializes elements of struct poly */
void initpoly ( struct poly *p )
{
        int i ;
        p -> noofterms = 0 ;
        for ( i = 0 ; i < MAX ; i++ )
        {
                p -> t[i].coeff = 0 ;
                p -> t[i].exp = 0 ;
        }
}

/* adds the term of polynomial to the array t */
void polyappend ( struct poly *p, int c, int e )
{
        p -> t[p -> noofterms].coeff = c ;
        p -> t[p -> noofterms].exp =  e ;
        ( p -> noofterms ) ++ ;
}

/* displays the polynomial equation */
void display ( struct poly p )
{
        int flag = 0, i ;
        for ( i = 0 ; i < p.noofterms ; i++ )
        {
                if ( p.t[i].exp != 0 )
                        printf ( "%d x^%d + ", p.t[i].coeff, p.t[i].exp ) ;
```

```
                else
                {
                        printf ( "%d", p.t[i].coeff ) ;
                        flag = 1 ;
                }
        }
        if ( !flag )
                printf ( "\b\b  " ) ;

}
/* adds two polynomials p1 and p2 */
struct poly polyadd ( struct poly p1, struct poly p2 )
{
        int i, j, c ;
        struct poly p3 ;
        initpoly ( &p3 ) ;

        if ( p1.noofterms > p2.noofterms )
                c = p1.noofterms ;
        else
                c = p2.noofterms ;

        for ( i = 0, j = 0 ; i <= c ; p3.noofterms++ )
        {
                if ( p1.t[i].coeff == 0 && p2.t[j].coeff == 0 )
                        break ;
                if ( p1.t[i].exp >= p2.t[j].exp )
                {
                        if ( p1.t[i].exp == p2.t[j].exp )
                        {
                                p3.t[p3.noofterms].coeff = p1.t[i].coeff + p2.t[j].coeff ;
                                p3.t[p3.noofterms].exp = p1.t[i].exp ;
                                i++ ;
                                j++ ;
                        }
                        else
                        {
                                p3.t[p3.noofterms].coeff = p1.t[i].coeff ;
                                p3.t[p3.noofterms].exp = p1.t[i].exp ;
                                i++ ;
                        }
                }
                else
                {
                        p3.t[p3.noofterms].coeff = p2.t[j].coeff ;
                        p3.t[p3.noofterms].exp = p2.t[j].exp ;
```

```
                        j++ ;
                }
        }
        return p3 ;
}

/* multiplies two polynomials p1 and p2 */
struct poly polymul ( struct poly p1, struct poly p2 )
{
        int coeff, exp ;
        struct poly temp, p3 ;

        initpoly ( &temp ) ;
        initpoly ( &p3 ) ;

        if ( p1.noofterms != 0 && p2.noofterms != 0 )
        {
                int i ;
                for ( i = 0 ; i < p1.noofterms ; i++ )
                {
                        int j ;

                        struct poly p ;
                        initpoly ( &p ) ;

                        for ( j = 0 ; j < p2.noofterms ; j++ )
                        {
                                coeff = p1.t[i].coeff * p2.t[j].coeff ;
                                exp = p1.t[i].exp + p2.t[j].exp ;
                                polyappend ( &p, coeff, exp ) ;
                        }

                        if ( i != 0 )
                        {
                                p3 = polyadd ( temp, p ) ;
                                temp = p3  ;
                        }
                        else
                                temp = p ;
                }
        }
        return p3 ;
}
```

O/P:- First polynomial:
1 x^4 + 2 x^3 + 2 x^2 + 2 x^1

Second polynomial:
2 x^3 + 3 x^2 + 4 x^1

Resultant polynomial:
2 x^7 + 7 x^6 + 14 x^5 + 18 x^4 + 14 x^3 + 8 x^2

Program 5: Program to add two polynomials

```
#include <stdio.h>
#include <conio.h>

#define MAX 10

struct term
{
        int coeff ;
        int exp ;
} ;

struct poly
{
        struct term t [10] ;
        int noofterms ;
} ;


void initpoly ( struct poly * ) ;
void polyappend ( struct poly *, int c, int e ) ;
struct poly polyadd ( struct poly, struct poly ) ;
void display ( struct poly ) ;

void main( )
{
        struct poly p1, p2, p3 ;

        clrscr( ) ;

        initpoly ( &p1 ) ;
        initpoly ( &p2 ) ;
        initpoly ( &p3 ) ;

        polyappend ( &p1, 1, 7 ) ;
        polyappend ( &p1, 2, 6 ) ;
        polyappend ( &p1, 3, 5 ) ;
```

```
        polyappend ( &p1, 4, 4 ) ;
        polyappend ( &p1, 5, 2 ) ;

        polyappend ( &p2, 1, 4 ) ;
        polyappend ( &p2, 1, 3 ) ;
        polyappend ( &p2, 1, 2 ) ;
        polyappend ( &p2, 1, 1 ) ;
        polyappend ( &p2, 2, 0 ) ;

        p3 = polyadd ( p1, p2 ) ;

        printf ( "\nFirst polynomial:\n" ) ;
        display ( p1 ) ;

        printf ( "\n\nSecond polynomial:\n" ) ;
        display ( p2 ) ;

        printf ( "\n\nResultant polynomial:\n" ) ;
        display ( p3 ) ;

        getch( ) ;
}

/* initializes elements of struct poly */
void initpoly ( struct poly *p )
{
        int i ;
        p -> noofterms = 0 ;
        for ( i = 0 ; i < MAX ; i++ )
        {
                p -> t[i].coeff = 0 ;
                p -> t[i].exp = 0 ;
        }
}

/* adds the term of polynomial to the array t */
void polyappend ( struct poly *p, int c, int e )
{
        p -> t[p -> noofterms].coeff = c ;
        p -> t[p -> noofterms].exp =  e ;
        ( p -> noofterms ) ++ ;
}

/* displays the polynomial equation */
void display ( struct poly p )
{
```

```
        int flag = 0, i ;
        for ( i = 0 ; i < p.noofterms ; i++ )
        {
                if ( p.t[i].exp != 0 )
                        printf ( "%d x^%d + ", p.t[i].coeff, p.t[i].exp ) ;
                else
                {
                        printf ( "%d", p.t[i].coeff ) ;
                        flag = 1 ;
                }
        }
        if ( !flag )
                printf ( "\b\b  " ) ;

}

/* adds two polynomials p1 and p2 */
struct poly polyadd ( struct poly p1, struct poly p2 )
{
        int i, j, c ;
        struct poly p3 ;
        initpoly ( &p3 ) ;

        if ( p1.noofterms > p2.noofterms )
                c = p1.noofterms ;
        else
                c = p2.noofterms ;

        for ( i = 0, j = 0 ; i <= c ; p3.noofterms++ )
        {
                if ( p1.t[i].coeff == 0 && p2.t[j].coeff == 0 )
                        break ;
                if ( p1.t[i].exp >= p2.t[j].exp )
                {
                        if ( p1.t[i].exp == p2.t[j].exp )
                        {
                                p3.t[p3.noofterms].coeff = p1.t[i].coeff + p2.t[j].coeff ;
                                p3.t[p3.noofterms].exp = p1.t[i].exp ;
                                i++ ;
                                j++ ;
                        }
                        else
                        {
                                p3.t[p3.noofterms].coeff = p1.t[i].coeff ;
                                p3.t[p3.noofterms].exp = p1.t[i].exp ;
                                i++ ;
```

```
                        }
                }
                else
                {
                        p3.t[p3.noofterms].coeff = p2.t[j].coeff ;
                        p3.t[p3.noofterms].exp = p2.t[j].exp ;
                        j++ ;
                }
        }
        return p3 ;
}
```

o/p :- First polynomial:
1 x^7 + 2 x^6 + 3 x^5 + 4 x^4 + 5 x^2

Second polynomial:
1 x^4 + 1 x^3 + 1 x^2 + 1 x^1 + 2

Resultant polynomial:
1 x^7 + 2 x^6 + 3 x^5 + 5 x^4 + 1 x^3 + 6 x^2 + 1 x^1 + 2

Program 6: Program for matrix operations like dertminant, singular, etc

```
#include <stdio.h>
#include <conio.h>
#include <math.h>

#define MAX 3

void matrix ( int [3][3] ) ;
void create ( int [3][3] ) ;
void display ( int [3][3] ) ;
void matmul ( int [3][3], int [3][3], int [3][3] ) ;
void transpose ( int [3][3], int [3][3] ) ;
int determinant ( int [3][3] ) ;
int isortho ( int [3][3] ) ;

void main( )
{
        int mat [3][3], d ;

        clrscr( ) ;

        printf ( "\nEnter elements for array: \n\n" ) ;
```

```
        create ( mat ) ;

        printf ( "\nThe Matrix: \n" ) ;
        display ( mat ) ;

        d = determinant ( mat ) ;
        printf ( "\nThe determinant for given matrix: %d.\n", d ) ;

        if ( d == 0 )
                printf ( "\nMatrix is singular.\n" ) ;
        else
                printf ( "\nMatrix  is not singular.\n" ) ;

        d = isortho ( mat ) ;

        if ( d != 0 )
                printf ( "\nMatrix is orthogonal.\n" ) ;
        else
                printf ( "\nMatrix is not orthogonal.\n" ) ;

        getch( ) ;
}

/* initializes the matrix mat with 0 */
void matrix ( int mat[3][3] )
{
        int i, j ;
        for ( i = 0 ; i < MAX ; i++ )
        {
                for ( j = 0 ; j < MAX ; j++ )
                        mat[i][j] = 0 ;
        }
}

/* creates matrix mat */
void create ( int mat[3][3] )
{
        int n, i, j ;
        for ( i = 0 ; i < MAX ; i++ )
        {
                for ( j = 0 ; j < MAX ; j++ )
                {
                        printf ( "Enter the element: " ) ;
                        scanf ( "%d", &n ) ;
                        mat[i][j] = n ;
                }
```

```
        }
}

/* displays the contents of matrix */
void display ( int mat[3][3] )
{
        int i, j ;
        for ( i = 0 ; i < MAX ; i++ )
        {
                for ( j = 0 ; j < MAX ; j++ )
                        printf ( "%d\t", mat[i][j] ) ;
                printf ( "\n" ) ;
        }
}

/* multiplies two matrices */
void matmul ( int mat1[3][3], int mat2[3][3], int mat3[3][3] )
{
        int i, j, k ;
        for ( k = 0 ; k < MAX ; k++ )
        {
                for ( i = 0 ; i < MAX ; i++ )
                {
                        mat3[k][i] = 0 ;
                        for ( j = 0 ; j < MAX ; j++ )
                                mat3[k][i] += mat1[k][j] * mat2[j][i] ;
                }
        }
}

/* obtains transpose of matrix m1 */
void transpose ( int mat[3][3], int m[3][3] )
{
        int i, j ;
        for ( i = 0 ; i < MAX ; i++ )
        {
                for ( j = 0 ; j < MAX ; j++ )
                        m[i][j] = mat[j][i] ;
        }
}

/* finds the determinant value for given matrix */
int determinant( int mat[3][3] )
{
        int sum, i, j, k, p ;
        sum = 0 ; j = 1 ; k = MAX - 1 ;
```

```
        for ( i = 0 ; i < MAX ; i++ )
        {
                p = pow ( -1, i ) ;

                if ( i == MAX - 1 )
                        k = 1 ;
                sum = sum + ( mat[0][i] * ( mat[1][j] *
                                                        mat[2][k] - mat[2][j] *
                                                        mat[1][k] ) ) * p ;

                j = 0 ;
        }

        return sum ;
}

/* checks if given matrix is an orthogonal matrix */
int isortho ( int mat[3][3] )
{
        /* transpose the matrix */
        int m1[3][3], m2[3][3], i ;
        transpose ( mat, m1 ) ;

        /* multiply the matrix with its transpose */
        matmul ( mat, m1, m2 ) ;

        /* check for the identity matrix */
        for ( i = 0  ; i < MAX ; i++ )
        {
                if ( m2[i][i] == 1 )
                        continue ;
                else
                        break ;
        }
        if ( i == MAX )
                return 1 ;
        else
                return 0 ;
}
```

O/P:- Enter elements for array:

```
Enter the element: 5
Enter the element: 2
Enter the element: 6
Enter the element: 7
Enter the element: 1
```

```
Enter the element: 8
Enter the element: 2
Enter the element: 4
Enter the element: 3

The Matrix:
5      2      6
7      1      8
2      4      3

The determinant for given matrix: 1.

Matrix  is not singular.

Matrix is not orthogonal.
```

Program 7: matrix operations like addition, multiplicaton, etc. on

```
#include <stdio.h>
#include <conio.h>

#define MAX 3

void create ( int [3][3] ) ;
void display ( int [3][3] ) ;
void matadd ( int [3][3], int [3][3], int [3][3] ) ;
void matmul ( int [3][3], int [3][3], int [3][3] ) ;
void transpose ( int [3][3], int [3][3] ) ;

void main( )
{
        int mat1[3][3], mat2[3][3], mat3[3][3], mat4[3][3], mat5[3][3] ;

        clrscr( ) ;

        printf ( "\nEnter elements for first array: \n\n" ) ;
        create ( mat1 ) ;

        printf ( "\nEnter elements for second array: \n\n" ) ;
        create ( mat2 ) ;

        printf ( "\nFirst Array: \n" ) ;
        display ( mat1 ) ;
        printf ( "\nSecond Array:\n" ) ;
        display ( mat2 ) ;
```

```
        matadd ( mat1, mat2, mat3 ) ;
        printf ( "\nAfter Addition: \n" ) ;
        display ( mat3 ) ;

        matmul ( mat1, mat2, mat4 ) ;
        printf ( "\nAfter Multiplication: \n" ) ;
        display ( mat4 ) ;

        transpose ( mat1, mat5 ) ;
        printf ( "\nTranspose of first matrix: \n" ) ;
        display ( mat5 ) ;

        getch( ) ;
}

/* creates matrix mat */
void create ( int mat[3][3] )
{
        int i, j ;

        for ( i = 0 ; i < MAX ; i++ )
        {
                for ( j = 0 ; j < MAX ; j++ )
                {
                        printf ( "Enter the element: " ) ;
                        scanf ( "%d", &mat[i][j] ) ;
                }
        }
}

/* displays the contents of matrix */
void display ( int mat[3][3] )
{
        int i, j ;

        for ( i = 0 ; i < MAX ; i++ )
        {
                for ( j = 0 ; j < MAX ; j++ )
                        printf ( "%d\t", mat[i][j] ) ;
                printf ( "\n" ) ;
        }
}

/* adds two matrices m1 and m2 */
void matadd ( int m1[3][3], int m2[3][3], int m3[3][3] )
{
```

```
        int i, j ;

        for ( i = 0 ; i < MAX ; i++ )
        {
                for ( j = 0 ; j < MAX ; j++ )
                        m3[i][j] = m1[i][j] + m2[i][j] ;
        }
}

/* multiplies two matrices m1 and m2 */
void matmul ( int m1[3][3], int m2[3][3], int m3[3][3] )
{
        int i, j, k ;
        for ( k = 0 ; k < MAX ; k++ )
        {
                for ( i = 0 ; i < MAX ; i++ )
                {
                        m3[k][i] = 0 ;
                        for ( j = 0 ; j < MAX ; j++ )
                                m3[k][i] += m1[k][j] * m2[j][i] ;
                }
        }
}

/* obtains transpose of matrix m1 */
void transpose ( int m1[3][3], int m2[3][3] )
{
        int i, j ;

        for ( i = 0 ; i < MAX ; i++ )
        {
                for ( j = 0 ; j < MAX ; j++ )
                        m2[i][j] = m1[j][i] ;
        }
}
```

O/P:-

```
First Array:
1       2       3
4       5       6
7       8       9

Second Array:
9       8       7
6       5       4
3       2       1
```

```
After Addition:
10      10      10
10      10      10
10      10      10

After Multiplication:
30      24      18
84      69      54
138     114     90

Transpose of first matrix:
1       4       7
2       5       8
3       6       9
```

Program 8 : to merge two 1-D arrays after sorting them

```
#include <stdio.h>
#include <conio.h>
#include <alloc.h>

#define MAX1 5
#define MAX2 7

int *arr ;

int* create ( int ) ;
void sort ( int *, int ) ;
void display ( int *, int ) ;
int* merge ( int *, int * ) ;
void main( )
{
        int *a, *b, *c ;

        clrscr( ) ;

        printf (  "\nEnter elements for first array: \n\n" ) ;
        a = create ( MAX1 ) ;

        printf ( "\nEnter elements for second array: \n\n" ) ;
        b = create ( MAX2 ) ;

        sort ( a, MAX1 ) ;
```

```
        sort ( b, MAX2 ) ;

        printf ( "\nFirst array: \n" ) ;
        display ( a, MAX1 ) ;
        printf ( "\n\nSecond array: \n" ) ;
        display ( b, MAX2 ) ;
        printf ( "\n\nAfter Merging: \n" ) ;

        c = merge ( a, b ) ;
        display ( c, MAX1 + MAX2 ) ;

        getch( ) ;
}

/* creates array of given size, dynamically */
int* create ( int size )
{
        int *arr, i ;
        arr = ( int * ) malloc ( sizeof ( int ) * size ) ;

        for ( i = 0 ; i < size ; i++ )
        {
                printf ( "Enter the element no. %d: ", i + 1 ) ;
                scanf ( "%d", &arr[i] ) ;
        }
        return arr ;
}

/* sorts array in ascending order */
void sort ( int *arr, int size )
{
        int i, temp, j ;
        for ( i = 0 ; i < size ; i++ )
        {
                for ( j = i + 1 ; j < size ; j++ )
                {
                        if ( arr[i] > arr[j] )
                        {
                                temp = arr[i] ;
                                arr[i] = arr[j] ;
                                arr[j] = temp ;
                        }
                }
        }
}
```

```
/* displays the contents of array */
void display ( int *arr, int size )
{
        int i ;
        for ( i = 0 ; i < size ; i++)
                printf (  "%d\t", arr[i] ) ;
}

/* merges two arrays of different size */
int* merge ( int *a, int *b )
{
        int *arr ;
        int i, k, j ;
        int size = MAX1 + MAX2 ;
        arr = ( int * ) malloc ( sizeof ( int ) * ( size ) ) ;

        for ( k = 0, j = 0, i = 0 ; i <= size ; i++ )
        {
                if ( a[k] < b[j] )
                {
                        arr[i] = a[k] ;
                        k++ ;
                        if ( k >= MAX1 )
                        {
                                for ( i++ ; j < MAX2 ; j++, i++ )
                                        arr[i] = b[j] ;
                        }
                }
                else
                {
                        arr[i] = b[j] ;
                        j++ ;
                        if ( j >= MAX2 )
                        {
                                for ( i++ ; k < MAX1 ; k++, i++ )
                                        arr[i] = a[k] ;
                        }
                }
        }

        return arr ;
}
```

O/P:-Enter the element no. 1: 2
Enter the element no. 2: 3
Enter the element no. 3: 4
Enter the element no. 4: 5

```
Enter the element no. 5: 6

Enter elements for second array:

Enter the element no. 1: 7
Enter the element no. 2: 8
Enter the element no. 3: 9
Enter the element no. 4: 0
Enter the element no. 5: 1
Enter the element no. 6: 1
Enter the element no. 7: 2

First array:
2      3      4      5      6

Second array:
0      1      1      2      7      8      9

After Merging:
0      1      1      2      2      3      4      5      6      7
8      9
```

Program 9: to implement an array(insertion,deletion,reversing the entire array and finding a element in an array).

```
#include <stdio.h>
#include <conio.h>

#define MAX 5

void insert ( int *, int pos, int num ) ;
void del ( int *, int pos ) ;
void reverse ( int * ) ;
void display ( int * ) ;
void search ( int *, int num ) ;

void main( )
{
        int arr[5] ;

        clrscr( ) ;

        insert ( arr, 1, 11 ) ;
        insert ( arr, 2, 12 ) ;
        insert ( arr, 3, 13 ) ;
        insert ( arr, 4, 14 ) ;
```

```
        insert ( arr, 5, 15 ) ;

        printf ( "\nElements of Array: " ) ;
        display ( arr ) ;

        del ( arr, 5 ) ;
        del ( arr, 2 ) ;
        printf ( "\n\nAfter deletion: " ) ;
        display ( arr ) ;

        insert ( arr, 2, 222 ) ;
        insert ( arr, 5, 555 ) ;
        printf ( "\n\nAfter insertion: " ) ;
        display ( arr ) ;
        reverse ( arr ) ;
        printf ( "\n\nAfter reversing: " ) ;
        display ( arr ) ;
        search ( arr, 222 ) ;
        search ( arr, 666 ) ;

        getch( ) ;
}

/* inserts an element num at given position pos */
void insert (int *arr, int pos, int num )
{
        /* shift elements to right */
        int i ;
        for ( i = MAX - 1 ; i >= pos ; i-- )
                arr[i] = arr[i - 1] ;
        arr[i] = num ;
}

/* deletes an element from the given position pos */
void del ( int *arr, int pos )
{
        /* skip to the desired position */
        int i ;
        for ( i = pos ; i < MAX ; i++ )
                arr[i - 1] = arr[i] ;
        arr[i - 1] = 0 ;
}

/* reverses the entire array */
void reverse ( int *arr )
{
```

```
        int i ;
        for ( i = 0 ; i < MAX / 2 ; i++ )
        {
                int temp = arr[i] ;
                arr[i] = arr[MAX - 1 - i] ;
                arr[MAX - 1 - i] = temp ;
        }
}

/* searches array for a given element num */
void search ( int *arr, int num )
{
        /* Traverse the array */
        int i ;
        for ( i = 0 ; i < MAX ; i++ )
        {
                if ( arr[i] == num )
                {
                        printf ( "\n\nThe element %d is present at %dth position.", num,
                                        i + 1 ) ;
                        return ;
                }
        }
        if ( i == MAX )
                printf ( "\n\nThe element %d is not present in the array.", num ) ;
}

/* displays the contents of a array */
void display ( int *arr )
{
        /* traverse the entire array */
        int i ;
        printf ( "\n" ) ;
        for ( i = 0 ; i < MAX ; i++ )
                printf ( "%d\t", arr[i] ) ;
}
```

O/P:-Elements of Array:
11 12 13 14 15
After deletion:
11 13 14 0 0
After insertion:
11 222 13 14 555
After reversing:
555 14 13 222 11
The element 222 is present at 4th position.
The element 666 is not present in the array.

Dyanmic memory

/* Example of brk()
Changes data-segment space allocation

Declaration:
- int brk(void *addr);
- void *sbrk(int incr);

Remarks:
brk dynamically changes the amount of space allocated to the calling program's heap by resetting the program's break value to addr.

sbrk changes the allocated space by adding incr bytes to the break value.

The amount of allocated space increases as the break value increases.

With sbrk, incr can be negative, which decreases the amount of allocated space.

brk and sbrk will fail without making any change in the allocated space if such a change would allocate more space than is allowable.

Return Value:
- On success,
  - brk returns 0
  - sbrk returns the old break value
- On error, both functions return -1 and set errno to ENOMEM (not enough memory).

```
#include <stdio.h>
#include <alloc.h>

int main(void)
{
  char *ptr;

  printf("Changing allocation with brk()\n");
  ptr = (char *) malloc(1);
  printf("Before brk() call: %lu bytes free\n", coreleft());
  brk(ptr+1000);
  printf(" After brk() call: %lu bytes free\n", coreleft());
  return 0;
```

```
} */


/* Example allocmem
 Allocates DOS memory segment

 Declaration:
  þ int allocmem(unsigned size, unsigned *segp);
  þ unsigned _dos_allocmem(unsigned size, unsigned *segp);

 Remarks:
allocmem and _dos_allocmem use the DOS system call 0x48 to allocate a block
of free memory and return the segment address of the allocated block.

  Arg. ³ What It Is
ÍÍÍÍÍÍØÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍÍ
  size ³ The number of 16-byte paragraphs requested
  segp ³ Pointer to a word that will be assigned the
       ³ segment address of the newly allocated block

If not enough room is available,
  þ allocmem makes no assignment to the word *segp
  þ _dos_allocmem stores the size of the largest
    available block in the word *segp.

All allocated blocks are paragraph-aligned.

þ NOTE: malloc can't coexist with either allocmem or _dos_allocmem.

 Return Value:
  þ On success,
     þ allocmem returns -1
     þ _dos_allocmem returns 0
  þ On error,
     þ allocmem returns the size of the largest available block and
       sets both _doserrno and errno to ENOMEM  (Not enough memory)
     þ _dos_allocmem returns the DOS error code and sets errno to ENOMEM

#include <dos.h>
#include <alloc.h>
#include <stdio.h>

int main(void)
{
  unsigned int size, segp;
  int stat;
```

```
  size = 64;                  /* (64 x 16) = 1024 bytes */
  stat = allocmem(size, &segp);
  if (stat == -1)
    printf("Allocated memory at segment: %x\n", segp);
  else
    printf("Failed: maximum number of paragraphs available is %u\n", stat);

  return 0;
} */

/* calloc example
 Allocates main memory

 Declaration:  void *calloc(size_t nitems, size_t size);

 Remarks:
calloc provides access to the C memory heap, which is available for dynamic
allocation of variable-sized blocks of memory.

Many data structures, such as trees and lists, naturally employ heap memory
allocation.

calloc allocates a block (nitems * size) bytes and clears it to 0. To
allocate a block larger than 64K, use farcalloc.

 Small Data Models
 ßßßßßßßßßßßßßßßßß
All the space between the end of the data segment and the top of the program
stack is available for use in the tiny, small, and medium models, except for
a small margin immediately before the top of the stack.

This margin allows room for the application to grow on the stack, plus a
small amount needed by DOS.

 Large Data Models
 ßßßßßßßßßßßßßßßßß
In the compact, large, and huge models, all space beyond the program stack
to the end of physical memory is available for the heap.

 Return Value:
  þ On success, returns a pointer to the newly allocated block.
  þ On failure (not enough space exists for the new block, or nitems or
    size is 0), returns null.
```

```
#include <stdio.h>
#include <alloc.h>
#include <string.h>

int main(void)
{
  char *str = NULL;

  /* allocate memory for string */
  str = (char *) calloc(10, sizeof(char));

  /* copy "Hello" into string */
  strcpy(str, "Hello");

  /* display string */
  printf("String is %s\n", str);

  /* free memory */
  free(str);

  return 0;
}
*/

/* coreleft example
```

þ coreleft returns a measure of unused memory
þ farcoreleft returns a measure of unused memory in the far heap

Declaration:
þ Tiny, small, and medium models: unsigned coreleft(void);
þ Compact, large, and huge models: unsigned long coreleft(void);
þ All except tiny models: unsigned long farcoreleft(void);

Remarks:
coreleft returns a measure of RAM memory not in use.

The value coreleft gives depends on whether the memory model is of the small data group or the large data group.

farcoreleft returns a measure of the amount of unused memory in the far heap beyond the highest allocated block.

A tiny model program can't use farcoreleft.

Return Value:

- coreleft:
  - Small data models: returns the amount of unused memory between the top of the heap and the stack.
  - Large data models: returns the amount of memory between the highest allocated block and the end of available memory.
- farcoreleft: returns the total amount of space left in the far heap, between the highest allocated block and the end of available memory.

```
#include <stdio.h>
#include <alloc.h>

int main(void)
{
  printf("The difference between the highest allocated block and\n");
  printf("the top of the heap is: %lu bytes\n", (unsigned long) coreleft());
  return 0;
}
*/
```

malloc

Allocates memory

Declaration: void *malloc(size_t size);

Remarks:
malloc allocates a block of size bytes from the memory heap. It allows a program to allocate memory explicitly as it's needed, and in the exact amounts needed.

The heap is used for dynamic allocation of variable-sized blocks of memory. Many data structures, such as trees and lists, naturally employ heap memory allocation.

All the space between the end of the data segment and the top of the program stack is available for use in the small data models, except for a small margin immediately before the top of the stack.

This margin is intended to allow the application some room to make the stack larger, in addition to a small amount needed by DOS.

In the large data models, all the space beyond the program stack to the end of available memory is available for the heap.

Return Value:

- On success, malloc returns a pointer to the newly allocated block of memory.
- On error (if not enough space exists for the new block), malloc returns null. The contents of the block are left unchanged.
- If the argument size == 0, malloc returns null.

```
#include <stdio.h>
#include <string.h>
#include <alloc.h>
#include <process.h>

int main(void)
{
  char *str;

  /* allocate memory for string */
  if ((str = (char *) malloc(10)) == NULL)
  {
    printf("Not enough memory to allocate buffer\n");
    exit(1);  /* terminate program if out of memory */
  }

  /* copy "Hello" into string */
  strcpy(str, "Hello");

  /* display string */
  printf("String is %s\n", str);

  /* free memory */
  free(str);

  return 0;
}
```

_dos_freemem

Frees a previously allocated DOS memory block

Declaration:
- unsigned _dos_freemem(unsigned segx);
- int freemem(unsigned segx);

Remarks:
- freemem frees a memory block allocated by a previous call to allocmem.

- _dos_freemem frees a memory block allocated by a previous call to _dos_allocmem.

segx is the segment address of the block.

Return Value:
- On success, both functions return 0
- On error,
  - _dos_freemem returns the DOS error code and sets errno to ENOMEM (Bad memory pointer)
  - freemem returns -1 and sets errno to ENOMEM (Insufficient memory)

```
#include <dos.h>
#include <stdio.h>

int main(void)
{
  unsigned int size, segp, err, maxb;
  size = 64; /* (64 x 16) = 1024 bytes */
  err = _dos_allocmem(size, &segp);
  if (err == 0)
    printf("Allocated memory at segment: %x\n", segp);
  else {
    perror("Unable to allocate block");
    printf("Maximum no. of paragraphs available is %u\n", segp);
    return 1;
  }
  if (_dos_setblock(size * 2, segp, &maxb) == 0)
    printf("Expanded memory block at segment: %X\n", segp);
  else {
    perror("Unable to expand block");
    printf("Maximum no. of paragraphs available is %u\n", maxb);
  }
  _dos_freemem(segp);
  return 0;
}
```

_dos_allocmem()

Allocates DOS memory segment

Declaration:
- int allocmem(unsigned size, unsigned *segp);
- unsigned _dos_allocmem(unsigned size, unsigned *segp);

Remarks:
allocmem and _dos_allocmem use the DOS system call 0x48 to allocate a block of free memory and return the segment address of the allocated block.

Arg.	What It Is
size	The number of 16-byte paragraphs requested
segp	Pointer to a word that will be assigned the segment address of the newly allocated block

If not enough room is available,
- þ allocmem makes no assignment to the word *segp
- þ _dos_allocmem stores the size of the largest available block in the word *segp.

All allocated blocks are paragraph-aligned.

þ NOTE: malloc can't coexist with either allocmem or _dos_allocmem.

Return Value:
- þ On success,
  - þ allocmem returns -1
  - þ _dos_allocmem returns 0
- þ On error,
  - þ allocmem returns the size of the largest available block and sets both _doserrno and errno to ENOMEM (Not enough memory)
  - þ _dos_allocmem returns the DOS error code and sets errno to ENOMEM

```
#include <dos.h>
#include <stdio.h>

int main(void)
{
  unsigned int size, segp, err, maxb;
  size = 64; /* (64 x 16) = 1024 bytes */
  err = _dos_allocmem(size, &segp);
  if (err == 0)
    printf("Allocated memory at segment: %x\n", segp);
  else {
    perror("Unable to allocate block");
    printf("Maximum no. of paragraphs available is %u\n", segp);
    return 1;
  }
  if (_dos_setblock(size * 2, segp, &maxb) == 0)
    printf("Expanded memory block at segment: %X\n", segp);
  else {
```

```
    perror("Unable to expand block");
    printf("Maximum no. of paragraphs available is %u\n", maxb);
  }
  _dos_freemem(segp);
  return 0;
}
```

_dos_setblock

Modifies the size of a previously allocated block

Declaration:
- unsigned _dos_setblock(unsigned newsize, unsigned segx, unsigned *maxp);
- int setblock(unsigned segx, unsigned newsize);

Remarks:
_dos_setblock and setblock modify the size of a memory segment.

Argument	What It Is/Does
segx	Segment address returned by a previous call to allocmem
newsize	New, requested size in paragraphs
maxp	Points to location where the size of the largest possible segment is stored (if the segment can't be changed to the new size)

Return Value:
- On success,
  - _dos_setblock returns 0
  - setblock returns -1
- On error,
  - _dos_setblock returns the DOS error code and sets errno to ENOMEM Not enough memory or bad segment address
  - setblock returns the size of the largest possible block (in paragraphs), and sets _doserrno.

```
#include <dos.h>
#include <stdio.h>

int main(void)
{
  unsigned int size, segp, err, maxb;
  size = 64; /* (64 x 16) = 1024 bytes */
  err = _dos_allocmem(size, &segp);
  if (err == 0)
    printf("Allocated memory at segment: %x\n", segp);
```

```
  else {
    perror("Unable to allocate block");
    printf("Maximum no. of paragraphs available is %u\n", segp);
    return 1;
  }
  if (_dos_setblock(size * 2, segp, &maxb) == 0)
    printf("Expanded memory block at segment: %X\n", segp);
  else {
    perror("Unable to expand block");
    printf("Maximum no. of paragraphs available is %u\n", maxb);
  }
  _dos_freemem(segp);
  return 0;
}
```

farcoreleft

- coreleft returns a measure of unused memory
- farcoreleft returns a measure of unused memory in the far heap

Declaration:

- Tiny, small, and medium models: unsigned coreleft(void);
- Compact, large, and huge models: unsigned long coreleft(void);
- All except tiny models: unsigned long farcoreleft(void);

Remarks:

coreleft returns a measure of RAM memory not in use.

The value coreleft gives depends on whether the memory model is of the small data group or the large data group.

farcoreleft returns a measure of the amount of unused memory in the far heap beyond the highest allocated block.

A tiny model program can't use farcoreleft.

Return Value:

- coreleft:
  - Small data models: returns the amount of unused memory between the top of the heap and the stack.
  - Large data models: returns the amount of memory between the highest allocated block and the end of available memory.
- farcoreleft: returns the total amount of space left in the far heap, between the highest allocated block and the end of available memory.

```
#include <stdio.h>
#include <alloc.h>
```

```
int main(void)
{
 printf("The difference between the highest allocated block in the far\n");
 printf("heap and the top of the far heap is: %lu bytes\n", farcoreleft());

   return 0;
}
```

heapwalk

- heapwalk walks through the heap node by node
- farheapwalk walks through the far heap node by node

Declaration:

- int heapwalk(struct heapinfo *hi);
- int farheapwalk(struct farheapinfo *hi);

Remarks:

heapwalk receives a pointer to a structure of type heapinfo.

farheapwalk receives a pointer to a structure of type farheapinfo.

For the first call to heapwalk or farheapwalk, set the hi.ptr field to null.

- hi.ptr: Both functions return with hi.ptr containing the address of the first block.
- hi.size: Holds the size of the block in bytes.
- hi.in_use: A flag that's set if the block is currently in use.

Both functions assume the heap is correct.

- Use heapcheck to verify the heap before using heapwalk.
- Use farheapcheck to verify the far heap before using farheapwalk.

_HEAPOK is returned with the last block on the heap; _HEAPEND is returned on the next call.

Return Value:

- On success, both return a value greater than 0
  - _HEAPEMPTY (= 1) No heap
  - _HEAPOK (= 2) Heap is verified (heapinfo or farheapinfo block contains valid data)
  - _HEAPEND (= 5) End of the heap has been reached
- On error, both return a value less than 0

```
#include <stdio.h>
#include <alloc.h>
```

```
#define NUM_PTRS  10
#define NUM_BYTES 16

int main( void )
{
  struct heapinfo hi;
  char *array[ NUM_PTRS ];
  int i;

  for( i = 0; i < NUM_PTRS; i++ )
    array[ i ] = (char *) malloc( NUM_BYTES );

  for( i = 0; i < NUM_PTRS; i += 2 )
    free( array[ i ] );

  hi.ptr = NULL;
  printf( "   Size   Status\n" );
  printf( "   ----   ------\n" );
  while( heapwalk( &hi ) == _HEAPOK )
  printf( "%7u    %s\n", hi.size, hi.in_use ? "used" : "free" );

  return 0;
}
```

File handling

Program 1

```
#include<stdio.h>
void main(int argc, char *argv[])
{
        FILE *fp;
        char string[81];

        if (argc != 2)
        {
                printf("Format : File9 filename");
                exit(1);
        }

        if ( (fp = fopen(argv[1], "r"))  == NULL)
        {
                printf("Cannot open file %s", argv[1]);
                exit(1);
        }
        while ( fgets(string, 80, fp) != NULL)
        {
                fputs(string, stdprn);
                putc('\r', stdprn);
        }
        fclose(fp);
}
```

Program 2

```
#include<stdio.h>
#include<stdlib.h>
int main(int argc, char *argv[])
{
        FILE *fp;
        char string[81];

        if (argc != 2)
        {
                printf("Format : file8 filename");
                exit(1);
        }

        if ( (fp=fopen(argv[1], "r")) == NULL)
        {
```

```
                printf("Cannot open file %s ", argv[1]);
                exit(1);
        }
        while (fgets(string,80,fp) != NULL)
        printf("%s", string);
        fclose(fp);
        return(0);
}
```

Program 3

```
#include<stdio.h>
#include<stdlib.h>
void main()
{
        FILE *fp;
        char string[81];
        fp = fopen("textfile.txt", "r");

        while(fgets(string, 80, fp) != NULL)
        {
                printf("%s", string);
        }
        fclose(fp);
}
```

Program 4

```
#include<stdio.h>
#include<stdlib.h>
void main()
{
        FILE *fp;
        char string[81];
        fp = fopen("textfile.txt", "w");

        while(strlen(gets(string) ) > 0)
        {
                fputs(string, fp);
                fputs("\n", fp);
        }
        fclose(fp);
}
```

Program 5

```
#include<stdio.h>
#include<stdlib.h>
void main(int argc, char *argv[])
{
        FILE *fp;
        char ch, string[81];
        int white = 1;
        int count = 0;

        if (argc != 2)
        {
                printf("Format : file5 <filename>");
                exit(1);
        }

        if ( (fp = fopen(argv[1], "r")) == NULL)
        {
                printf("Cannot open file %s", argv[1]);
                exit(1);
        }

        while ((ch=getc(fp)) != EOF)
        {
                switch(ch)
                {
                        case ' ':
                        case '\t':
                        case '\n':
                                white++;
                                break;
                        default :
                        if (white)
                        {
                                white = 0;
                                count ++;
                        }
                }
        }
                fclose(fp);
                printf("File %s contains %d words", argv[1], count);
}
```

Program 7

```
#include<stdio.h>
#include<stdlib.h>
void main(int argc, char *argv[])
{
        FILE *fp;
        char string[81];
        int count = 0;

        if (argc != 2)
        {
                printf("Format : file4 <filename>");
                exit(1);
        }

        if ( (fp = fopen(argv[1], "r")) == NULL)
        {
                printf("Cannot open file %s", argv[1]);
                exit(1);
        }

        while ((getc(fp)) != EOF)
                count ++;
        fclose(fp);

        printf("File %s contains %d characters.", argv[1], count);
}
```

Program 8

```
#include<stdio.h>
#include<stdlib.h>
void main()
{
        FILE *fp;
        char ch;

        if ( (fp = fopen("textfile.txt", "r")) == NULL)
        {
                printf("Cannot open file textfile.txt");
                exit(1);
        }

        while ( (ch = getc(fp)) != EOF)
```

```
                printf("%c", ch);

        fclose(fp);
}
```

Program 9

```
#include<stdio.h>
void main()
{
        FILE *fp;
        char ch;

        fp = fopen("textfile.txt", "r");

        while ( (ch = getc(fp)) != EOF)
                printf("%c", ch);

        fclose(fp);
}
```

Program 10

```
#include<stdio.h>
#include<stdlib.h>
void main()
{
        struct student
        {
                int roll;
                char name[40];
        } stu;

        char numstr[81];

        FILE *fp;
        int recno;
        long int offset;

        if ( (fp=fopen("agents.rec", "r")) == NULL)
        {
                printf("Cannot open agents.rec");
                exit(1);
        }

        printf("Enter record number : ");
```

```
        scanf("%d", &recno);

        offset = (recno - 1) * sizeof(stu);

        if (fseek(fp, offset, 0) != 0)
        {
                printf("Cannot move pointer ");
                exit(1);
        }

        fread(&stu, sizeof(stu), 1, fp);
        printf("Roll Number %d\n", stu.roll);
        printf("Name %s\n", stu.name);
        fclose(fp);
}
```

Program 11

```
#include<stdio.h>
#include<stdlib.h>
void main()
{
        struct student
        {
                int roll;
                char name[40];
        } stu;

        char numstr[81];

        FILE *fp;

        if ( (fp=fopen("agents.rec", "rb")) == NULL)
        {
                printf("Cannot open agents.rec");
                exit(1);
        }

        while (fread(&stu, sizeof(stu), 1, fp) == 1)
        {
                printf("\nRoll number : %03d",  stu.roll);
                printf("\nName : %s" , stu.name);
        }
        fclose(fp);
}
```

Program 12

```
#include<stdio.h>
#include<stdlib.h>
void main()
{
        struct student
        {
                int roll;
                char name[40];
        } stu;

        char numstr[81];

        FILE *fp;

        if ( (fp=fopen("agents.rec", "rb")) == NULL)
        {
                printf("Cannot open agents.rec");
                exit(1);
        }

        while (fread(&stu, sizeof(stu), 1, fp) == 1)
        {
                printf("\nRoll number : %03d",  stu.roll);
                printf("\nName : %s" , stu.name);
        }
        fclose(fp);
}
```

Program 13

```
#include<stdio.h>
#include<stdlib.h>
void main()
{
        struct student
        {
                int roll;
                char name[40];
        } stu;


        FILE *fp;

        if ( (fp=fopen("agents.rec", "wb")) == NULL)
        {
```

```
                printf("Cannot open agents.rec");
                exit(1);
        }
        do
        {
                printf("\nEnter roll number : ");
                scanf("%d", &stu.roll);

                printf("\nEnter name : ");
                scanf("%s", stu.name);

                fwrite(&stu, sizeof(stu), 1, fp);
                printf("Wish to add another student record ");
        } while (getche() == 'y');
        fclose(fp);
}
```

Program 14

```
#include<stdio.h>
#include<stdlib.h>
void main()
{
        int table[10] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

        FILE *fp;

        if ( (fp=fopen("table.rec", "w")) == NULL)
        {
                printf("Cannot open agents.rec");
                exit(1);
        }

        fwrite(table, sizeof(table), 1, fp);

        fclose(fp);
}
```

Program 15

```
#include<stdio.h>
#include<stdlib.h>
void main()
{
        FILE *fp;
        char name[40];
```

```
        int rollno;

        char flag = 'y';

        fp = fopen("textfile.txt", "w");
        do
        {
                printf("Type Roll Number, Name ");
                scanf("%d %s",  &rollno, name);
                fprintf(fp, " %s:%d\n", name, rollno);
                fflush(stdin);
                printf("Wish to continue");
                scanf("%c", &flag);
                clrscr();
        }
        while (flag == 'y');

        fclose(fp);
}
```

Program 16

```
#include<stdio.h>
void main(int argc, char *argv[])
{
        FILE *fp1, *fp2;
        char string[81];

        if (argc != 2)
        {
                printf("Format : File10 filename");
                exit(1);
        }

        if ( (fp1 = fopen(argv[1], "r"))  == NULL)
        {
                printf("Cannot open file %s", argv[1]);
                exit(1);
        }
        if ( (fp2 = fopen("PRN", "w") ) == NULL)
        {
                printf("Cannot access printer");
                exit(1);
        }
        while ( fgets(string, 80, fp1) != NULL)
        {
```

```
                fputs(string, fp2);
        }
        fclose(fp1);
        fclose(fp2);
}
```

Program 17

```
#include<stdio.h>
void main()
{
        FILE *fp;
        char ch;

        fp = fopen("textfile.txt", "w");

        while ( (ch = getche()) != '\r')
                putc(ch, fp);

        fclose(fp);
}
```

LINKED LIST

Program 1: To add char to a list

```
#include<stdio.h>
#include<stdlib.h>
#define TRUE 1
void newname(void);
void listall(void);

struct prs
{
        char name[1];
        struct prs *ptrnext;
};
struct prs *ptrfirst, *ptrcurr, *ptrnew ;

void main()
{
        char ch;
        clrscr();
        ptrfirst = (struct prs *) NULL;
        while (TRUE)
        {
          printf("\n Type 'e' to enter new name");
          printf("\n 'l' to list all name,");
          printf("\n 'q' to quit : ");
          ch = getche();

          switch(ch)
          {
                case 'e' :
                        newname(); break;
                case 'l' :
                        listall(); break;
                case 'q' :
                        exit(0);
                default :
                        puts("\nEnter only selections listed");
          }
        }
}

void newname()
{
        char numstr[81];
```

```
        ptrnew = (struct prs *) malloc (sizeof(struct prs) );

        if (ptrfirst == (struct prs *) NULL)
           ptrfirst = ptrcurr = ptrnew;
        else
        {
           ptrcurr = ptrfirst;
           while(ptrcurr->ptrnext != (struct prs *) NULL)
                 {
            ptrcurr = ptrcurr->ptrnext;
                 }
           ptrcurr->ptrnext = ptrnew;
           ptrcurr = ptrnew;
        }
        printf("\nEnter name : ");
     gets(ptrcurr->name);
     ptrcurr->ptrnext = (struct prs *) NULL;
}
void listall()
{
        if (ptrfirst == (struct prs *) NULL)
        {
                printf("\n Empty List\n");
                return;
        }
     ptrcurr = ptrfirst;
        do
        {
           printf("\nName %s", ptrcurr->name);
           ptrcurr = ptrcurr->ptrnext;
     } while (ptrcurr != (struct prs *) NULL);
}
```

Program 2: list to implement adding,deleting,listing,searching and inserting of names

```
/* linked list example */
#include <stdio.h>
#include <alloc.h>
#include <stdlib.h>
#include <conio.h>
#include <ctype.h>
#include <string.h>

/* function prototypes */
```

```
struct node * initnode( char *, int );
void printnode( struct node * );
void printlist( struct node * );
void add( struct node * );
struct node * searchname( struct node *, char * );
void deletenode( struct node * );
void insertnode( struct node * );
void deletelist( struct node * );

/* definition of a data node for holding student information */
struct node {
  char name[20];
  int  id;
  struct node *next;
};

/* head points to first node in list, end points to last node in list */
/* initialise both to NULL, meaning no nodes in list yet */
struct node *head = (struct node *) NULL;
struct node *end = (struct node *) NULL;

/* this initialises a node, allocates memory for the node, and returns   */
/* a pointer to the new node. Must pass it the node details, name and id */
struct node * initnode( char *name, int id )
{
  struct node *ptr;
  ptr = (struct node *) calloc( 1, sizeof(struct node ) );
  if( ptr == NULL )                    /* error allocating node?      */
     return (struct node *) NULL;       /* then return NULL, else      */
  else {                              /* allocated node successfully */
     strcpy( ptr->name, name );         /* fill in name details       */
     ptr->id = id;                     /* copy id details            */
     return ptr;                        /* return pointer to new node  */
  }
}

/* this prints the details of a node, eg, the name and id                */
/* must pass it the address of the node you want to print out            */
void printnode( struct node *ptr )
{
  printf("Name ->%s\n", ptr->name );
  printf("ID   ->%d\n", ptr->id );
}

/* this prints all nodes from the current address passed to it. If you    */
/* pass it 'head', then it prints out the entire list, by cycling through */
```

```
/* each node and calling 'printnode' to print each node found          */
void printlist( struct node *ptr )
{
  while( ptr != NULL )          /* continue whilst there are nodes left */
  {
    printnode( ptr );          /* print out the current node          */
    ptr = ptr->next;           /* goto the next node in the list      */
  }
}

/* this adds a node to the end of the list. You must allocate a node and  */
/* then pass its address to this function                                 */
void add( struct node *new )  /* adding to end of list */
{
  if( head == NULL )      /* if there are no nodes in list, then        */
     head = new;         /* set head to this new node                  */
  end->next = new;        /* link in the new node to the end of the list */
  new->next = NULL;       /* set next field to signify the end of list   */
  end = new;             /* adjust end to point to the last node        */
}

/* search the list for a name, and return a pointer to the found node     */
/* accepts a name to search for, and a pointer from which to start. If    */
/* you pass the pointer as 'head', it searches from the start of the list */
struct node * searchname( struct node *ptr, char *name )
{
   while( strcmp( name, ptr->name ) != 0 ) {    /* whilst name not found */
     ptr = ptr->next;                        /* goto the next node    */
     if( ptr == NULL )                        /* stop if we are at the */
       break;                               /* of the list          */
   }
   return ptr;                               /* return a pointer to   */
}                                          /* found node or NULL    */

/* deletes the specified node pointed to by 'ptr' from the list          */
void deletenode( struct node *ptr )
{
  struct node *temp, *prev;
  temp = ptr;    /* node to be deleted */
  prev = head;   /* start of the list, will cycle to node before temp    */

  if( temp == prev ) {                  /* are we deleting first node  */
     head = head->next;                  /* moves head to next node     */
     if( end == temp )                  /* is it end, only one node?   */
       end = end->next;                  /* adjust end as well         */
     free( temp );                     /* free space occupied by node */
```

```
  }
  else {                              /* if not the first node, then */
     while( prev->next != temp ) {       /* move prev to the node before*/
        prev = prev->next;             /* the one to be deleted       */
     }
     prev->next = temp->next;           /* link previous node to next  */
     if( end == temp )                 /* if this was the end node,   */
        end = prev;                    /* then reset the end pointer  */
     free( temp );                     /* free space occupied by node */
  }
}

/* inserts a new node, uses name field to align node as alphabetical list */
/* pass it the address of the new node to be inserted, with details all   */
/* filled in                                                              */
void insertnode( struct node *new )
{
  struct node *temp, *prev;              /* similar to deletenode      */

  if( head == NULL ) {                   /* if an empty list,          */
     head = new;                         /* set 'head' to it           */
     end = new;
     head->next = NULL;                  /* set end of list to NULL    */
     return;                             /* and finish                 */
  }

  temp = head;                           /* start at beginning of list */
                 /* whilst currentname < newname to be inserted then */
  while( strcmp( temp->name, new->name) < 0 ) {
        temp = temp->next;               /* goto the next node in list */
        if( temp == NULL )               /* dont go past end of list   */
           break;
  }

  /* we are the point to insert, we need previous node before we insert  */
  /* first check to see if its inserting before the first node!          */
  if( temp == head ) {
     new->next = head;                   /* link next field to original list   */
     head = new;                         /* head adjusted to new node          */
  }
  else {     /* okay, so its not the first node, a different approach    */
     prev = head;   /* start of the list, will cycle to node before temp */
     while( prev->next != temp ) {
        prev = prev->next;
     }
     prev->next = new;                   /* insert node between prev and next  */
```

```
    new->next = temp;
    if( end == prev )           /* if the new node is inserted at the */
      end = new;               /* end of the list the adjust 'end'   */
  }
}

/* this deletes all nodes from the place specified by ptr               */
/* if you pass it head, it will free up entire list                     */
void deletelist( struct node *ptr )
{
  struct node *temp;

  if( head == NULL ) return;   /* dont try to delete an empty list      */

  if( ptr == head ) {      /* if we are deleting the entire list        */
     head = NULL;         /* then reset head and end to signify empty   */
     end = NULL;          /* list                                       */
  }
  else {
     temp = head;          /* if its not the entire list, readjust end  */
     while( temp->next != ptr )        /* locate previous node to ptr  */
        temp = temp->next;
     end = temp;                      /* set end to node before ptr   */
  }

  while( ptr != NULL ) {   /* whilst there are still nodes to delete    */
    temp = ptr->next;     /* record address of next node               */
    free( ptr );          /* free this node                            */
    ptr = temp;           /* point to next node to be deleted          */
  }
}

/* this is the main routine where all the glue logic fits               */
main()
{
  char name[20];
  int id, ch = 1;
  struct node *ptr;

  clrscr();
  while( ch != 0 ) {
    printf("1 add a name \n");
    printf("2 delete a name \n");
    printf("3 list all names \n");
    printf("4 search for name \n");
    printf("5 insert a name \n");
```

```
printf("0 quit\n");
scanf("%d", &ch );
switch( ch )
{
   case 1:  /* add a name to end of list */
          printf("Enter in name -- ");
          scanf("%s", name );
          printf("Enter in id -- ");
          scanf("%d", &id );
          ptr = initnode( name, id );
          add( ptr );
          break;
   case 2:  /* delete a name */
          printf("Enter in name -- ");
          scanf("%s", name );
          ptr = searchname( head, name );
          if( ptr ==NULL ) {
             printf("Name %s not found\n", name );
          }
          else
            deletenode( ptr );
          break;

   case 3:  /* list all nodes */
          printlist( head );
          break;

   case 4:  /* search and print name */
          printf("Enter in name -- ");
          scanf("%s", name );
          ptr = searchname( head, name );
          if( ptr ==NULL ) {
             printf("Name %s not found\n", name );
          }
          else
            printnode( ptr );
          break;
   case 5:  /* insert a name in list */
          printf("Enter in name -- ");
          scanf("%s", name );
          printf("Enter in id -- ");
          scanf("%d", &id );
          ptr = initnode( name, id );
          insertnode( ptr );
          break;
```

```
    }
  }
  deletelist( head );
}
```

Program 3 : Program to form a linked list of integers

```
#include<stdio.h>
#include<conio.h>
#include<alloc.h>
struct node
{
        int a;
        struct node *next;
} *p, *r, *n;
struct node *create()
{
        int s, k;
        printf("Enter elements -1 to stop (please enter integers only): ");
        scanf("%d", &k);
        p = r = NULL;

        while (k != -1 )
        {
                n = (struct node *) malloc(sizeof (struct node));
                n->a = k;
                n->next = NULL;

                if (r == NULL)
                        r=n;
                else
                        p->next = n;
                        p = n;
                        printf("Enter the element -1 to stop : ");
                        scanf("%d", &k);
        }
        return (r);
}
void display(r)
struct node *r;
{
        printf("\n Root ");
        while (r != NULL)
        {
                printf("\t%d", r->a);
```

```
                r = r->next;
        }
        printf(" NULL ");
}
void main()
{
        struct node *r;
        clrscr();
        r=create();
        display(r);
        getch();
}
```

O/P:-

```
Enter elements -1 to stop : 1
Enter the element -1 to stop : 4
Enter the element -1 to stop : 3
Enter the element -1 to stop : 5
Enter the element -1 to stop : -1

 Root   1      4      3      5 NULL
```

Program 4: Create,insert a value after last node,remove first node,adda node after a key value,remove a node that has key value, Delete all nodes of a list

```
#include <stdio.h>
#include <conio.h>
#include <ctype.h>
#include <stdlib.h>

/*
1) TNODE -- the list nodes type;
        - lkey -- list key = a value which is different for each node of the list; it can be
useful for some applications
                it's recommendended to use it
        - name - a string information used only for example; here must be the real
information of the list
        - next - the next node pointer;

2) void CreateList() -- creates a list; for any TNODE structure (general function)
3) void ViewAllList() -- shows the list items for any TNODE structure (general function);
4) void DeleteList() -- removes completely the entire list ; (general function)
5) TNODE* FindNode(int key) -- returns a pointer to a list-node which has the lkey-value
equal-to key-parameter;
        (general function)
6) TNODE* InsertAfterKey(int key) -- inserts a new node (list item) after a list-node which
has the lkey-value equal-to
```

key-parameter; (general function)
7) TNODE* InsertAfterLast() -- inserts a new node (list item) after the last-node of the list ; (general function)
8) TNODE* InsertBeforeFirst() -- inserts a new node (list item) before the first-node of the list; (general function)
9) TNODE* InsertBeforeKey(int key) -- inserts a new node (list item) before a list-node which has the lkey-value equal-to
key-parameter; (general function)
10) void RemoveByKey(int key) -- removes a list-node which has the lkey-value equal-to
key-parameter; (general function)
11) void RemoveFirst() -- removes the first node of the list; (general function)
12) void RemoveLast() -- removes the last node of the list; (general function)

I ALSO HAVE WRITTEN A FEW FUNCTIONS WHICH ARE DEPENDENT TO THE TNODE STRUCTURE; THEY NEED TO BE REWRITTEN FOR EVERY
APPLICATION

1) void FreeNode(TNODE *p)//function specific to the application -- deallocates the memory for the p node
2) int LoadNode(TNODE *p) //function specific to the application -- loads the information into the nodes of the list
but should always return these values
0 - Error
1 - ok;
-1 - no more data to load
In this case (meaning my function) you shuld reply - anything for yes

- n/N for no
3) void PrintNode(TNODE *p) //function specific to the application -- shows the information of the p node

PLEASE ALSO READ THE COMMENTS IN THE CODE FOR MORE INFORMATION

****

*/

```
typedef struct node {
        int lkey;          /* key node */
        char name[10]; /*specific to the application; node's information */
        struct node* next;
} TNODE;

TNODE *first, *last; /*pointers to the first and last element of the linked list*/
```

```
int LoadNode(TNODE *p);
void FreeNode(TNODE *p);
void PrintNode(TNODE *p);


void CreateList() /*this function can be used no matter what the structure TNODE looks
like*/
{                                          /* meaning it can be used for any type of applications
concerning lists*/
        TNODE *p;               /*general function*/
        int n=sizeof(TNODE);

        first=last=0; /*empty list*/
        for(;;)
        {
                if( (p=(TNODE*)malloc(n))==0 ) /*allocation could not be made*/
                {
                        printf("\nNot enough memory");
                        break;
                }

                if(LoadNode(p)!=1)
                {
                        FreeNode(p);
                        break;
                }

                p->next=0;
                if (first==0) /*this list was empty since now*/
                        first=last=p;
                else
                {
                        last->next=p;
                        last=p;
                }
        }
}


int LoadNode(TNODE *p) /*function specific to the application*/
{                                                  /*but should always return these values*/
        /* 0 - Error
           1 - ok;
    -1 - no more data to load */
```

```
	char opt;
	printf("\nNew node?");
	opt=getche();
	opt=toupper(opt);
	if(opt!='N')
	{
		puts("\nPlease insert data for the current node:");
		printf("\nlkey:\t");
		if (scanf("%d",&(p->lkey))!=1) return 0; /*could not read lkey value for
current node*/

		printf("\nname:\t");if (scanf("%s",p->name)!=1) return 0;
		return 1;
	}
	else
		return -1;
}
void FreeNode(TNODE *p)/*function specific to the application*/
{
	free(p);
}

void ViewAllList()/*general function*/
{
	TNODE *p;
	p=first;
	while(p)
	{
		PrintNode(p);
		p=p->next;
	}
}

TNODE* FindNode(int key) /*general function*/
/*serches and returns the node having the lkey value equal to the key parameter*/
{
	TNODE *p;
	p=first;
	while(p)
	{
		if(p->lkey == key) return p;
		p=p->next;
	}
	return 0; /*value not found*/
}
```

```
void PrintNode(TNODE *p) /*function specific to the application*/
{
        if(p) printf("\n%d\t%s",p->lkey,p->name);
}

TNODE* InsertBeforeFirst()
/*general function
returns the node inserted
or 0 for failed insertion*/
{
        TNODE *p;
        int n=sizeof(TNODE);
        if (((p=(TNODE*)malloc(n))!=0) && (LoadNode(p)==1)) /*a new node has been
succesfully allocated and loaded*/
        {
                if (first==0) /*list was empty*/
                {
                        p->next=0;
                        first=last=p;
                }
                else
                {
                        p->next=first;
                        first=p;
                }
                return p;
        }
        if(p==0) /*not enough memory*/
                printf("\nNot enough memory");
        else /*the node could not be loaded*/
                FreeNode(p);
        return 0; /*there is no node inserted before first -- insertion failed*/
}

TNODE* InsertBeforeKey(int key)
/*general function*/
/*returns the node inserted*/
/*or 0 for failed insertion */
{
        TNODE *p, *q, *q1;
        /*p=the new node to insert
        q=key
        q1=the node before key*/
        int n=sizeof(TNODE);

        /*find q, q1*/
```

```
	q1=0;
	q=first;
	while(q)
	{
		if(q->lkey == key) break; /*key node found*/
		q1=q; /*keep on searching for key node*/
		q=q->next;
	}

	if(q==0)
	{
		printf("\nThere is no node having such a key or the list is empty");/*this
case also includes the case of empty list*/
		return 0;/*there is no node having such a key -- insertion can't be made*/
	}

	/*now the key was found -- so we try to make the insertion*/
	if (((p=(TNODE*)malloc(n))!=0) && (LoadNode(p)==1)) /*a new node has been
succesfully allocated and loaded*/
	{
		if(q==first) /*we have to insert before the first node*/
		{
			p->next=first;
			first=p;
		}
		else /*the key node is not the first one*/
		{
			p->next=q;
			q1->next=p;
		}
		return p;
	}
	if(p==0) /*not enough memory*/
		printf("\nNot enough memory");
	else /*the node could not be loaded*/
		FreeNode(p);
	return 0; /*there is no node inserted before key -- insertion failed*/
}

TNODE* InsertAfterKey(int key)
/*general function
returns the node inserted
or 0 for failed insertion*/

{
	TNODE *p, *q;
```

```
        /*p=the new node to insert
        //q=key*/
        int n=sizeof(TNODE);

        /*find q*/
        q=first;
        while(q)
        {
                if(q->lkey == key) break; /*key node found*/
                q=q->next; /*keep on searching for key node*/
        }
        if(q==0)
        {
                printf("\nThere is no node having such a key or the list is empty");/*this
case also includes the case of empty list*/
                return 0;/*there is no node having such a key -- insertion can't be made*/
        }

        /*now the key was found -- so we try to make the insertion*/
        if (((p=(TNODE*)malloc(n))!=0) && (LoadNode(p)==1)) /*a new node has been
succesfully allocated and loaded*/
        {
                if(q==last) /*we have to insert after the last node*/
                {
                        p->next=0;
                        last->next=p;
                        last=p;
                }
                else /*the key node is not the last one*/
                {
                        p->next=q->next;
                        q->next=p;
                }
                return p;
        }
        if(p==0) /*not enough memory*/
                printf("\nNot enough memory");
        else /*the node could not be loaded*/
                FreeNode(p);
        return 0; /*there is no node inserted after key -- insertion failed*/

}

TNODE* InsertAfterLast()
/*general function
//returns the node inserted
```

```
//or 0 for failed insertion*/
{
        TNODE *p;
        int n=sizeof(TNODE);
        if (((p=(TNODE*)malloc(n))!=0) && (LoadNode(p)==1)) /*a new node has been
succesfully allocated and loaded*/
        {
                p->next=0;
                if (first==0) /*list was empty*/
                        first=last=p;
                else
                {
                        last->next=p;
                        last=p;
                }
                return p;
        }
        if(p==0) /*not enough memory*/
                printf("\nNot enough memory");
        else /*the node could not be loaded*/
                FreeNode(p);
        return 0; /*there is no node inserted after last -- insertion failed*/
}

void RemoveFirst()
/*general function
//removes the first node of the list; pre and post-conditions: none*/
{
        TNODE *p;

        if(first==0)/*list was empty*/
                return;

        if(first==last)/*there is just one node*/
        {
                FreeNode(first);
                first=last=0;
                return;
        }

        p=first;
        first=first->next;
        FreeNode(p);
}
```

```
void RemoveLast()
/*general function
//removes the last node of the list; pre and post-conditions: none*/
{
        TNODE *p, *q;

        if(first==0)/*list was empty*/
                return;

        if(first==last)/*there is just one node*/
        {
                FreeNode(first);
                first=last=0;
                return;
        }

        q=0;/*there are at least 2 nodes*/
        p=first;
        while(p!=last)
        {
                q=p;
                p=p->next;
        }/*so we have q=the node before the last one*/

        p=last;/*now we're going to remove the last node*/
        FreeNode(p);
        q->next=0;
        last=q;

}

void RemoveByKey(int key)
{
        TNODE *p, *q;
        /*p - the key node
        //q=the node before the key node*/

        if(first==0)/*list was empty*/
                return;

        q=0;/*there are at least 2 nodes*/
        p=first;
        while(p)
        {
                if(p->lkey == key) break; /*key node found*/
                q=p;
```

```
            p=p->next;
        }/*so we have q=the node before the key one; p=key node*/

        if(!p)/*There is no node having such a key*/
        {
            printf("\nThere is no node having such a key");
            return;
        }

        if(first==last)/*there is just one node which is the key node*/
        {
            FreeNode(first);
            first=last=0;
            return;
        }

        if(p==first) /*first is the key node*/
        {
            first=first->next;
            FreeNode(p);
            return;
        }

        if(p==last) /* last was the key node*/
        {
            q->next=0;
            last=q;
            FreeNode(p);
            return;
        }

        q->next=p->next;
        FreeNode(p);
}

void DeleteList()
{
        TNODE *p;

        p=first;
        while(p)
        {
            first=first->next;
            FreeNode(p);
            p=first;
        }
```

```
        last=0;
}

void main()
{
  int key;
  clrscr();
  printf("\n Create a list");
  CreateList();/*this is an example of using these fonctions*/
  ViewAllList();
  getch();
  printf("\nInsert a value after last node");
  InsertAfterLast();
  ViewAllList();
  getch();
  printf("\nRemove first node from list");
  RemoveFirst();
  ViewAllList();
  getch();
  printf("\nEnter the value of key to add a node after it.\nKEY: ");
  scanf("%d",&key);
  InsertAfterKey(key);/*by example*/
  ViewAllList();
  getch();
  printf("\nEnter the value of key to remove that node\nKEY: ");
  scanf("%d",&key);
  RemoveByKey(key);
  ViewAllList();
  getch();
  DeleteList();
  ViewAllList();
  printf("\nDeleted the whole list");
  getch();
}
```

Program 5: create a list; add and delete nodes

```
# include <stdio.h>
struct list
{
        char name[20];
        struct list *next;
};

void create (struct list *start);
void list   (struct list *start);
```

```
struct list   *insert (struct list *start);
struct list   *find   (struct list *, char *);
struct list   *delete (struct list *start);
struct list   *tag;

main()
{
  struct list *head;
  clrscr();
  head=(struct list *)malloc(sizeof(struct list));
  create(head);
  list(head);
  head=insert(head);
  list(head);
  head=delete(head);
  list(head);
  getch();
}

void create(struct list *start)
{
  printf("\nEnter Name : [end] To Stop  : ");
  scanf("\n\n%s", start->name);

  if(strcmp(start->name, "end")==0)
    start->next=NULL;

   else
    {
        start->next=(struct list *)malloc(sizeof(struct list));
        create(start->next);
    }
 return;
}

void list(struct list *start)
{
  if (start -> next != NULL)

    {
        printf("\n%s", start->name);
        list(start->next);
     }

return;
}
```

```
struct list *insert(struct list *start)
{
  struct list *new;

  char newitem[20];
  char target[20];
  printf("\nNew Data Item : ");
  scanf("%s", newitem);
  printf("\nPlace Before (type \'end\' if last) ");
  scanf("%s", target);

  if (strcmp(start->name, target) == 0)
    {
        new=(struct list *)malloc(sizeof(struct list));
        strcpy(new->name, newitem);
        new->next=start;
    }

   else

     {
         tag = find(start, target);

         if(tag==NULL)
                printf("\nMatch Not Found !!");

        else
        {
          new=(struct list *) malloc(sizeof(struct list));
          strcpy(new->name, newitem);
          new->next=tag->next;
          tag->next=new;
         }
       }
 return(start);
}

struct list *find(struct list *rec, char target[])
{
  if(strcmp(rec->next->name, target)==0)
     return(rec);

   else

        if(rec->next->next==NULL)
```

```
            return(NULL);

      else
             find(rec->next, target);
}

struct list *delete(struct list *start)
{
struct list *find(struct list *rec, char target[]);
struct list *tag;
struct list *temp;
char   target[20];

  printf("\nData Item To Be Deleted : ");
  scanf("%s", target);

  if(strcmp(start->name, target) ==0)
    {
          temp  = start->next;
          free(start);
          start = temp;
     }

   else
         tag = find(start, target);

         if(tag == NULL)
           printf("Match Not Found !!");

    else
         {
            temp = tag->next->next;
            free(tag->next);
            tag->next = temp;
         }

return(start);
}
```

program 6: Link List Code Examples For Ordered Lists

```
#~~~
```

```
typedef struct
{
   char key[26];
   int  ssn;
} data_t;

typedef struct node_s
{
        struct node_s *next;
        data_t entry;
}node_t;


/*************************************************
* Name -- addlist : adds into the link list in
*       ascending order
*
* parameters :
*        parameter1 :  link list node/head pointer
*        Parameter2 :  element to add
*
* return : success (0) or
*        failure to add (-1)
*
* Pre-cond:
*        link list must have been initilized
* Post-cond:
*        link list is altered.
*************************************************/
int addlist(node_t **a, data_t b)
{
  node_t *work;
  node_t *prev;
  node_t *curr;

  curr=(node_t *)malloc(sizeof(node_t));
  if ( curr == NULL)
    return(-1);

  curr->next=NULL;
  curr->entry=b;

  if (*a == NULL)
  {
    *a=curr;
```

```
  }
  else
  {
    work=*a;
    prev=NULL;

    while(work != NULL &&
      (strcmp(work->entry.key,curr->entry.key) < 0))
    {
      prev=work;
      work=work->next;
    }
    /* Now have a spot to insert */
    if (prev == NULL)
    {
      *a=curr;
      curr->next=work;
    }
    else
    {
      curr->next = prev->next;
      prev->next=curr;
    }
  }

  return (0);

}


/*************************************************
* Name -- <LSearch> : Linear Search for an element.
*
* parameters :
*        parameter1 :  link list node
*        Parameter2 :  element to look for
* return :  address to a data element or null
*
* Pre-cond:
*        link list must exist
* Post-cond:
*        link list unmodified
*************************************************/
data_t *LSearch(node_t *a, char *keya)
{
        node_t *curr = a;
```

```
        while (curr!=NULL)
        {
                if (strcmp(curr->entry.key,keya)==0)
                        return (&(current->entry));
                curr=curr->next;
        }
        return NULL;
}


/*************************************************
* Name -- <initlist> : Initializing the link list
*
* parameters :
*       parameter1 :  address of head pointer to the link list
* return :     void
* Pre-cond:
*
* Post-cond:
*       link list head initilized to NULL
*************************************************/
void initlist(node_t **a)
{
        *a = NULL;
}



/*************************************************
* Name -- <dellist> : Delete an element in the link list
*
* parameters :
*       parameter1 :  address of head pointer to the link list
*      parameter2 : element to be deleted
*
* return :  success(0) or failure (-1)
*
* Pre-cond:
*       List must be properly built
* Post-cond:
*       element removed if found
*************************************************/
int dellist(node_t **a, char *keya)
{
  node_t *prev = NULL,
       *work = *a;
```

```
  if (*a == NULL)
  {
    return (-1);
  }

  while (work != NULL &&
        strcmp(work->entry.key, keya) != 0))
  {
    prev = work;
    work = work->next;
  }

  if (work == NULL)
  {
    return (-1);
  }
  else
  {
    if (prev == NULL)
    {
      *a = work->next;
    }
    else
    {
      prev->next = work->next;
    }

    free(work);
    return (0);
  }
}

/*************************************************
* Name -- <lengthlist> : Get the length of the list
*
* parameters :
*         parameter1 : head pointer to the link list
*
* return :  count
*
* Pre-cond:
*         List must be properly built
* Post-cond:
*         No change in the list
*************************************************/
```

```
int lengthlist(node_t *a)
{
  int count = 0;
  node_t *work = a;

  while(work != NULL)
  {
    count++;
    work = work->next;
  }

  return (count);
}


#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~#
#  End of Link List Code Examples For Ordered Lists.  #
#_____________________________________________________________________#
```

Program7: Link List Code Examples For Last In First Out

```
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~#
#   Link List Code Examples For Last In First Out.   #
#_____________________________________________________________________#

/************************************************
* Name -- addlist : adds to the end of the link list
*
* parameters :
*        parameter1 :  link list node
*        Parameter2 :  element to add
*
* return : success or failure to add
*
* Pre-cond:
*        link list must exist
* Post-cond:
*
************************************************/
int addlist(node_t **a, data_t b)
{
  node_t *work;
  node_t *prev;
```

```
  node_t *curr;

  curr=(node_t *)malloc(sizeof(node_t));
  if ( curr == NULL)
    return(-1);

  curr->next=NULL;
  curr->entry=b;

  if (*a == NULL)
  {
    *a=curr;
  }
  else
  {
    work=*a;
    prev=NULL;

    while(work!=NULL)
    {
      prev=work;
      work=work->next;
    }

    if (prev == NULL)
    {
      *a=curr;
    }
    else
    {
      prev->next=curr;
    }
  }

  return (0);

}


/*************************************************
* Name -- <LSearch1> : Linear Search for an element.
*
* parameters :
*        parameter1 :  link list node
*        Parameter2 :  element to look for
* return :  address to a data element
```

```
*
* Pre-cond:
*       link list must exist
* Post-cond:
*
*************************************************/
data_t *LSearch(node_t *a, char *keya)
{
        node_t *work = a;
        while (work != NULL)
        {
                if (strcmp(work->entry.key,keya) == 0)
                        return (&(work->entry));
                work = work->next;
        }
        return NULL;
}


/*************************************************
* Name -- <initlist> : Initializing the link list
*
* parameters :
*       parameter1 :  address of head pointer to the link list
* return :    void
* Pre-cond:
*
* Post-cond:
*
*************************************************/
void initlist(node_t **a)
{
        *a = NULL;
}


/*************************************************
* Name -- dellist : Delete an element from the
*    end of the link list
*
* parameters :
*       parameter1 :  address of head pointer to the link list
*     parameter2 : element to be deleted
*
* return :    void
*
```

```
* Pre-cond:
*
* Post-cond:
*
************************************************/
int dellist(node_t **a)
{
  node_t *prev,
       *work;

  if (*a == NULL)
  {
    return (-1);
  }

  work = *a;
  prev = NULL;

  while (work->next != NULL)
  {
    prev = work;
    work = work->next;
  }

  if (prev == NULL)
  {
    *a = NULL;
  }
  else
  {
    prev->next = NULL;
  }

  free(work);
  return (0);
}

/************************************************
* Name -- <lengthlist> : determine link list length
*
* parameters :
*         parameter1 : head pointer to the link list
*
* return :  count of the number of nodes
*
* Pre-cond:
```

```
*           correctly initilized link list
* Post-cond:
*           no change in the list
*************************************************/
int lengthlist(node_t *a)
{
  int count = 0;
  node_t *work = a;

  while (work != NULL)
  {
    count++;
    work = work->next;
  }

  return (count);
}


#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~#
#   End of Link List Code Examples For Last In First Out.   #
#_____________________________________________________________________________#
```

Program 8: Create a list. Add & delete components and display

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
#include<stdlib.h>

struct list_element
{
        char item[40];
        struct list_element *next;
};

typedef struct list_element node;

int menu(void);
void create(node *pt);
node *insert(node *pt);
node *delete(node *pt);
void display(node *pt);
int choice;
void main()
```

```
{
  node *start;
  clrscr();
  do
  {
     choice = menu();
     switch(choice)
     {
          case 1 :
                    start = (node *) malloc(sizeof(node));
                    create(start);
                    printf("\n");
                    display(start);
                    continue;
          case 2 :
                    start = insert(start);
                    printf("\n");
                    display(start);
                    continue;
          case 3 :
                    start = delete(start);
                    printf("\n");
                    display(start);
                    continue;
          case 4 :
                    display(start);
                    break;
          default :
                    printf("End of Computation\n");
          }
     } while (choice != 5);
}

int menu(void)
{
          do
          {
                    printf("\nMain Menu : \n");
                    printf("1.Create the linked list\n");
                    printf("2.Add a component\n");
                    printf("3.Delete a component\n");
                    printf("4.Display Contents\n");
                    printf("5.End List\n");
                    printf("Please enter your choice (1,2,3 or 4) -> ");
                    scanf("%d", &choice);
                    if (choice < 1 || choice > 5)
```

```
                        printf("\nError - Please try again\n");
        } while (choice < 1 || choice > 5);
        printf("\n");
        return(choice);
}

void create(node *record)
{
  printf("Data item (type \'END\' when finished) : ");
  scanf("%s", record->item);

  if (strcmp(record->item, "END") == 0)
        record->next = NULL;
  else
   {
        record->next = (node *) malloc (sizeof(node));
        create(record->next);
   }
   return;
}

void display(node *record)
{
        if (record->next != NULL)
        {
          printf("%s\n", record->item);
          display(record->next);
        }
        return;
}

node *insert(node *first)
{
        node *locate(node *, char []);
        node *newrecord;
        node *tag;
        char newitem[40];
        char target[40];
        printf("New data item : ");
        scanf("%s", newitem);
        printf("Place before (type \'END\' if last) : ");
        scanf("%s", target);

        if (strcmp(first->item, target) == 0)
        {
                newrecord = (node *) malloc(sizeof(node));
```

```
                strcpy(newrecord->item, newitem);
                newrecord->next = first;
                first = newrecord;
        }
        else
        {
                tag = locate(first, target);
                if (tag == NULL)
                printf("\nMatch not found - Try again\n");
                else
                {
                        newrecord = (node *) malloc(sizeof(node));
                        strcpy(newrecord->item, newitem);
                        newrecord->next=tag->next;
                        tag->next=newrecord;
                }
        }
        return(first);
}

node *locate(node *record, char target[])
{
        if (strcmp(record->next->item, target) == 0)
                return(record);
        else
                if (record->next->next == NULL)
                        return(NULL);
                else
                        locate(record->next, target);
}

node *delete(node *first)
{
  node *locate(node *, char []);
  node *tag;
  node *temp;
  char target[40];

  printf("\nData item to be deleted : ");
  scanf("%s", target);

  if (strcmp(first->item, target) == 0)
  {
        temp = first->next;
        free(first);
        first = temp;
```

```
 }
 else
 {
   tag = locate(first, target);
   if (tag == NULL)
   printf("\nMatch not found - Please try again\n");
   else
   {
        temp = tag->next->next;
        free(tag->next);
        tag->next = temp;
   }
 }
 return(first);
}
```

Program 9: Program to illustrate traversing a list

```
/* Program to illustrate traversing a list */
#include <stdio.h>
struct list
{
        int         value;
        struct list *next;
};
main()
{
        struct list n1, n2, n3, n4;
        struct list *list_pointer = &n1;
        n1.value = 100;
        n1.next = &n2;
        n2.value = 200;
        n2.next = &n3;
        n3.value = 300;
        n3.next = &n4;
        n4.value = 400;
        n4.next = 0;

        while( list_pointer != 0 )
        {
                printf("%d\n", list_pointer->value);
                list_pointer = list_pointer->next;
        }
}
```

Program 10(check)

```
#include<stdio.h>
#include<conio.h>
        struct node
        {
                int data;
                struct node *next;
        };
void main()
{
        struct node *n;
        n = b();
        printf("%d", n);
}


struct node *b()
        {
                struct node *head = NULL;
                struct node *second = NULL;
                struct node *third = NULL;

                head = malloc(sizeof(struct node));
                second = malloc(sizeof(struct node));
                third = malloc(sizeof(struct node));

                head->data = 1;
                head->next = second;

                second->data = 1;
                second->next = third;

                third->data = 3;
                third->next = NULL;

                return head;
        }
```

POINTERS

Program 1:Demononstration

```
#include<stdio.h>
#include<conio.h>
void main()
{
  int x = 3;
  int *px;
  px = &x;
  clrscr();
  printf("%u ", x);
  printf("%u ", *px);
  getch();
}
```

Program 2 : Demo - Arrays and pointers

```
#include<stdio.h>
#include<conio.h>
void main()
{
   int i[] = {11,22,33,44};
   int *j;
   int *k;
   j = i;
   k = &i[0] + 3;
   clrscr();
   printf("%u %u %u %u", &i[0], j, *(&i[1]), *k);
}
```

O/P :- 65518 65518 22 44

Program 3: Demo

```
#include<stdio.h>
#include<conio.h>
void main()
{
          int i = 2;
          int *j = &i;
          clrscr();
          printf("\nThe address of i is %u", &i);
          printf("\nThe value   of i is %d", i);
```

```
        printf("\n\nThe value of   j is %u", j);
        printf("\nThe address of j is %u", &j);
        printf("\nThe value of *j is %d", *j);
        getch();
}
```

O/P:- The address of i is 65524
The value of i is 2

The value of j is 65524
The address of j is 65522
The value of *j is 2

Program 4: Futute values of series of monthly deposits

```
#include<stdio.h>
#include<stdlib.h>
#include<ctype.h>
#include<math.h>

void table (double (*pf) (double i, int m, double n), double a, int m, double n);
double md1(double i, int m, double n);
double md2(double i, int m, double n);
double md3(double i, int m, double n);

void main()
{
        int m;
        double n, a;
        char freq;

        printf("\nFUTURE VALUE OF A SERIES OF MONTHLY DEPOSITS\n\n");
        printf("Amount of Each Monthly Payment : ");
        scanf("%lf", &a);
        printf("Enter Number of Years : ");
        scanf("%lf", &n);

        do
        {
          printf("Frequency of Compounding (A, S, Q, M, D, C) : ");
          scanf("%1s", &freq);
          freq = toupper(freq);
          if (freq == 'A')
          {
            m = 1;
            printf("\nAnnual Compounding\n");
          }
```

```
            else if (freq == 'S')
            {
              m = 2;
              printf("\nSemiannual Compounding\n");
            }
            else if (freq == 'Q')
            {
              m = 4;
              printf("\nQuarterly Compounding\n");
            }
            else if(freq == 'M')
            {
              m = 12;
              printf("\nMonthly Compounding\n");
            }
            else if(freq == 'D')
            {
              m = 360;
              printf("\nDaily Compounding\n");
            }
            else if(freq == 'C')
            {
              m = 0;
              printf("\nContinous Compounding\n");
            }
            else
                    printf("\nError - Please Repeat\n\n");
          } while (freq != 'A' && freq != 'S' && freq != 'Q' && freq != 'M' && freq != 'D' &&
freq != 'C');

          if (freq == 'C')
                    table(md3, a, m, n);
          else if (freq == 'D')
                    table(md2, a, m, n);
          else
                    table(md1, a, m, n);
}
void table (double (*pf) (double i, int m, double n), double a, int m, double n);
{
int count;
double i, f;
printf("\nIntrest Rate   Future Amount\n\n");
for (count = 1; count <= 20; count++)
{
  i = 0.01 * count;
  f = a * (*pf) (i, m, n);
```

```
  printf("      %2d        %2f\n", count, f);
}
}

double md1(double i, int m, double n)
{
        double factor, ratio;
        factor = 1 + i/m;
        ratio = 12 * (pow(factor, m*n) - 1) / i;
        return (ratio);
}

double md2(double i, int m, double n)
{
        double factor, ratio;
        factor = 1 + i/m;
        ratio = (pow(factor, m*n) - 1) / (pow(factor, m/12) - 1);
        return(ratio);
}

double md3(double i, int dummy, double n)
{
        double ratio;
        ratio = (exp(i*n) - 1) / (exp(i/12 - 1);
        return(ratio);
}
```

Program 5: Sorting(Reordering) of strings - Linear

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>

void reorder(int n, char *x[]);

void main()
{
        int i, n = 0;
        char *x[10];
        clrscr();
        printf("Enter each string on a separate line below\n\n");
        printf("Type \'END\' when finished\n\n");
        do
        {
                x[n] = (char *) malloc(12 * sizeof(char));
```

```
                printf("string %d : ", n + 1);
                scanf("%s", x[n]);
        }while(strcmp(x[n++], "END"));
        reorder(--n, x);
        printf("\n\nReordered List of Strings :\n");
        for (i = 0; i < n; i++)
        printf("\nString %d: %s", i+1, x[i]);
        getch();
}
void reorder(int n, char *x[])
{
  char *temp;
  int i, item;
  for(item = 0; item < n-1; item++)
  for(i = item + 1; i < n; i++)
  if (strcmp(x[item], x[i]) > 0)
  {
     temp = x[item];
     x[item] = x[i];
     x[i] = temp;
  }
}
```

O/P:- Enter each string on a separate line below

Type 'END' when finished

string 1 : z
string 2 : a
string 3 : g
string 4 : d
string 5 : END

Reordered List of Strings :

String 1: a
String 2: d
String 3: g
String 4: z

Program 7: Addition of two matrices(size dynamic)

```
#include<stdio.h>
#include<stdlib.h>
#define MAXROWS 20

void readinput(int *a[], int, int);
```

```
void computesums(int *a[], int *b[], int *c[], int , int );
void writeoutput(int *c[], int, int);

void main()
{
        int row, nrows, ncols;

        int *a[MAXROWS], *b[MAXROWS], *c[MAXROWS];
        clrscr();
        printf("How many rows ? ");
        scanf("%d", &nrows);
        printf("How many columns ? ");
        scanf("%d", &ncols);

        for (row = 0; row < nrows; row++)
        {
          a[row] = (int *) malloc (ncols * sizeof(int));
          b[row] = (int *) malloc (ncols * sizeof(int));
          c[row] = (int *) malloc (ncols * sizeof(int));
        }

        printf("\nFirst table : \n");
        readinput(a, nrows, ncols);

        printf("\nSecond table : \n");
        readinput(b, nrows, ncols);

        computesums(a, b, c, nrows, ncols);

        printf("\nSums of the elements :\n\n");
        writeoutput(c, nrows, ncols);
}

void readinput(int *a[MAXROWS], int m, int n)
{
        int row, col;
        for (row = 0; row < m; row++)
        {
          printf("\nEnter data for row no. %2d\n", row +1);
          for (col = 0; col < n; col++)
          scanf("%d", (*(a+row) + col));
        }
}

void computesums(int *a[MAXROWS], int *b[MAXROWS], int *c[MAXROWS], int m, int n)
{
```

```
   int row, col;
   for (row = 0; row < m; row++)
   for (col = 0; col < n; col++)
     *(*(c+row) + col) = *(*(a+row) + col) + *(*(b+row) + col);
}
void writeoutput(int *a[MAXROWS], int m, int n)
{
  int row, col;
  for (row = 0; row < m; row++)
  {
  for (col = 0; col < n; col++)
  printf("%4d", *(*(a+row) + col));
  printf("\n");
  }
}
```

Program 8: Program to count the no of vowels, consonants, digits, blanks and other special characters

```
#include<stdio.h>
#include<ctype.h>
char line[80];
int vowels, consonants, digits, whitespace, other ;
void scan_line(char line[], int *, int *, int *, int *, int *);
void main()
{

          clrscr();
          printf("\nEnter a line of text below :\n");
          scanf("%[^\n]", line);
          scan_line(line, &vowels, &consonants, &digits, &whitespace, &other);
          printf("\nNumber of vowels           %d ", vowels);
          printf("\nNumber of consonants       %d ", consonants);
          printf("\nNumber of digits           %d ", digits);
          printf("\nNumber of whitespace       %d ", whitespace);
          printf("\nNumber of other characters %d ", other);
          getch();
}
void scan_line(char line[], int *pv, int *pc, int *pd, int *pw, int *po)
{
          char c;
          int count = 0;
          while ((c = toupper(line[count])) != '\0')
          {
            if (c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U')
            ++*pv;
```

```
            else if (c >= 'A' && c <= 'Z')
            ++*pc;
            else if (c >='0' && c <= '9')
            ++*pd;
            else if (c == ' ' || c == '\t')
            ++*pw;
            else
             ++*po;
          ++count;
          }
}
```

O/P :- Enter a line of text below :
abhishek is great !

Number of vowels 6
Number of consonants 9
Number of digits 0
Number of whitespace 3
Number of other characters 1

Program 9: Outputs and Errors in programs with explaination

```
#include<stdio.h>
#include<conio.h>
#include<stdarg.h>
/*
void main()
{
   int a[] = {10,20,30,40,50};
   int j;
   for (j = 0; j < 5; j++)
   {
         printf("\n%d", *a);
         a++;
   }
} */
```

/* Output :- Error message : lvalue required in function main
Explanation :- Whenever we mention the name of the array, we get its base address. Therefore, first time through the loop, the printf() should print the value at this base address. There is no problem upto this. The problem lies in the next statement, a++. Since C does not perform bounds checking on an array, the only thing that it remembers about an array once declared is its base address. And a++ attempts to change this base address which C won't allow because if it does so, it would be unable to remember the beginning of the array. Anything which can change in complier's

language is called lvalue. Since value of a cannot be changed through ++, it flashes the error saying 'Lvalue required' so that ++ operator can change it*/

```
/*void main()
{
  float a[] = {13.24, 1.5, 1.5, 5.4, 3.5};
  float *j, *k;
  j = a;
  k = a + 4;
  j = j * 2;
  k = k / 2;
  printf("\n%f %f", *j, *k);
} */
```

/* Output : Error message: Illegal use of pointer in function main */
/* Explanation : j and k have been declared as pointer variables, which would contain the addresses of floats. In other words, j and k are float pointers. To begin with the base address of a[] is stored in j. The next statement is right; the address of the 4th float from the base address is stored in k. The next two statements are erroneous. This is because the only operations that can be performed on pointers are addition and subtraction.Multiplication or division of a pointer is not allowed */

```
/*
void main()
{
        int n[25];
        n[0] = 100;
        n[24] = 200;
        printf("\n%d %d", *n, *(n+24) + *(n+0));
}*/
/*
void main()
{
  int b[] = {10,20,30,40,50};
  int i, *k;
  k = &b[4] - 4;
  clrscr();
  for (i = 0; i <= 4; i++)
  {
    printf("%d ", *k);
    k++;
  }
} */
/*
void main()
```

```
{
        int a[] = {2, 4, 6, 8, 10};
        int i;
        clrscr();
        for (i = 0; i <=4; i++)
        {
                *(a+i) = a[i] + i[a];
                printf("%d ", *(i+a));
        }
}
```

output :- 4 8 12 16 20

Explanation :-

1) Mentioning the name of the array gives the base address of the array.
2) Array elements are stored in contiguous memory locations.
3) On adding 1 to the address of an integer, we get the address of the next integer.

Remember that internally C always accesses array elements using pointers. Thus, we say a[i], internally c converts it to *(a+i), which means value of ith integer from the base address. Now, if the expression a[i] is same as *(a+i) then *(i+a) must be same as i[a]. But *(a+i) is same as *(i+a). Therefore a[i] is i[a].*/

```
/*void main()
{
        char s[] = "Kapil";
        clrscr();
        printf("\n%d", *(s+strlen(s)) );
}
```

Output : 0

Explanation : Mentioning the name of the string gives the base address of the string. The function strlen(s) returns the length of the string s[], which in this case is 5. In printf(), using the value at address operator we are trying to print out the contents of the 5th address from the base address of the string. At this address there is \0 which is automatically stored to mark the end of the string. The ascii value of \0 is 0 which is what is being printed by printf().
*/

```
/*void main()
{
        char ch[20];
        int i;
        clrscr();
        for (i = 0; i < 19; i++)
                *(ch + i) = 67;
        *(ch+i) = '\0';
```

```
        printf("\n%s", ch);
}
O/P:- CCCCCCCCCCCCCCCCCC
*/

/* Displays the address of the function*/
/*void main()
{
        int display();
        clrscr();
        printf("The address of function display is %u", display);
        display();
}
display()
{
}*/
/*
main()
{
        int display();
        int (*func_ptr)();

        func_ptr = display;
        clrscr();
        printf("\nAddress of function display is %u", func_ptr);
        (*func_ptr)();
}
display()
{
}*/
/* Function to accept variable length arguments */
void main()
{
        int max;
        max = findmax(5,23,15,1,92,50);
        printf("\nMax = %d", max);
        max = findmax(3,100,300,29);
        printf("\nMax = %d", max);
}
findmax(int tot_num)16
{15
14      int max, count, num;
13
12      va_list ptr;
11      va_start(ptr, tot_num);
10      max = va_arg(ptr, int);
```

```
9
8       for (count = 1; count < tot_num; count++)
7       {
6         num = va_arg(ptr, int);
5         if (num > max)
4               max = num;
3       }
2       return(max);
}1
```

Program 10: Pointers(multiple) and some others basic functions

```
#include<stdio.h>
#include<conio.h>
/*
void main()
{
        int a, *b, **c, ***d, ****e;

        a = 10;
        b = &a;
        c = &b;
        d = &c;
        e = &d;
        clrscr();
        printf("\na = %d b = %u  c = %u d = %u e = %u ", a, b, c, d, e);
        printf("\n%d %d %d", a, a+ *b, **c + ***d + ****e);
        getch();
}
O/P:- a = 10 b = 65524  c = 65522 d = 65520 e = 65518
10 20 30
*/
/*
void main()
{
        char c, *cc;
        int i;
        long l;
        float f;

        c = 'Z';
        i = 15;
        l = 77777;
        f = 3.14;
        cc = &c;
        printf("\nc = %c cc = %u", *cc, cc);
```

```
        cc = &i;
        printf("\ni = %d cc = %u", *cc, cc);
        cc = &f;
        printf("\nf = %f cc = %u", *cc, cc);
        getch();
}
```

O/P:- a = 10 b = 65524 c = 65522 d = 65520 e = 65518
10 20 30
c = Z cc = 65525
i = 15 cc = 65522
f = -1.82582955860730588000000000000000000000e+259 cc = 16456

```
*/
/*
void main()
{
   int a = 5, *aa;
   aa = &a;
   a = power(&aa);
   clrscr();
   printf("\n a = %d aa = %u", a, aa);
}
power(int **ptr)
{
  int b;
  b = **ptr * **ptr;
  return(b);
}
```

O/P:- a = 25 aa = 65524

```
*/
/*
void main()
{
  int i = 10, j = 20, diff;
  diff = &j - &i;
  clrscr();
  printf("\naddress of i = %u address of j = %u", &i, &j);
  printf("\ndifference of address of i and j is %d", diff);
}
```

O/P:- address of i = 65524 address of j = 65522
difference of address of i and j is -1

```
*/
```

Program 11: Pointers and arrays (address of values in that array)

```
#include<stdio.h>
#include<conio.h>
#define MAX 3
void main()
{
static char *list[MAX] =
                {
                        "Dravid",
                        "Sachin",
                        "Prasad"
                };
int i = 0;
        clrscr();
        for (i = 0; i < 3; i++)
        {
          printf("%u ", &list[i]);
          printf("\n");
        }

}
```

O/p:- 168 170 172

Program 12: passing address thro functions

```
/* passing address thro functions  */
#include<stdio.h>
#include<conio.h>
struct book
{
        char name[25];
        char author[20];
        int callno;
};

void display(struct book *b);

void main()
{
  struct book b1 = {"C programming Lang", "Lafore", 34};
  display(&b1);
}
void display(struct book *b)
```

```
{
        clrscr();
        printf("%s   %s  %d", b->name, b->author, b->callno);
        getch();
}
```
O/P:- C programming Lang Lafore 34

Program 13: static arrays and pointers(truncation)

```
#include<stdio.h>
#include<conio.h>
void main()
{
        static char as[] = "Greetings";
        static char *ps = "Greetings";
        /* as++; */
        ps++;
        clrscr();
        puts(ps);
}
```
O/P:- reetings

Program 14:Structures and pointers

```
#include<stdio.h>
#include<conio.h>
void main()
{
        struct book
        {
                char name[25];
                char author[25];
                int callno;
        }b1, *ptr;

        clrscr();
        strcpy(b1.name,"C programming Language");
        strcpy(b1.author, "Ritchie");
        b1.callno = 10;
        printf("\n%s   %s     %d", b1.name, b1.author, b1.callno);

        strcpy(ptr->name, "C P L");
        strcpy(ptr->author,"Dennis");
        ptr->callno = 15;
```

```
        printf("\n\n\n%s   %s     %d", ptr->name, ptr->author, ptr->callno);
}
```

O/P:- *C programming Language   Ritchie   10*

(null)   Dennis   15Null ☼ inter assignment

program 15: return and store pointer values

```
/* return and store pointer values */
void main()
{
        int *c;
        c = check(10,20);
        clrscr();
        printf("\n c = %u", c);
}
check(int i, int j)
{
        int *p, *q;
        p = &i;
        q = &j;
        if ( i >= 45)
                return(p);
        else
                return(q);
}
```

O/p:- c = 65522

Program 16: pointers used to swap 2 array variables

```
/* pointers used to swap 2 array variables */`
#include<stdio.h>
void main()
{
        char *names[] = {
                                "Dravid",
                                "Kapil",
                                "Sachin",
                                "Srinath",
                                "Irfan"
                        };
        char *temp;

        printf("\nOriginal %s  %s", names[2], names[3]);
```

```
        temp = names[2];
        names[2] = names[3];
        names[3] = temp;

        printf("\nNew %s %s", names[2], names[3]);
}
```

O/P:- Original Sachin Srinath
New Srinath Sachin

Program 17: function returning pointers

```
/* function returning pointers */
#include<stdio.h>
#include<conio.h>

void main()
{

        int *p;
        int *fun();
        clrscr();
        p = fun();
        printf("\nValue of *p %u", *p);
        printf("\n%u", p);
}
int *fun()
{
        int i = 20;
        return (&i);
}
```

O/P:- Value of *p 20
65518

Program 18: return muliple values from a function

```
/* return muliple values from a function */
#include<stdio.h>
#include<conio.h>
void main()
{
        int radius;
        float area, perimeter;
        radius = 0;
```

```
        area = 0.0;
        perimeter = 0.0;
        clrscr();
        printf("\nEnter radius of a circle ");
        scanf("%d", &radius);
        areaperi(radius, &area, &perimeter);
        printf("\nArea = %f", area);
        printf("\nPerimeter = %f", perimeter);
}
areaperi(int r, float *a, float *p)
{
        *a = 3.14 * r * r;
        *p = 2 * 3.14 * r;
}
```

O/P:- Enter radius of a circle 5

Area = 78.500000
Perimeter = 31.400000

Program 19: call by reference

```
/* call by reference */
#include<stdio.h>
#include<conio.h>
void main()
{
        int a = 10;
        int b = 20;
        swapr(&a, &b);
        clrscr();
        printf("\n a = %d", a);
        printf("\n b = %d", b);
}
swapr(int *x, int *y)
{
        int t;
        t = *x;
        *x = *y;
        *y = t;
}
```

program 20: Call by value

```
/* call by value */
#include<stdio.h>
#include<conio.h>
void main()
{
        int a = 10;
        int b = 20;
        clrscr();
        swapv(a,b);
        printf("\n a = %d", a);
        printf("\n b = %d", b);
}
swapv(int x, int y)
{
        int t;
        t = x;
        x = y;
        y = t;
        printf("\n x = %d", x);
        printf("\n y = %d", y);
}
```

Program 21: Pointers to pointers

```
/* Pointers to pointers */
#include<stdio.h>
#include<conio.h>
void main()
{
        int i = 3;
        int *j;
        int **k;

        j = &i;
        k = &j;
        clrscr();

        printf("\nAddress of i = %u", &i);
        printf("\nValue of j = %u", j);
        printf("\nValue of *k = %u", *k);
        printf("\nAddress of j = %u", &j);
        printf("\nValue of k = %u", k);
        printf("\nAddress of k = %u", &k);
```

```
        printf("\n\nValue of j = %u", j);
        printf("\nValue of k = %u", k);
        printf("\nValue of i = %d", i);
        printf("\nValue of *j = %d", *j);
        printf("\nValue of **k = %u", **k);
}
```

O/P:- Address of i = 65524
Value of j = 65524
Value of *k = 65524
Address of j = 65522
Value of k = 65522
Address of k = 65520

Value of j = 65524
Value of k = 65522
Value of i = 3
Value of *j = 3
Value of **k = 3

Program 22: Pointers of different data types

```
/* Pointers of different data types */
#include<stdio.h>
#include<conio.h>
void main()
{       char c, *cc;
        int i, *ii;
        float f, *ff;
        c = 'A';
        i = 3;
        f = 3.14;
        cc = &c;
        ii = &i;
        ff = &f;
        clrscr();
        printf("\nAddress contained in cc = %u", cc);
        printf("\nAddress contained in ii = %u", ii);
        printf("\nAddress contained in aa = %u", ff);

        printf("\n\nValue of c = %c", *cc);
        printf("\nValue of i = %d", *ii);
        printf("\nValue of f = %f", *ff);
        getch();
}
```

O/P:-

```
Address contained in cc = 65525
Address contained in ii = 65520
Address contained in aa = 65516

Value of c = A
Value of i = 3
Value of f = 3.140000
```

Program 23: Programs using pointers

```
/* Programs using pointers */
#include<stdio.h>
#include<conio.h>
void main()
{
        int i = 3;
        int *j;
        j = &i;
        clrscr();
        printf("\nAddress of i = %u", &i);
        printf("\nAddress of i (stored in j) = %u", j);
        printf("\nAddress of j = %u", &j);
        printf("\nValue of j = %u", j);
        printf("\nValue of i = %d", i);
        printf("\nValue of *j = %d", *j);
        getch();
}
```

O/P:- Address of i = 65524
Address of i (stored in j) = 65524
Address of j = 65522
Value of j = 65524
Value of i = 3
Value of *j = 3

Program 24: What would be the output

```
/* What would be the output */
#include<stdio.h>
#include<conio.h>
void main()
{
  static int a[] = {0,1,2,3,4};
  static int *p[] = {a, a+1, a+2, a+3, a+4};
  clrscr();
  printf("\n%u ", &a[0]);
```

```
  printf("\n%u ", &p[0]);
  printf("\n%u %u %d", p, *p, *(*p));
  getch();
}
```

O/P:- 168
178
178 168 0

Program 25: array of pointers

```
/* array of pointers */
/*void main()
{
   int *arr[4];
   int i = 31, j = 5, k = 19, l = 71, m;
   arr[0] = &i;
   arr[1] = &j;
   arr[2] = &k;
   arr[3] = &l;
   clrscr();
   for (m = 0; m < 4; m++)
     printf("\n%d ", *(*(arr+m)));
     getch();
}
```

O/P:- 31
5
19
71

Program 26: 3-d arrays and pointers

```
/* 3-d arrays */
#include<stdio.h>
#include<conio.h>
void main()
{
  int a[3][4][2] = {
                              {
                                {2,4},
                                {7,8},
                                {3,4}
                                {5,6}
                              },
```

```
                        {
                          {7,6},
                          {3,4},
                          {5,3},
                          {2,3}
                        },
                        {
                          {8,9},
                          {7,2},
                          {3,4},
                          {5,1}
                        }
                    };
  clrscr();
  printf("\n%u ", a);
  printf("\n%u ", *a);
  printf("\n%u ", **a);
  printf("\n%u ", ***a);
  printf("\n%u ", a + 1);
  printf("\n%u ", *a + 1);
  printf("\n%u ", **a + 1);
  printf("\n%u ", ***a + 1);
}
```

O/P:- 65478
65478
65478
2
65494
65482
65480
3

Program 27: finding the address of elements in arrays using pointers(*,&)

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int stud[5][2] = {
                                {1,1},
                                {2,2},
                                {3,3},
                                {4,4},
                                {5,5}
                            };
        int i, j;
```

```
        clrscr();
        for (i = 0; i < 5; i++)
        {
                printf("\n");
                for (j = 0; j <= 1; j++)
                printf("%u ", &stud[i][j]);
         }
         printf("\n");
         for (i = 0; i < 5; i++)
         {
                  printf("%u ", *(stud+i) + 1);
         }
}
```

O/P:- 65506 65508
65510 65512
65514 65516
65518 65520
65522 65524
65508 65512 65516 65520 65524

Program 28: Address in 2-d array

```
/* 2-d array */
#include<stdio.h>
#include<conio.h>
void main()
{
        int stud[5][2] = {
                                {1234, 56},
                                {1212, 33},
                                {1434, 80},
                                {1312, 78},
                                {1203, 75}
                        };
        int i, j;
        clrscr();
        for (i = 0; i <= 4; i++)
                printf("\nAddress of %dth 1-D array = %u", i, stud[i]);
}
```

O/P:- Address of 0th 1-D array = 65506
Address of 1th 1-D array = 65510
Address of 2th 1-D array = 65514
Address of 3th 1-D array = 65518
Address of 4th 1-D array = 65522

Program 29 :Pointers and arrays (displaying of values using different notations)

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int num[] = {1,2,3,4,5,6};
        int i = 0;
        clrscr();
        while (i < 6 )
        {
          printf("\nElement = %d", num[i]);
          printf("%2d", *(num+i));
          printf("%2d", *(i+num));
          printf("%2d", i[num]);
           i++;
         }
}
```

O/P:- Element = 1 1 1 1
Element = 2 2 2 2
Element = 3 3 3 3
Element = 4 4 4 4
Element = 5 5 5 5
Element = 6 6 6 6

Program 30: pointers throu functions

```
/* pointers throu functions */
#include<stdio.h>
#include<conio.h>
void main()
{
        int num[] = {1,2,3,4,5,6};
         clrscr();
         display(&num[0], 6);
}
display(int *j, int n)
{
        int i = 1;
        while (i <=n)
        {
          printf("\nElement = %d", *j);
          i++;
          j++;
```

```
        }
}
```

O/P:- Element = 1
Element = 2
Element = 3
Element = 4
Element = 5
Element = 6

Program 31: Arrays thro pointers

```
/* Arrays thro pointers */
#include<stdio.h>
#include<conio.h>
void main()
{
        int num[] = {24,34,12,44,56,17};
        int i = 0, *j;
        j = num;
        clrscr();
        for (i = 0; i<=5; i++, j++)
        {
          printf("\nAddress = %u ", &num[i]);
          printf("Element = %d ", *j);
        }
}
```

O/P:- Address = 65514 Element = 24
Address = 65516 Element = 34
Address = 65518 Element = 12
Address = 65520 Element = 44
Address = 65522 Element = 56
Address = 65524 Element = 17

Program 32: increment pointers

```
/* increment pointers */
void main()
{
        int i = 3, *x;
        float j = 1.5, *y;
        char k = 'c', *z;
        clrscr();
        printf("\nValue of i = %d", i);
        printf("\nValue of j = %f", j);
```

```
        printf("\nValue of k = %c", k);

        x = &i;
        y = &j;
        z = &k;

        printf("\n");
        printf("\nOriginal value in x = %u", x);
        printf("\nOriginal value in y = %u", y);
        printf("\nOriginal value in z = %u", z);

        x++;
        y++;
        z++;
        printf("\n");
        printf("\nNew Value of x = %u", x);
        printf("\nNew Value of y = %u", y);
        printf("\nNew Value of z = %u", z);
}
```

O/P:-

```
Value of i = 3
Value of j = 1.500000
Value of k = c

Original value in x = 65524
Original value in y = 65518
Original value in z = 65517

New Value of x = 65526
New Value of y = 65522
New Value of z = 65518
```

Program 33: increament Pointers through Function

```
#include<stdio.h>
#include<conio.h>

void main()
{
        float *j();
        float p = 23.5, *q;
        q = &p;
        clrscr();
        printf("\nq before call = %u", q);
        q = j(&p);
```

```
        printf("\nq after call  = %u", q);
}
float *j(float *r)
{
        r = r + 1;
        return(r);
}
```

O/P:- q before call = 65522
q after call = 65526

Program 34: printing the value with pointer notation without using pointers

```
/* printing the value with pointer notation without using pointers */

#include<stdio.h>
#include<conio.h>
void main()
{
        int i = 3 ;
        clrscr();
        printf("\n Address of i = %u", &i);
        printf("\n Value of i = %d", i);
        printf("\n Value of i = %d", *(&i)); // same as printing i
}
```

O/P:-

Address of i = 65524
Value of i = 3
Value of i = 3

Program 35: Some More programs on pointers

```
/* printing the value with pointer notation
   without using pointers */
/*
#include<stdio.h>
#include<conio.h>
void main()
{
        int i = 3 ;
        clrscr();
        printf("\n Address of i = %u", &i);
        printf("\n Value of i = %d", i);
        printf("\n Value of i = %d", *(&i)); // same as printing i
```

```
}
*/
/*
void main()
{
        int i = 3;
        int *j;
        j = &i;
        clrscr();
        printf("\nAddress of i = %u", &i);
        printf("\nAddress of i (stored in j) = %u", j);
        printf("\nAddress of j = %u", &j);
        printf("\nValue of j = %u", j);
        printf("\nValue of i = %d", i);
        printf("\nValue of *j = %d", *j);
        getch();
}
*/
/*
void main()
{
        int i = 3;
        int *j;
        int **k;

        j = &i;
        k = &j;
        clrscr();

        printf("\nAddress of i = %u", &i);
        printf("\nValue of j = %u", j);
        printf("\nValue of *k = %u", *k);
        printf("\nAddress of j = %u", &j);
        printf("\nValue of k = %u", k);
        printf("\nAddress of k = %u", &k);

        printf("\n\nValue of j = %u", j);
        printf("\nValue of k = %u", k);
        printf("\nValue of i = %d", i);
        printf("\nValue of *j = %d", *j);
        printf("\nValue of **k = %u", **k);
}
*/
/*
void main()
{
```

```
        char c, *cc;
        int i, *ii;
        float f, *ff;

        c = 'A';
        i = 3;
        f = 3.14;

        cc = &c;
        ii = &i;
        ff = &f;
        clrscr();
        printf("\nAddress contained in cc = %u", cc);
        printf("\nAddress contained in ii = %u", ii);
        printf("\nAddress contained in aa = %u", ff);

        printf("\n\nValue of c = %c", *cc);
        printf("\nValue of i = %d", *ii);
        printf("\nValue of f = %f", *ff);
        getch();
}*/
/* call by value
void main()
{
        int a = 10;
        int b = 20;
        clrscr();
        swapv(a,b);
        printf("\n a = %d", a);
        printf("\n b = %d", b);
}
swapv(int x, int y)
{
        int t;
        t = x;
        x = y;
        y = t;
        printf("\n x = %d", x);
        printf("\n y = %d", y);
}*/
/* call by reference
void main()
{
        int a = 10;
        int b = 20;
        swapr(&a, &b);
```

```
        clrscr();
        printf("\n a = %d", a);
        printf("\n b = %d", b);
}
swapr(int *x, int *y)
{
        int t;
        t = *x;
        *x = *y;
        *y = t;
}*/
/* return muliple values
void main()
{
        int radius;
        float area, perimeter;
        radius = 0;
        area = 0.0;
        perimeter = 0.0;
        clrscr();
        printf("\nEnter radius of a circle ");
        scanf("%d", &radius);
        areaperi(radius, &area, &perimeter);
        printf("\nArea = %f", area);
        printf("\nPerimeter = %f", perimeter);
}
areaperi(int r, float *a, float *p)
{
        *a = 3.14 * r * r;
        *p = 2 * 3.14 * r;
}
*/
/* function returning pointers
void main()
{
        int *p;
        int *fun();

        p = fun();
        printf("\n%u", p);
}
int *fun()
{
        int i = 20;
        return (&i);
}
```

```
*/
/*
void main()
{
        float a = 7.9999999;
        float *b, *c;
        b = &a;
        c = b;
        clrscr();
        printf("\n%u %u %u", &a, b, c);
        printf("\n%.2f %.2f %.2f %.2f", a, *(&a), *b, *c);
}*/
/*
void main()
{
        int *c;
        c = check(10,20);
        clrscr();
        printf("\n c = %u", c);
}
check(int i, int j)
{
        int *p, *q;
        p = &i;
        q = &j;
        if ( i >= 45)
                return(p);
        else
                return(q);
}
*/
/*
void main()
{
        float *j();
        float p = 23.5, *q;
        q = &p;
        clrscr();
        printf("\nq before call = %u", q);
        q = j(&p);
        printf("\nq after call  = %u", q);
}
float *j(float * r)
{
        r = r + 1;
        return(r);
```

```
}
*/
/*
void main()
{
        int i = 3, *x;
        float j = 1.5, *y;
        char k = 'c', *z;
        clrscr();
        printf("\nValue of i = %d", i);
        printf("\nValue of j = %f", j);
        printf("\nValue of k = %c", k);

        x = &i;
        y = &j;
        z = &k;

        printf("\n");
        printf("\nOriginal value in x = %u", x);
        printf("\nOriginal value in y = %u", y);
        printf("\nOriginal value in z = %u", z);

        x++;
        y++;
        z++;
        printf("\n");
        printf("\nNew Value of x = %u", x);
        printf("\nNew Value of y = %u", y);
        printf("\nNew Value of z = %u", z);
} */
/*
void main()
{
        int i = 4, *j, *k;
        j = &i;
        clrscr();
        printf("%u ", j);
        printf("\n%u ", *j);
}*/
/*
void main()
{
        int num[] = {24,34,12,44,56,17};
        int i = 0, *j;
        j = num;
        clrscr();
```

```
        for (i = 0; i<=5; i++, j++)
        {
          printf("\nAddress = %u ", &num[i]);
          printf("Element = %d ", *j);
        }
}
*/

/*void main()
{
        int num[] = {1,2,3,4,5,6};
        // display(&num[0], 6);
        display(num, 6);
}
display(int *j, int n)
{
        int i = 1;
        while (i <=n)
        {
          printf("\nElement = %d", *j);
          i++;
          j++;
         }
}*/
/*
void main()
{
        int num[] = {1,2,3,4,5,6};
        int i = 0;
        clrscr();
        while (i < 6 )
        {
          printf("\nElement = %d", num[i]);
          printf("%2d", *(num+i));
          printf("%2d", *(i+num));
          printf("%2d", i[num]);
           i++;
         }
}*/
/*void main()
{
        int stud[5][2] = {
                                {1234, 56},
                                {1212, 33},
                                {1434, 80},
                                {1312, 78},
```

```
                                {1203, 75}
                        };
        int i, j;
        clrscr();
        for (i = 0; i <= 4; i++)
                printf("\nAddress of %dth 1-D array = %u", i, stud[i]);
}*/
/*void main()
{
  int i = 10 ;
  int *j;
  j = &i;
  *j = *j + 20;
  clrscr();
  printf("%u ", *j);
  printf("\n%u ", i);
  getch();
}*/
/*
void main()
{
        int i[] = {1,2,3,4,5};
        int x;
        clrscr();
        for (x = 0; x < 5; x++)
        printf("%u ", *(i+x));
}
*/

/*
void main()
{
        int stud[5][2] = {
                                {1,1},
                                {2,2},
                                {3,3},
                                {4,4},
                                {5,5}
                        };
        int i, j;
        clrscr();
        for (i = 0; i < 5; i++)
        {
                printf("\n");
                for (j = 0; j <= 1; j++)
                printf("%u ", &stud[i][j]);
```

```
        }
        printf("\n");
        for (i = 0; i < 5; i++)
        {
                printf("%u ", *(stud+i) + 1);
        }
} */
/* accessing elements of 2'd array using pointers
void main()
{
  int stud[5][2] = {
                      {1, 1},
                      {2, 2},
                      {3, 3},
                      {4, 4},
                      {5, 5}
                    };
  int i, j;
  for (i = 0; i < 5; i++)
  {
    printf("\n");
    for (j = 0; j <= 1; j++)
      printf("%d ", *(*(stud + i) + j) );
  }
}
*/
/*
void main()
{
  int a[2][3][2] = {
                          {
                            {2,4},
                            {7,8},
                            {3,4}
                          },
                          {
                            {2,2},
                            {2,3},
                            {3,4}
                          }
                    };
  clrscr();
  printf("\n%u ", a);
  printf("\n%u ", *a);
  printf("\n%u ", **a);
  printf("\n%u ", ***a);
```

```
  printf("\n%u ", a + 1);
  printf("\n%u ", *a + 1);
  printf("\n%u ", **a + 1);
  printf("\n%u ", ***a + 1);
}
*/
/*void main()
{
   int *arr[4];
   int i = 31, j = 5, k = 19, l = 71, m;
   arr[0] = &i;
   arr[1] = &j;
   arr[2] = &k;
   arr[3] = &l;
   clrscr();
   for (m = 0; m < 4; m++)
     printf("\n%d ", *(*(arr+m)));
     getch();
}*/
void main()
{
  static int a[] = {0,1,2,3,4};
  static int *p[] = {a, a+1, a+2, a+3, a+4};
  clrscr();
  printf("\n%u ", &a[0]);
  printf("\n%u ", &p[0]);
  printf("\n%u %u %d", p, *p, *(*p));
  getch();
}
```

Program 36: Arrays and pointers (eg: ptr[-i])

```
#include<stdio.h>
#include<conio.h>
void main()
{
          int arr[] = {0,1,2,3,4};
          int i, *ptr;
          clrscr();
          for (ptr = arr + 4, i=0; i < 5; i++)
            printf("%d ", ptr[-i]);
}
```

O/P:- 4 3 2 1 0

Program 37: Arrays and pointers fundamental

```
#include<stdio.h>
#include<conio.h>
void main()
{
  int arr[] = {0,1,2,3,4};
  int *ptr;
  clrscr();
  for (ptr = &arr[0]; ptr <= &arr[4]; ptr++)
   printf("%d ", *ptr);
}
```

O/P:- 0 1 2 3 4

Program 38: Lvalue Error

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int a[] = {2,4,6,8,10};
        int i;
        clrscr();
        for (i = 0; i < 5; i++)
        {
          printf("\n%d %d", *a);
          a++;
        }
}
```

O/P:- Error- lvalue required

Program 39: pointers and func

```
#include<stdio.h>
#include<conio.h>
int f(int, int *);
void main()
{
        int a[] = {2,4,6,8,10};
        int i, b = 5;
        clrscr();
        for (i = 0; i < 5; i++)
        {
          f(a[i], &b);
          printf("\n%d %d", a[i], b);
```

```
        }
}
f(int x, int *y)
{
  x = *y = *y + 2;
}
```

O/P:-

```
2 7
4 9
6 11
8 13
10 15
```

Program 40: Incrementing pointers(arrays);

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int b[] = {10,20,30,40,50};
        int i, *k;
        k = &b[4] - 4;
        for (i = 0; i < 5; i++)
        {
          printf("%d ", *k);
          k++;
        }
}
```

O/P:- 10 20 30 40 50

Program 41: Pointers(multiple) and arrays

```
#include<stdio.h>
#include<conio.h>
void main()
{
  static int a[] = {0,1,2,3,4};
  static int *p[] = {a, a+1, a+2, a+3, a+4};
  int **ptr = p;
  clrscr();
  ptr++;
  printf("\n %4d %4d %4d", ptr - p, *ptr -a , **ptr);

  *ptr++;
```

```
 printf("\n %4d %4d %4d", ptr - p, *ptr -a , **ptr);

 *++ptr;
 printf("\n %4d %4d %4d", ptr - p, *ptr -a , **ptr);

 ++*ptr;
 printf("\n %4d %4d %4d", ptr - p, *ptr -a , **ptr);
}
```

O/P:-

```
 1   1   1
 2   2   2
 3   3   3
 3   4   4
```

Program 42: Address of array thru pointers

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int arr[] = {0,1,2,3,4};
        int i;
        int *ptr;
        clrscr();
        for (i = 0; i < 5; i++)
        {
                printf("\t %u", &arr[i]);
        }
        printf("\n");
        for (ptr = arr + 4; ptr >= arr; ptr --)
          printf("\t %u ", &arr[ptr-arr]);
}
```

O/P:- 65516 65518 65520 65522 65524
65524 65522 65520 65518 65516

Program 43: 3'd arrays

```
/* 3'd arrays */
/*
void main()
{
int a[3][4][2] = {
                        {
                                {2,4},   /* 0th 2-d array */
```

```
                {7,8},
                {3,4},
                {5,6}
        },
        {
                {7,6},   /* 1st 2-d array */
                {3,4},
                {5,3},
                {2,3}
        },
        {
                {8,9},
                {7,2},   /* 2nd 2-d array */
                {3,4},
                {5,1}
        }
};

clrscr();
printf("%d ", a[2][3][1]);
getch();
}
```

O/P:- 1

```
*/
```

RECURSION

Program 1: Tower of Hanoi

```
/* Header File */
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <time.h>


void hanoi(int,int,char,char,char);
void move(char,char,char);
void ddraw(void);
int peg[3][50];
int top[3]={0,0,0};
static char disc[13][26];
void InitialDiscGraph (int,int);
int n;




/* Driver program */
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <time.h>
void InitialDiscGraph (int,int);


int main(void)
{
  int i = 0 , j = 0 ;


  InitialDiscGraph (i,j);
```

```
  return 0;

}



/* main program*/
==================== Program description ==================
// This program will solve the hanoi problem and also
// draw the graph for the peg situation. The maximum
// number of discs to process is 50.
//============================================================



#include"tower2.h"


void InitialDiscGraph (int i,int j)  /* Initialize the disc graph */
{

        for (i=0; i<=12; i++)
        {
                for (j=0; j<=11; j++)
                {
                        if (11-j>=i)
                        {
                                disc[i][j]=' ';
          disc[i][24-j]=' ';
         }

                        else
                        {
                                disc[i][j]='*';
          disc[i][24-j]='*';
         }

                }

                disc[i][12]='|';
      disc[i][25]='\0';

        }

   printf(" Please input the number of disc(1-12):");
   scanf("%d",&n);
```

```
printf("Hanoi Tower with %d discs:\n",n);
printf("==================================================\n\n");

for (i=1; i<=n; i++) /* initialize the peg status */
{
                top[0]++;
   peg[0][top[0]]=n-i+1;

      }
ddraw(); /* Draw the initial status */


while (n != 0)
{
                hanoi(n,n,'A','B','C'); /* Do n discs */
   printf(" Please input the number of disc:");
   scanf("%d",&n);
   printf("Hanoi Tower with %d discs:\n",n);
   printf("==================================================\n\n");

                /* initial top */

                for (i=0; i<=2; i++)
                {
                        top[i]=0;
                }

   for (i=0; i<=2; i++) /* Clean up the discs in the pegs */
                {
                        for (j=0; j<=20; j++)
                        {
                                peg[i][j]=0;
                        }


                }

                for (i=1; i<=n; i++)  /* initialize the peg status */
  {
                        top[0]++;
      peg[0][top[0]]=n-i+1;

                }

   if (n!=0)
```

```
      ddraw();  /* Draw the initial status */

        }

        return  ;


}


//*************************************************************
//Main function of hanoi tower program:
//It will move the disc recursively
//hanoi(n, A, B, C) =
//hanoi(n-1, A, C, B) + hanoi(1, A, B, C)
//+hanoi(n-1, B, A, C)
//*************************************************************


void hanoi(int num,int Disc_num,char beg,char aux,char tem)
{
        if (num==1) /* only one disc */
   {
                printf("move disc %d from peg %c to %c \n",Disc_num,beg,tem);
      move(beg,aux,tem); /* update the graph status */

                ddraw(); /* disc status draw */
    }

        else
   {
                hanoi(num-1,Disc_num-1,beg,tem,aux); /* move n-1 disc from beg to
aux */
      hanoi(1,Disc_num,beg,aux,tem);/* move 1 disc from beg to tem
*/
      hanoi(num-1,Disc_num-1,aux,beg,tem);/* move n-1 disc from aux
to tem */
   }
}


//*************************************************************
//Move: move the discs between the pegs by updating
//the top pointer
//*************************************************************
```

```
void move(char beg,char aux,char tem)
{
        if (beg=='A')  /* Move disc from A to B */
   {
                if (tem=='B')
     {
                        top[1]++;
        peg[1][top[1]]=peg[0][top[0]];
        peg[0][top[0]]=0;
        top[0]--;
      }

      else
                {
                        top[2]++;
        peg[2][top[2]]=peg[0][top[0]];
        peg[0][top[0]]=0;
        top[0]--;
      }
    }

        else
   {
                if (beg=='B') /* Move disc from B to A */
     {
                        if (tem=='A')
                        {
                                top[0]++;
          peg[0][top[0]]=peg[1][top[1]];
          peg[1][top[1]]=0;
          top[1]--;
                        }

       else       /* Move disc from B to C */
       {
                         top[2]++;
         peg[2][top[2]]=peg[1][top[1]];
         peg[1][top[1]]=0;
         top[1]--;
        }

     }

     else
     {
                if (tem=='A')  /* Move disc from C to A */
```

```
        {
                              top[0]++;
           peg[0][top[0]]=peg[2][top[2]];
           peg[2][top[2]]=0;
           top[2]--;
        }

       else         /* Move disc from C to B */
       {
                             top[1]++;
          peg[1][top[1]]=peg[2][top[2]];
          peg[2][top[2]]=0;
          top[2]--;
        }

      }

        }

        return;

}


//*********************************************************************
//Draw the disc status
//*********************************************************************

void ddraw(void)
{
        int i = 0;
   int j = 0;
   int k = 0;


        for (i=n; i>=1; i--)
   {
                printf("  ");
      for (j=0; j<=2; j++)
                {
                        for (k=0; k<=25; k++)
                                printf("%c",disc[peg[j][i]][k]);
           printf("  ");
                }

    printf("\n");
```

```
        }

        for (i=0; i<81; i++) printf("-");


   printf("\n\n\n");

        return;

}
```

Program 2: Implement recursive/primitive recursive functions in C

```
/* recursion.c: Implement recursive/primitive recursive functions in C

        This file declares a number of functions that build up to the
        world's slowest implementation of the power function (x^y). The main
      program is just a simple test platform for the power function. If
      you wish, you could build other recursive functions and a different
      test platform once you understand the pattern set out below.

      compile: cc -o recursion -DPROFILE recursion.c

      You could leave off -DPROFILE, but that includes variables that track
      how many times each building block routine is called. You can then
      have your main platform report these values. Otherwise, you just have
      a very slow implementation of simple arithmetic functions.

*/

/* By Terry R. McConnell */

#include <stdio.h>
#include <stdlib.h>

/*
  The idea is that, after defining the base functions and rules, the
  code for the remaining functions represents the proof that they are
  primitive recursive (recursive).

  A reference to the theory of recursive functions, which uses the notation
  we use below, is "Introduction to Mathematical Logic, 3rd Ed," Elliot
```

```
  Mendelson, Wadsworth&Brooks/Cole, 1987, pp 132-149.

  Except for some of the base routines, all functions have the declaration

                int foo(int n, int* args)

  The pointer argument indicates the first in a variable length list of
  integer args. Argument n is the length of the list. Increment args in the
  body of foo to access each succesive variable. The function returns the
  value it computes.

*/


/* The following are used for profiling */
#ifdef PROFILE
int Z_calls;
int N_calls;
int substitute_calls;
int project_calls;
int recurse_calls;
int mu_calls;
int add_calls;
int mult_calls;
int power_calls;
int one_calls;
#endif

/* Base functions */

/* Z: the zero function */

int Z( int n, int *x){
#ifdef PROFILE
        Z_calls++;
#endif
        return 0;       /* This is an easy one */
}

/* N: the successor function */

int N( int n, int *x){
#ifdef PROFILE
        N_calls++;
#endif
        return (*x) + 1;
```

```
}

/* The projection functions. Since there are infinitely many of these, we
   must cheat a bit by including an extra variable in the declaration to
   select which projection. Thus, strictly speaking the following is really
   a schema, or rule, for constructing projection functions. */

int project(int n, int i, int *x){

#ifdef PROFILE
        project_calls++;
#endif
        return x[i-1];
}


/* The composition  rule: we are allowed to form

        f(h_1(x_1...,x_n), h_2(...), ... , h_n(...))

from recursive functions f and h_1, ..., h_n. */

int substitute(int n, int *x, int(*f)(int, int *), int(**h)(int,int *)){

        int *hvals, rval, i;
#ifdef PROFILE
        substitute_calls++;
#endif
        /* Allocate and fill array of return values to pass to f */

        hvals = (int *)malloc(n*sizeof(int));
        for(i=0;i<n;i++)
                hvals[i] = h[i](n,x);

        rval = f(n, hvals);

        /* clean up */
        free(hvals);
        return rval;
}

/* The recursion rule: given recursive functions g and h (data), and
   parameters x_1...x_n, we are allowed to recursively define a function
   f(x_1,...,x_n,y) of n+1 variables as follows:

        f(x_1,...,x_n,0) = g(x_1,...,x_n), and
```

```
        f(x_1,...,x_n,y+1) = h(x_1,...,x_n,y,f(x_1,...,x_n,y)).

   The case of no parameters (n = 0) is allowed.
*/

int recurse(int n, int *x, int y, int(*g)(int,int*), int(*h)(int, int *)){

        int *xyf, rval, i;

#ifdef PROFILE
        recurse_calls++;
#endif
        if(y==0)
                return g(n,x);
        else {
                /* build argument for h */
                xyf = (int *)malloc((n+2)*sizeof(int));
                for(i=0;i<n;i++)xyf[i] = x[i];
                xyf[i++] = y;
                xyf[i] = recurse(n,x,y-1,g,h);  /* here's the recursion! */
                rval = h(n+2,xyf);
                free(xyf);  /* clean up */
                return rval;
        }
}

/* The mu operator: searches (possibly unboundedly,) for the smallest value
  y such that f(x_1,...,x_n,y) = 0. This value must exist.  It defines a
  function of x_1,...,x_n. A function calling this is recursive, but not
  primitive recursive unless there is another way to build it without using mu.
*/

int mu(int n, int *x, int(*f)(int, int *)){

        int *xy, i, rval, y;

#ifdef PROFILE
        mu_calls++;
#endif

        /* allocate and fill args to pass to f */
        xy = (int *)malloc((n+1)*sizeof(int));

        for(i=0;i<n;i++)xy[i] = x[i];
        xy[n]=0; /* initial value of y */
```

```
        while(f(n+1,xy)) xy[n] += 1;  /* search */

        rval = xy[n];
        free(xy);    /* clean up */
        return rval;
}

/* From here on, all functions must use only the base functions, rules above,
   and previously defined functions in their definitions. Those that avoid use
   of mu are primitive recursive. The rules assume, and you must conform to,
   the following declaration for all derived functions foo:

        int foo( int, int *);

   Note that functions are allowed to handle the case of 0 variables by
   returning a constant value.
*/

/* The identity function of 1 variable */

int id(int n, int *x){ /* really only depends on first var */

        return project(1,1,x);
}

/* Helpers for defining addition */
/* last: return last variable.  */

int last(int n, int *x){

        return project(n,n,x);
}

/* inc_4th: return 4th variable + 1. This will be the h function in the
   recursion for add() below. Thus it has 2 variables beyond the 2 arguments
   of add. The last (4th) variable holds the recursively passed function
   value, so that's what gets added to. */

int inc_4th(int n, int *x){

        int rval;
        int (**hs)(int, int*);

        hs = ( int(**)(int,int*) )malloc(4*sizeof(int(*)(int,int*)));
        hs[0]=hs[1]=hs[2]=hs[3]=last;  /* only the last one gets used */
```

```
        rval = substitute(4, x, N, hs);
        free(hs);   /* clean up */
        return rval;
}

/* add(2,x,y): return x + y */

int add(int n, int *x){

#ifdef PROFILE
        add_calls++;
#endif
        return recurse(2,x,x[1],id,inc_4th);
}

/* Similarly, we prepare for multiplication. h data for the multiplication
  recursion. */

int addto4th(int n, int *x){

        int rval;
        int (**hs)(int, int*);

        hs = ( int(**)(int,int*) )malloc(4*sizeof(int(*)(int,int*)));
        hs[0]= id;
        hs[1]=hs[2]=hs[3]=last;  /* only the 3rd gets used */

        rval = substitute(4, x, add, hs);
        free(hs);
        return rval;
}


int mult(int n, int *x){

#ifdef PROFILE
        mult_calls++;
#endif
        return recurse(2,x,x[1],Z,addto4th);
}


/* h data for power recursion */
int multonto4th(int n, int *x){

        int rval;
```

```
        int (**hs)(int, int*);

        hs = ( int(**)(int,int*) )malloc(4*sizeof(int(*)(int,int*)));
        hs[0]= id;
        hs[1]=hs[2]=hs[3]=last;  /* only the 3rd gets used */

        rval = substitute(4, x, mult, hs);
        free(hs);
        return rval;
}

/* The function identically one: base of power recursion */

int one(int n, int *x){

        int rval;
        int (**hs)(int, int*);

#ifdef PROFILE
        one_calls++;
#endif

        hs = ( int(**)(int,int*) )malloc(2*sizeof(int(*)(int,int*)));
        hs[0]= Z;
        hs[1]=id;
        rval =  substitute(2,x,N,hs);
        free(hs);
        return rval;
}

/* power(2,x,y): calculate x^y */

int power(int n, int *x){

#ifdef PROFILE
        power_calls++;
#endif
        return recurse(2,x,x[1],one,multonto4th);
}


/* Test platform */

int main()
{
        int args[2] = {8,4};
```

```
        printf("8 to power 4 is %d\n", power(2,args));
#ifdef PROFILE
        printf("%s\t%s\t%s\t%s\t%s\t%s\n","Z","N","proj","subst","recurse",
         "add");
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",Z_calls, N_calls, project_calls,
                substitute_calls, recurse_calls, add_calls);
#endif
        return 0;
}
```

Program 3 : Queues using recursion

```
#include<stdio.h>
#include<process.h>

#define QUEUE_SIZE 5

int qfull(int r);
void insert_rear(int item, int q[], int *t);
int qempty(int f, int r);
void delete_front(int q[], int *f, int *r);
void display(int q[], int f, int r);

void main()
{
  int choice, item, f, r, q[10];
  f = 0;
  r = -1;
  for(;;)
  {
    printf("1 : Insert  , 2 :Delete , 3 : Display 4: Exit\n");
    printf("Enter choice");
    scanf("%d", &choice);
    switch(choice)
    {
        case 1 :
                printf("Enter the item to be inserted \n");
                scanf("%d", &item);
                insert_rear(item, q, &r);
                break;
        case 2 :
                delete_front(q, &f, &r);
                break;
        case 3 :
```

```
                display(q,f,r);
                break;
        default :
                exit(0);
   }
}
}

/* function to check whether queue is full */
int qfull(int r)
{
  return (r == QUEUE_SIZE - 1) ? 1 : 0;
}

/* function to insert at rear end of queue */
void insert_rear(int item, int q[], int *r)
{
 if (qfull(*r))
 {
        printf("Queue overflow\n");
        return;
 }
 q[++(*r)] = item;
}

/* function to check for underflow of queue */
int qempty(int f, int r)
{
        return(f > r) ? 1 : 0;
}

/* function to delete from the front end */
void delete_front(int q[], int *f, int *r)
{
        if (qempty(*f, *r))
        {
                printf("Queue undeflow");
                return;
        }
        printf("The elements deleted are %d\n", q[(*f)++]);

        if (*f > *r)
        {
                *f = 0, *r = -1;
        }
}
```

```
/* function to display the contents of the queue */
void display(int q[], int f, int r)
{
        int i;
        if (qempty(f,r))
        {
                printf("Queue is empty\n");
                return;
        }
        printf("Contents of queue is \n");
        for (i = f; i <= r; i++)
                printf("%d\n", q[i]);
}
```

Program 4:Queue Strings implimentaton

```
#include<string.h>
#include<stdlib.h>
#include<stdio.h>
#include<ctype.h>

#define MAX 100

char *p[MAX], *qretrieve();

int spos , rpos ;

void enter();
void qstore(char *q);
void review();
void delete_ap();

void main()
{
        char s[80];
        int t;

    clrscr();
for (;;)
{
        printf("[E]nter, [L]ist, [R]emove, [Q]uit : ");
        gets(s);
        *s = toupper(*s);
```

```
        switch(*s)
        {
          case 'E' :
                enter();
                break;
          case 'L' :
                review();
                break;
          case 'R' :
                delete_ap();
                break;
          case 'Q' :
                exit(0);
        }
}

}

void enter()
{
        char s[256], *p;
        do
        {
          printf("Enter appointment %d : ", spos + 1);
          gets(s);

          if (*s == 0)
                break;
          p = (char * ) malloc(strlen(s) + 1);
          if (!p)
          {
                printf("Out Of Memory\n");
                return;
          }
          strcpy(p, s);
          if (*s)
                qstore(p);
          } while(*s);
}

void review()
{
  int t;
  for (t = rpos; t < spos; ++t)
  printf("%d. %s\n", t+1, p[t]);
```

```
}

void delete_ap()
{
  char *p;
  if ((p=qretrieve()) == NULL)
          return;
  printf("%s\n", p);
}

void qstore(char *q)
{
          if (spos == MAX)
          {
                    printf("List Full\n");
                    return;
          }
          p[spos] = q;
          spos++;
}
char *qretrieve()
{
  if (rpos == spos)
  {
          printf("No More Appointments \n");
          return NULL;
  }
rpos++;
return p[rpos-1];
}
```

Program 5:Product of two nos using Recursion

```
#include<stdio.h>
#include<conio.h>
int mul(int, int);
void main()
{
          int m,n;
          clrscr();
          printf("Enter Value  of m and n:\n");
          scanf("%d %d", &m, &n);
          printf("Product (%d, %d) = %d\n",m, n, mul(m,n));
          getch();
```

```
}

int mul(int m, int n)
{

        if (m == 0 || n == 0) return 0;

        if (n == 1) return m;

        return mul(m,n-1) + m;
}
```

O/P:- Enter Value of m and n:
9
10
Product (9, 10) = 90

Program 6: GCD using recursion

```
#include<stdio.h>
#include<conio.h>
/* The procedure to find GCD of two numbers m and n can be obtained
as shown below. It is clear from teh table that whenever m is greater
than n computer m - n and asssign it to m and if m is less than n,
exchange m and n. Repeat the above process till m and n are equal.
When m and n are equal we display either m or n which is the GCD
of the given two numbers
_____________________________________________
m  |  n | m(>, <, =) n                |
_____________________________________________|
10 |  6 | 10 > 6 so m = 10 - 6 = 4     |
-----------------------------------------
4  |  6 | 4 < 6 so exchange m and n    |
_____________________________________________|
6  |  4 | 6 > 4 so m = 6 - 4 = 2       |
_____________________________________________|
2  |  4 | 2 < 4 so exchange m and n    |
_____________________________________________|
4  |  2 | 4 > 2 so m = 4 - 2 = 2       |
_____________________________________________|
2  |  2 | m and n are same So, GCD is 2 |
_____________________________________________| */
```

```
int gcd(int, int);
void main()
{
        int m, n;
        clrscr();
        printf("Enter 2 Nos : ");
        scanf("%d %d" , &m, &n);
        printf("GCD(%d %d) = %d" , m,n, gcd(m,n));
        getch();
}

int gcd(int m, int n)
{
        int temp;

        while (m != n)
        {
          if (m > n)
                m = m - n;
        else
                temp = m, m = n, n = temp;
        }
        return m;
}
```

O/P:-Enter 2 Nos : 118
56
GCD(118 56) = 2

Program 7: Fibonocci Using Recursion

```
#include<stdio.h>
#include<conio.h>
int fact(int);
void main()
{
        int n;
        clrscr();
        printf("Enter Value : ");
        scanf("%d", &n);
        printf("The fibonocci of %d = %d\n", n, fib(n));
        getch();
}

int fib(int n)
{
```

```
        if (n == 1)
                return 0;
        if (n == 2)
                return 1;

        return fib(n-1) + fib(n-2);
}
```

O/P:- Enter Value : 5
The fibonocci of 5 = 3

Program 8:Factorial Using Recursion

```
#include<stdio.h>
#include<conio.h>
int fact(int);
void main()
{
        int n, a;
        clrscr();
        printf("Enter Value : ");
        scanf("%d", &n);
        a = fact(n);
        printf("The factorial of %d = %d\n", n, a);
        getch();
}

int fact(int n)
{
        int x, y, res = 0;

        if (n == 0)
                return 1;

        x = n - 1;
        y = fact(x);
        res = n * y;

        return res;
}
```

O/P:- Enter Value : 4
The factorial of 4 = 24

Program 11: Factorial Using Recursion(Alternate and better)

```
#include<stdio.h>
#include<conio.h>
```

```
int fact(int);
void main()
{
        int n;
        clrscr();
        printf("Enter Value : ");
        scanf("%d", &n);
        printf("The factorial of %d = %d\n", n, fact(n));
        getch();
}

int fact(int n)
{
        if (n == 0)
                return 1;

                return n * fact (n - 1);
}
```

O/P:- Enter Value : 4
The factorial of 4 = 24

Program 12 : Reversing a string using recursion

```
#include<stdio.h>
void reverse();
#define EOLN '\n'
void reverse();
main()
{
        clrscr();
        printf("Enter a line of text : ");
        reverse();
}

void reverse()
{
        char c;
        if ((c = getchar()) != EOLN) reverse();
        putchar(c);
}
```

O/P:- Enter a line of text : abhishek

kehsihba

SORTINGS AND SEARCHINGS

Shell Sort

```
#include<stdio.h>
#include<conio.h>
void shell_sort(int n, int a[]);
void main()
{
  int arr[10] = { 34, 94, 63, 78, 3, 8, 9, 22, 14, 29};
  int n = 10;
  int i;
  printf("\n Array before sorting ");
  for (i = 0; i < 10; i++)
  {
    printf("%4d", arr[i]);
  }

  shell_sort(n,arr);
  printf("\n Sorted Array ");
  for (i = 0; i < 10; i++)
  {
    printf("%4d", arr[i]);
  }
}
void shell_sort(int n, int a[])
{
  int i, j, gap, item;
  for (gap = (n-1)/2; gap > 0; gap /= 2)
  {
   for (i = 0; i < n; i+= gap)
   {
     item = a[i];
     j = i - gap;

     while (j >= 0 && item < a[j])
     {
          a[j+gap] = a[j];
          j = j - gap;
     }
     a[j+gap]= item;
   }
```

```
  }
}
```

Shell sort

```
#include<stdio.h>
void selection_sort(int a[], int n);
void main()
{
  int i, n, a[20];
  printf("Enter the number of elements to sort\n");
  scanf("%d", &n);

  printf("Enter %d elements to sort\n", n);
  for (i= 0; i < n; i++)
  scanf("%d", &a[i]);
  selection_sort(a,n);

  printf("The sorted elements are\n");
  for (i = 0; i < n; i++)
  printf("%4d", a[i]);
}

void selection_sort(int a[], int n)
{
  int i, j, pos, small, temp;

  for (i =0 ; i < n -1; i++)
  {
     small = a[i];
     pos = i;

     for (j = i + 1; j < n; j++)
     {
           if (a[j] < small)
           {
              small = a[j];
              pos = j;
           }
     }
     temp = a[pos];
     a[pos] = a[i];
     a[i] = temp;
```

```
  }
}
```

Quick Sort

```
#include<stdio.h>
#include<conio.h>

void quicksort(int a[], int low, int high);
int a[5] = { 55, 1, 78, 13, 45};

void main()
{
  int i, n;
  /*printf("\nOriginal array");
  for(i=0; i < 5; i++)
  printf("%4d", a[i]);*/

  quicksort(a, 0, 4);

  printf("\nThe sorted array is \n");
  for(i=0; i < 5; i++)
  printf("%4d", a[i]);
}
void quicksort(int a[], int low, int high)
{
  int j;
  if (low < high)
  {
     j = partition(a, low, high);
     quicksort(a, low, j - 1);
     quicksort(a, j+1, high);
  }
}
int partition(int a[], int low, int high)
{
  int i, j, temp, key;

  key = a[low];
  i = low + 1;
  j = high;

  while(1)
  {
    while (i < high && key >= a[i])
      i++;
    while (key < a[j])
```

```
    j--;

    if (i < j)
    {
     temp = a[i];
     a[i] = a[j];
     a[j] = temp;
    }
    else
     {
          temp = a[low];
          a[low] = a[j];
          a[j] = temp;
     }
          return j;
     }
  }
```

Quick Sort

```
#include<stdio.h>
#include<conio.h>

void quicksort(int *, int, int);
int split(int *, int, int);
int arr[10] = {11, 2, 9, 13, 57, 25, 17, 1, 90, 3};

void main()
{

   int i;
   clrscr();

   printf("Quick Sort\n");
   printf("\nArray before sorting:");

   for (i=0; i < 10; i++)
          printf("%4d", arr[i]);

   quicksort(arr,0,9);
   printf("\n Array after sorting:");

   for(i = 0; i < 10; i++)
```

```
   printf("%4d", arr[i]);

   getch();
}

void quicksort(int a[], int lower, int upper)
{
   int i;
   if (upper > lower)
   {
      i = split(a, lower,upper);
      quicksort(a, lower, i -1);
      quicksort(a, i + 1, upper);
   }
}

int split(int a[], int lower, int upper)
{
  int i, p, q, t;

  p = lower + 1;
  q = upper;
  i = a[lower];

  while (q >= p)
  {
    while(a[p] < i)
     p++;

    while(a[q] > i)
     q--;

    if (q > p)
    {
      t = a[p];
      a[p] = a[q];
      a[q] = t;
    }
  }
  t = a[lower];
  a[lower] = a[q];
  a[q] = t;

  return q;
}
```

Linear Search

```
#include<stdio.h>
int linear(int, int, int a[]);
void main()
{
  int a[] = {4, 9, 12, 89, 87, 78};
  int key, pos;
  clrscr();
  printf("Enter the value to be searched :");
  scanf("%d", &key);
  pos = linear(key,6,a);

  if (pos == -1)
        printf("Element not found");
  else
        printf("Element found at %d\n", pos);
  getch();
}
int linear(int key, int n, int a[])
{
        if (n < 0)
                return - 1;
        if (key == a[n-1])
                return n;

        return linear(key, n-1, a);
}
```

Linear Search

```
#include<stdio.h>
int linear(int, int, int a[]);
void main()
{
  int a[] = {4, 9, 12, 89, 87, 78};
  int key, pos;
  clrscr();
  printf("Enter the value to be searched :");
  scanf("%d", &key);
  pos = linear(key,6,a);
```

```
  if (pos == -1)
        printf("Element not found");
  else
        printf("Element found at %d\n", pos);
  getch();
}
int linear(int key, int n, int a[])
{
        int i;
        for (i = 0; i < n; i++)
        {
                if (key == a[i]) return i+1;
                if (key < a[i]) return -1;
        }
        return -1;
}
```

Insertion Sort

```
#include<stdio.h>
#include<conio.h>

void main()
{
        int arr[5] = {25, 17, 31, 13, 2};
        int i, j, k, temp;

        clrscr();

        printf("\nInsertion Sort");
        printf("\nArray before sorting");

        for (i = 0; i < 5; i++)
        printf("%4d", arr[i]);

        for (i = 1; i < 5; i++)
        {
          for (j = 0; j < i; j++)
          {
            if (arr[j] > arr[i])
            {
              temp = arr[j];
              arr[j] = arr[i];
```

```
                for (k = i; k > j; k--)
                      arr[k] = arr[k - 1];

                arr[k + 1] = temp;
              }
             }
           }
           printf("\nArray after sorting\n");
           for (i = 0; i < 5; i++)
           printf("%4d", arr[i]);

           getch();
}
```

Insertion Search

```
#include<stdio.h>
void insertion_sort(int a[], int n);
void main()
{
   int i, a[] = { 22, 9, 14, 5, 75, 74};
   insertion_sort(a,6);
   printf("The Sorted array is : ");
   for (i = 0; i < 6; i++)
   printf("%4d", a[i]);
}

void insertion_sort(int a[], int n)
{
   int i, j, key;

   for (i = 0; i < n;i++)
   {
          key = a[i];
          j = i - 1;

          while (j >= 0 && key < a[j])
          {
                    a[j+1] = a[j];
                    j--;
          }
          a[j+1] = key;
   }
}
```

Insertion Search

```
#include<stdio.h>
int search(int , int a[], int, int);
void main()
{
 int i, a[] = {3, 5, 7, 9, 14, 22, 54, 74, 89, 96};
 int key, pos;

 printf("Enter the element to be searched");
 scanf("%d", &key);

 pos = search(key, a, 0, 9);

 if (pos == -1)
        printf("Key element not found");
 else
        printf("key element found at %d position", pos);
}
int search(int key, int a[], int low, int high)
{
   int mid;

   if (low > high)
        return - 1;

   mid = low +(high - low) * ( (key - a[low]) / (a[high] - a[low]));

   return(key == a[mid] ? mid + 1 : key < a[mid] ? search(key, a, low, mid -1) : search(key, a,
mid + 1, high));
}
```

Insertion Search

```
/* insertion sort using strings */

#include<string.h>
#include<stdio.h>
#include<stdlib.h>

void insert(char *items, int count);

void main()
{
        char s[255];
        clrscr();
        printf("\nEnter a string : ");
        gets(s);
        insert(s, strlen(s));
        printf("\nThe sorted string is : %s", s);
        getch();
}

void insert(char *items, int count)
{
        int a, b;
        char t;

        for (a = 1; a < count; a++)
        {
         t = items[a];
        for (b = a - 1; (b >= 0) && (t < items[b]); b--)
          items[b+1] = items[b];
            items[b+1] = t;
        }
}
```

Bubble Sort

```
/* bubble sort using strings */

#include<string.h>
```

```
#include<stdio.h>
#include<stdlib.h>

void bubble(char *items, int count);

void main()
{
        char s[255];
        clrscr();
        printf("\nEnter a string : ");
        gets(s);
        bubble(s, strlen(s));
        printf("\nThe sorted string is : %s", s);
        getch();
}

void bubble(char *items, int count)
{
        int a, b;
        char t;

        for (a = 1; a < count; a++)
        for (b = count - 1; b >= a; b--)
        {
          if(items[b-1] > items[b])
          {
            t = items[b-1];
            items[b-1] = items[b];
            items[b] = t;
          }
        }
}
```

Bubble Sort

```
/* Program to implement bubble sort in ascending order*/
#include<stdio.h>
void bubble_sort(int a[], int n); /* function prototype */
void main()
{
        int i, n, a[20];
        clrscr();
        printf("Enter the number of values to sort\n");
        scanf("%d", &n);
```

```
        printf("Enter the values to sort\n");
        for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
        bubble_sort(a, n);

        printf("The sorted items are \n");
        for (i = 0; i < n; i++)
        {
                printf("%4d", a[i]);
        }
}
void bubble_sort(int a[], int n)
{
        int i /* to access subsequent values while comparing */;
        int j /* keep track of the passes */;
        int temp /* variable used to exchange values */;
        int sum /* variable to hold number of exchanges*/;
        int pass/* variable to hold the number of passes required*/;
        int exchg /* to hold number of exchanges in each pass */;
        int flag /* indicates wether exchange happend or not */;

        sum = pass = 0;

        for (j = 1; j < n; j++)
        {
          exchg = 0;
          flag = 0;

          for (i = 0; i < n - j; i++)
          {
            if (a[i] >= a[i+1])
            {
                  temp = a[i];
                  a[i] = a[i+1];
                  a[i+1] = temp;
                  exchg++;
                  sum++;
                  flag = 1;
            }
          }
          pass++;
          printf("No of exchanges in pass %d = %d\n", j, exchg);
          if (flag == 0)
                break;
          }
          printf("Total number of passes = %d\n", pass);
```

```
        printf("Total number of exchanges = %d\n", sum);
}
```

Binary Search

```
#include<stdio.h>
int binary(int , int a[], int, int);
void main()
{
 int i, a[] = {3, 5, 7, 9, 14, 22, 54, 74, 89, 96};
 int key, pos;

 printf("Enter the element to be searched");
 scanf("%d", &key);

 pos = binary(key, a, 0, 9);

 if (pos == -1)
        printf("Key element not found");
 else
        printf("key element found at %d position", pos);
}
int binary(int key, int a[], int low, int high)
{
   int mid;

   if (low > high)
        return - 1;

   mid = (low + high) / 2;

  return(key == a[mid] ? mid : key < a[mid] ? binary(key, a, low, mid -1) : binary(key, a, mid + 1,
high));
}
```

STACKS AND QUEUES

Stack 1

```
#include<stdio.h>
#include<process.h>
#define STACK_SIZE 5
void push(int item, int *top, int s[]);
int pop(int *top, int s[]);
void display(int top, int s[]);
void main()
{
        int top;   /* Points to top of the stack  */
        int s[10]; /* Holds the stack items */
        int item;  /* Item to be inserted or deleted */
        int choice; /* User choice for push, pop & display */

        top = -1;   /* stack is empty to begin with */

        for (;;)
        {
                printf("1 : Push\n");
                printf("2 : PoP\n");
                printf("3 : DisplaY\n");
                printf("4 : ExiT\n");
                printf("\nEnter choice \n");
                scanf("%d", &choice);

                switch(choice)
                {
                        case 1 :
                                printf("Enter the items to be inserted\n");
                                scanf("%d", &item);
                                push(item, &top, s);
                                break;
                        case 2 :
                                item = pop(&top, s);
                                if (item == 0)
                                  printf("Stack is empty\n");
                                else
                                  printf("Item Deleted = %d\n", item);
                                  break;
```

```
                        case 3 :
                                display(top, s);
                                break;
                        default :
                                exit(0);

                }
        }
}

void push(int item, int *top, int s[])
{
        if (*top == STACK_SIZE - 1)
        {
                printf("Stack Overflow\n");
                return;
        }
        s[++(*top)] = item;
}

int pop(int *top, int s[])
{
        int item;

        if (*top == -1)
        {
                return 0;
        }
        item = s[(*top)--];
        return item;
}

void display(int top, int s[])
{
        int i;
        if (top == -1)
        {
                printf("Stack is empty\n");
                return;
        }
        printf("Contents of the stack\n");
        for (i = 0; i <= top; i++)
        {
                printf("%d\n", s[i]);
        }
}
```

Queues 1

```
#include<stdio.h>
#include<process.h>

#define QUEUE_SIZE 5

int qfull(int r);
void insert_rear(int item, int q[], int *t);
int qempty(int f, int r);
void delete_front(int q[], int *f, int *r);
void display(int q[], int f, int r);

void main()
{
  int choice, item, f, r, q[10];
  f = 0;
  r = -1;
  for(;;)
  {
    printf("1 : Insert  , 2 :Delete , 3 : Display 4: Exit\n");
    printf("Enter choice");
    scanf("%d", &choice);
    switch(choice)
    {
          case 1 :
                    printf("Enter the item to be inserted \n");
                    scanf("%d", &item);
                    insert_rear(item, q, &r);
                    break;
          case 2 :
                    delete_front(q, &f, &r);
                    break;
          case 3 :
                    display(q,f,r);
                    break;
          default :
                    exit(0);
    }
}
}
```

```
/* function to check whether queue is full */
int qfull(int r)
{
  return (r == QUEUE_SIZE - 1) ? 1 : 0;
}

/* function to insert at rear end of queue */
void insert_rear(int item, int q[], int *r)
{
 if (qfull(*r))
 {
          printf("Queue overflow\n");
          return;
 }
 q[++(*r)] = item;
}

/* function to check for underflow of queue */
int qempty(int f, int r)
{
          return(f > r) ? 1 : 0;
}

/* function to delete from the front end */
void delete_front(int q[], int *f, int *r)
{
          if (qempty(*f, *r))
          {
                    printf("Queue undeflow");
                    return;
          }
          printf("The elements deleted are %d\n", q[(*f)++]);

          if (*f > *r)
          {
                    *f = 0, *r = -1;
          }
}
/* function to display the contents of the queue */
void display(int q[], int f, int r)
{
          int i;
          if (qempty(f,r))
          {
                    printf("Queue is empty\n");
                    return;
```

```
        }
        printf("Contents of queue is \n");
        for (i = f; i <= r; i++)
                printf("%d\n", q[i]);
}
```

Queues 2

```
#include<string.h>
#include<stdlib.h>
#include<stdio.h>
#include<ctype.h>

#define MAX 100

char *p[MAX], *qretrieve();

int spos , rpos ;

void enter();
void qstore(char *q);
void review();
void delete_ap();

void main()
{
        char s[80];
        int t;

     clrscr();
for (;;)
{
        printf("[E]nter, [L]ist, [R]emove, [Q]uit : ");
        gets(s);
        *s = toupper(*s);

        switch(*s)
        {
          case 'E' :
                enter();
                break;
          case 'L' :
                review();
                break;
```

```
            case 'R' :
                    delete_ap();
                    break;
            case 'Q' :
                    exit(0);
          }
}

}

void enter()
{
          char s[256], *p;
          do
          {
            printf("Enter appointment %d : ", spos + 1);
            gets(s);

            if (*s == 0)
                    break;
            p = (char * ) malloc(strlen(s) + 1);
            if (!p)
            {
                    printf("Out Of Memory\n");
                    return;
            }
            strcpy(p, s);
            if (*s)
                    qstore(p);
            } while(*s);
}

void review()
{
   int t;
   for (t = rpos; t < spos; ++t)
   printf("%d. %s\n", t+1, p[t]);
}

void delete_ap()
{
  char *p;
  if ((p=qretrieve()) == NULL)
          return;
  printf("%s\n", p);
}
```

```
void qstore(char *q)
{
        if (spos == MAX)
        {
                printf("List Full\n");
                return;
        }
        p[spos] = q;
        spos++;
}
char *qretrieve()
{
  if (rpos == spos)
  {
        printf("No More Appointments \n");
        return NULL;
  }
rpos++;
return p[rpos-1];
}
```

STRUCTURES

Program 1:Basics fo Union using int86 and REGS

```
#include<dos.h>
#include<stdio.h>
void main()
{
        union REGS inregs, outregs;

        inregs.h.ah =  2;
        inregs.h.dh = 10;
        inregs.h.dl =  2;

        int86(0x10, &inregs, &outregs);
        printf("Hello there!");
}
```

Program 2: Struct and Unions interupts

```
#include<stdio.h>
#define VIDEO 0x12
void main()
{
        struct WORDREGS
        {
                unsigned int ax;
                unsigned int bx;
                unsigned int cx;
                unsigned int dx;
                unsigned int si;
                unsigned int di;
                unsigned int flags;
        };
        struct BYTEREGS
        {
                unsigned char al, ah;
                unsigned char bl, bh;
                unsigned char cl, ch;
                unsigned char dl, dh;
        };
        union REGS
        {
```

```
                struct WORDREGS x;
                struct BYTEREGS h;
        };
        union REGS regs;
        int size;
        int86(VIDEO, &regs, &regs);
        size = regs.x.ax;
        clrscr();
        printf("Memory size is %d kbytes", size);
        getch();
}
```

O/P:- Memory size is 640 kbytes

Program 3: Interupts

```
#include<dos.h>
#define CURSIZE 1                       /* set cursor size service */
#define VIDEO 0x10                      /* video bios interrupt no */
#define STOPBIT 0x20                            /* this bit turns cursor off */
void main()
{
        union REGS regs;
        regs.h.ch = STOPBIT;            /* turn off cursor */
        regs.h.ah = CURSIZE;           /* service number */
        int86(VIDEO, &regs, &regs);     /* call video interrupt */
}
```

O/p:-

program 4: Structures within Unions

```
#include<stdio.h>
void main()
{
        struct twoints
        {
          int intnum1;
          int intnum2;
        } st;
        union intflo
        {
         struct twoints st;
         float fltnum;
        } unex;
```

```
        printf("sizeof(union intflo) = %d\n", sizeof(union intflo) );
        unex.st.intnum1 = 234;
        unex.st.intnum2 = 456;

        printf("unex.st.intnum1 = %d\n", unex.st.intnum1);
        printf("unex.st.intnum2 = %d\n", unex.st.intnum2);

        unex.fltnum = 657.23;
        printf("unex.fltnum = %f\n", unex.fltnum);
}
```

O/P:- sizeof(union intflo) = 4
unex.st.intnum1 = 234
unex.st.intnum2 = 456
unex.fltnum = 657.229980

Program 5:Unions

```
#include<stdio.h>
void main()
{
        union intflo
        {
                int intnum;
                float fltnum;
        } unex;

        printf("sizeof (union intflo) = %d\n", sizeof(union intflo) );
        unex.intnum = 734;
        printf("unex.intnum=%d\n",  unex.intnum);
        unex.fltnum= 867.43;
        printf("unex.fltnum=%.2f\n", unex.fltnum);
        getch();
}
```

O/P:- sizeof (union intflo) = 4
unex.intnum=734
unex.fltnum=867.43

Program 6: Records for agents and inches

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>

#define TRUE 1
```

```
struct personnel
{
  char name[30];
  int agnumb;
  float height;
};

void newname();
void listall();

struct personnel agent[50];
int n = 0;

void main()
{
          char ch;
          clrscr();
          while (TRUE)
          {
            printf("\n 'e' to enter new agent,");
            printf("\n 'l' to list all agents,");
            printf("\n 'q' to quit : ");
            ch = getche();
            switch (ch)
            {
              case 'e' :
                 newname();
                 break;
              case 'l' :
                    listall();
                    break;
              case 'q' :
                    exit(0);
              default :
                    puts("\nEnter only selections listed");
              }
           }
}
void newname()
{
   char numstr[81];
   printf("\nEnter name   : ");
   gets(agent[n].name);
   printf("Enter number : ");
   gets(numstr);
```

```
   agent[n].agnumb = atoi(numstr);
   printf("Enter height in inches : ");
   gets(numstr);
   agent[n++].height = atof(numstr);
}
void listall()
{
   int j;
   if (n < 0)
          printf("List is empty");

   for (j = 0; j < n ; j++)
   {
      printf("\n Record number %d\n", j+1);
      printf("Name %s\n", agent[j].name);
      printf("Number %d\n", agent[j].agnumb);
      printf("Height %4.2f\n", agent[j].height);
   }
}
```

O/P:- 'e' to enter new agent,
'l' to list all agents,
'q' to quit : e
Enter name : ab
Enter number : 1
Enter height in inches : 12

'e' to enter new agent,
'l' to list all agents,
'q' to quit : l
Record number 1
Name ab
Number 1
Height 12.00

'e' to enter new agent,
'l' to list all agents,
'q' to quit :

Program 7: Agent name and no

```
#include<stdio.h>
#include<conio.h>

struct per
{
  char name[30];
  int agnumb;
};

struct per newname();
void list (struct per);

void main()
{
          struct per ag1;
          struct per ag2;
          clrscr();
          ag1 = newname();
          ag2 = newname();

          list(ag1);
          list(ag2);
}

struct per newname()
{
   char numstr[81];
   struct per agent;
   printf("\nEnter Name : ");
   gets(agent.name);
   printf("Agent number : ");
   gets(numstr);
   agent.agnumb = atoi(numstr);
   return(agent);
}
void list(struct per agex)
{
   printf("\nAgent : \n");
   printf("Name   : %s\n",  agex.name);
   printf("Number : %d\n",  agex.agnumb);
}
```

Program 8: Example on Struct

```
#include<stdio.h>
#include<conio.h>
struct per
{
  char name[30];
  int agnumb;
};
struct team
{
  struct per cheif;
  struct per ind;
};
struct team team1 = {
                        {"Dravid", 29},
                        {"Sachin", 30}
                    };
void main()
{
  clrscr();
  printf("Cheif : \n");
  printf("Name   : %s \n", team1.cheif.name);
  printf("Number : %d \n", team1.cheif.agnumb);

  printf("Ind : \n");
  printf("Name   : %s \n", team1.ind.name);
  printf("Number : %d \n", team1.ind.agnumb);
}
```

Program 9:Example2 assigning a Struct to another

```
#include<stdio.h>
#include<conio.h>
void main()
{
        struct personnel
        {
          char name[30];
          int agnumb;
        }agent2;

        struct personnel agent1 = { "Dravid", 29 };
        agent2 = agent1;
```

```
        clrscr();

        printf("\n List of agents:\n");
        printf("Name   : %s\n", agent1.name);
        printf("Number : %d\n", agent1.agnumb);

        printf("Name   : %s\n", agent2.name);
        printf("Number : %d\n", agent2.agnumb);

}
```

Program 10: initializing struct

```
#include<stdio.h>
#include<conio.h>
void main()
{
        struct personnel
        {
          char name[30];
          int agnumb;
        };
        struct personnel agent1 = { "Dravid", 29 };
        struct personnel agent2 = { "Sachin", 30 };

        printf("\n List of agents:\n");
        printf("Name   : %s\n", agent1.name);
        printf("Number : %d\n", agent1.agnumb);

        printf("Name   : %s\n", agent2.name);
        printf("Number : %d\n", agent2.agnumb);

}
```

Program 11: Reading values to struct items

```
#include<stdio.h>
#include<conio.h>
void main()
{
        struct personnel
        {
          char name[30];
```

```
    int agnumb;
  } agent1, agent2;

  char numstr[81];

  clrscr();

  printf("\nAgent 1. \nEnter name : ");
  gets(agent1.name);
  printf("\nEnter agent number : ");
  gets(numstr);

  agent1.agnumb = atoi(numstr);

  printf("\nAgent 2. \nEnter name : ");
  gets(agent2.name);
  printf("\nEnter agent number : ");
  gets(numstr);

  agent2.agnumb = atoi(numstr);

  printf("\n List of agents:\n");
  printf("Name   : %s\n", agent1.name);
  printf("Number : %d\n", agent1.agnumb);

  printf("Name   : %s\n", agent2.name);
  printf("Number : %d\n", agent2.agnumb);

}
```

Program 12: Item initialization of struct

```
#include<stdio.h>
#include<conio.h>
void main()
{
  struct easy
  {
    int num;
    char ch;
  } ez1 ,ez2;

  clrscr();
```

```
        ez1.num = 2;
        ez1.ch = 'A';

        ez2.num = 10;
        ez2.ch = 'S';

        printf("ez1.num = %d", ez1.num);
        printf("\nez1.ch = %c", ez1.ch);

        printf("\nez2.num = %d", ez2.num);
        printf("\nez2.ch = %c", ez2.ch);
}
```

Program 13: Example 3

```
#include<stdio.h>
#include<conio.h>
void main()
{
        struct easy
        {
          int num;
          char ch;
        };
        struct easy ez1;
        ez1.num = 2;
        ez1.ch = 'A';

        printf("ez1.num = %d", ez1.num);
        printf("\nez1.ch = %c", ez1.ch);
}
```

program 14: Use of enum

```
#include<stdio.h>
void main()
{
enum empcats { management, research, clerical, sales };
struct
{
        char name[30];
        float salary;
```

```
        enum empcats category;
} employee;

clrscr();

strcpy(employee.name, "Kapil Dev");
employee.salary = 118.45;

printf("Name = %s\n", employee.name);
printf("Salary = %6.2f\n", employee.salary);
printf("Category = %d\n", employee.category);

if (employee.category == clerical)
        printf("Employee category is clerical \n");
else
        printf("Employee category is not clerical \n");
}
```

O/P:- Name = Kapil Dev
Salary = 118.45
Category = -28837
Employee category is not clerical

Program 15: CUSTOMER BILLING SYSTEM

```
#include<stdio.h>

void readinput(int i);
void writeoutput(int i);

struct date
{
        int month;
        int day;
        int year;
};
struct account
{
        char name[80];
        char street[80];
        char city[80];
        int acct_no;
        char acct_type;
        float oldbalance;
        float newbalance;
```

```
        float payment;
        struct date lastpayment;
} customer[100];

void main()
{
        int i, n;
        clrscr();
        printf("CUSTOMER BILLING SYSTEM\n\n");
        printf("How many customers are there ?");
        scanf("%d", &n);

        for (i = 0; i < n; ++i)
        {
                readinput(i);

        if (customer[i].payment > 0)
          customer[i].acct_type = (customer[i].payment < 0.1 * customer[i].oldbalance) ? 'O' :
'C';
        else
          customer[i].acct_type = (customer[i].oldbalance > 0) ? 'D' : 'C';
customer[i].newbalance  = customer[i].oldbalance - customer[i].payment;
}
clrscr();
        for (i = 0; i < n; ++i)
        writeoutput(i);
        getch();
}

void readinput(int i)
{
        printf("\nCustomer no %d\n", i + 1);
        printf(" Name : ");
        scanf("%s", customer[i].name);
        printf(" Street : ");
        scanf("%s", customer[i].street);
        printf(" City : ");
        scanf("%s", customer[i].city);
        printf("Account number ");
        scanf("%d", &customer[i].acct_no);
        printf("Previous balance : ");
        scanf("%f", &customer[i].oldbalance);
        printf("Current payment : ");
        scanf("%f", &customer[i].payment);
        printf("Payment date (dd/mm/yy) : ");
        scanf("%d/%d/%d", &customer[i].lastpayment.day,
```

```
                        &customer[i].lastpayment.month,
                        &customer[i].lastpayment.year);
        return;
}

void writeoutput(int i)
{
        printf("\nName   : %s", customer[i].name);
        printf("\nAccount number : %d", customer[i].acct_no);
        printf("\nPrevious balance : %.2f", customer[i].oldbalance);
        printf("\nCurrent payment : %.2f", customer[i].payment);
        printf("\nNew Balance : %.2f", customer[i].newbalance);
        printf("\nAccount Status : ");
        switch(customer[i].acct_type)
        {
          case 'C' :
                printf("Current\n\n");
                break;
          case 'O' :
                printf("OverDue\n\n");
                break;
          case 'D' :
                printf("Suspended\n\n");
                break;
          default :
                printf("Error\n\n");
          }
        return;
}
```

Program 16: use of typedef and passing structures thro functions for CUSTOMER BILLING SYSTEM

```
/* use of typedef and passing structures thro functions */

#include<stdio.h>
typedef struct
{
        int month;
        int day;
        int year;
}date;

typedef struct
{
```

```
        char name[80];
        char street[80];
        char city[80];
        int acct_no;
        char acct_type;
        float oldbalance;
        float newbalance;
        float payment;
         date lastpayment;
}record;

record readinput(int i);
void writeoutput(record customer);
void main()
{
        int i, n;
        record customer[100];
        clrscr();
        printf("CUSTOMER BILLING SYSTEM\n\n");
        printf("How many customers are there ?");
        scanf("%d", &n);

        for (i = 0; i < n; ++i)
        {
                customer[i] = readinput(i);

        if (customer[i].payment > 0)
          customer[i].acct_type = (customer[i].payment < 0.1 * customer[i].oldbalance) ? 'O' :
'C';
        else
          customer[i].acct_type = (customer[i].oldbalance > 0) ? 'D' : 'C';
customer[i].newbalance  = customer[i].oldbalance - customer[i].payment;
}
clrscr();
        for (i = 0; i < n; ++i)
        writeoutput(customer[i]);
        getch();
}

record readinput(int i)
{
        record customer;
        printf("\nCustomer no %d\n", i + 1);
        printf(" Name : ");
        scanf("%s", customer.name);
        printf(" Street : ");
```

```
        scanf("%s", customer.street);
        printf(" City : ");
        scanf("%s", customer.city);
        printf("Account number ");
        scanf("%d", &customer.acct_no);
        printf("Previous balance : ");
        scanf("%f", &customer.oldbalance);
        printf("Current payment : ");
        scanf("%f", &customer.payment);
        printf("Payment date (dd/mm/yy) : ");
        scanf("%d/%d/%d", &customer.lastpayment.day,
                          &customer.lastpayment.month,
                          &customer.lastpayment.year);
        return(customer);
}

void writeoutput(record customer)
{
        printf("\nName   : %s", customer.name);
        printf("\tAccount number : %d", customer.acct_no);
        printf("\nPrevious balance : %.2f", customer.oldbalance);
        printf("\tCurrent payment : %.2f", customer.payment);
        printf("\nNew Balance : %.2f", customer.newbalance);
        printf("\nAccount Status : ");
        switch(customer.acct_type)
        {
          case 'C' :
                printf("Current\n\n");
                break;
          case 'O' :
                printf("OverDue\n\n");
                break;
          case 'D' :
                printf("Suspended\n\n");
                break;
          default :
                printf("Error\n\n");
          }
        return;
}
```

O/P:- Name : as Account number : 12
Previous balance : 10.00 Current payment : 1.00
New Balance : 9.00
Account Status : OverDue

Program 17: Example on argc

```
#include<stdio.h>
int j;
void main(int argc, char *argv[])
{
        printf("Number of arguments is %d \n", argc);
        for (j = 0; j < argc; j++)
          printf("Argument number %2d is %s\n", j , argv[j]);
}
O/P:- Number of arguments is 1
Argument number  0 is D:\TC\TC\BIN\NONAME02.EXE
```

Program 18: Maintaining Student Info

```
#include<stdio.h>
#include<conio.h>
struct student
{
        char name[30];
        unsigned age : 6;
        unsigned rollno : 8;
        unsigned branch : 2;
};

void main()
{
        struct student a[10];
        int i, n, rollno, branch, age;

        printf("Enter the number of students\n");
        scanf("%d", &n);
        fflush(stdin);

        for (i = 0; i < n ; i++)
        {
                printf("Enter the information of the student = %d\n", i+1);
                printf("Name  : ");
                scanf("%s", a[i].name);

                printf("Age   :");
                scanf("%d", &age);

                printf("Roll number : ");
                scanf("%d", &rollno);
```

```
                printf("Branch : ");
                scanf("%d", &branch);

                fflush(stdin);
                a[i].age = age;
                a[i].rollno = rollno;
                a[i].branch = branch;
        }
        printf("  Name    Age  Rollno Branch\n");
        for (i = 0; i < n; i++)
        {
                printf("%20s ", a[i].name);
                printf("%4d %5d", a[i].age, a[i].rollno);
                switch(a[i].branch)
                {
                        case 0:
                          printf("Computer Science\n");
                          break;
                        case 1:
                          printf("Information Science\n");
                          break;
                        case 2:
                          printf(" Electrical Science\n");
                          break;
                        default:
                          printf(" Electronics\n");
                          break;
                }
        }
}
O/P:- Enter the number of students
2
Enter the information of the student = 1
Name  : a
Age   :22
Roll number : 1
Branch : cs
Enter the information of the student = 2
Name  : b
Age   :23
Roll number : 2
Branch : e
 Name    Age  Rollno Branch
            a   22     1Computer Science
            b   23     2Computer Science
```

TREES

program 1: Insert, preorder, inorder,postorder,search

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
        int info;
        struct node *llink;
        struct node *rlink;
};

typedef struct node *NODE;


NODE getnode()
{
        NODE x;
        x = (NODE) malloc(sizeof(struct node));
        if (x == NULL)
        {
                printf("Out of Memory");
                exit(0);
        }
        return x;
}

void freenode (NODE x)
{
        free(x);
}


NODE insert(int item, NODE root)
{
        NODE temp;
        NODE cur;
        NODE prev;
        char direction[10];
        int i;

        temp = getnode();
        temp->info = item;
```

```
        temp->llink = temp->rlink = NULL;

        if (root == NULL)
        return temp;

        printf("Press [L] for Left insertion [R] for Right insertion\n");
        fflush(stdin);
        scanf("%s", direction);
        toupper(direction);

        prev = NULL;
        cur = root;

        for (i = 0; i < strlen(direction) && cur != NULL; i++)
        {
                prev = cur;
                if (direction[i] == 'L')
                  cur = cur->llink;
                else
                  cur = cur->rlink;
        }
        if (cur != NULL || i != strlen(direction))
        {
                printf("Insertion not possible\n");
                freenode(temp);
                return root;
        }
        if (direction[i-1] == 'L')
                prev->llink = temp;
        else
                prev->rlink = temp;
        return root;
}

void preorder(NODE root)
{
        if (root != NULL)
        {
                printf("%d", root->info);
                preorder(root->llink);
                preorder(root->rlink);
        }
}
void inorder(NODE root)
{
        if (root != NULL)
```

```
        {
                inorder(root->llink);
                printf("%d ", root->info);
                inorder(root->rlink);
        }
}

void postorder(NODE root)
{
        if (root != NULL)
        {
                postorder(root->llink);
                postorder(root->rlink);
                printf("%d ", root->info);
        }
}

void search(int item, NODE root, int *flag)
{
        if (root != NULL)
        {
                search(item, root->llink, flag);
                if (item == root->info)
                {
                        *flag = 1;
                        return;
                }
                search(item, root->rlink, flag);
        }
}

int choice, item, flag;
void main()
{
        NODE root = NULL;
        clrscr();
        for (;;)
        {
                printf("1:Insert  ");
                printf("2:Preorder  ");
                printf("3:Inorder  ");
                printf("4:Postorder  ");
                printf("5:Search  ");
                printf("6:Exit ");
                printf("Enter choice");
                scanf("%d", &choice);
```

```
switch(choice)
{
        case 1 :
                printf("Enter item to be inserted \n");
                scanf("%d", &item);
                root = insert(item, root);
                break;
        case 2 :
                if (root == NULL)
                        printf("Tree is empty\n");
                else
                {
                        printf("Preorder traversal\n");
                        preorder(root);
                        printf("\n");
                }
                break;
        case 3:
                if (root == NULL)
                        printf("Tree is empty\n");
                else
                {
                        printf("Inorder traversal is\n");
                        inorder(root);
                        printf("\n");
                }
                break;
        case 4:
                if (root == NULL)
                        printf("Tree is empty\n");
                else
                {
                        printf("Postorder traversal is\n");
                        postorder(root);
                        printf("\n");
                }
                break;
        case 5:
                if (root == NULL)
                        printf("Tree is empty\n");
                else
                {
                        printf("Enter item to search\n");
                        scanf("%d", &item);
                        flag = 0;
                        search(item, root, &flag);
```

```
                                        if (flag == 1)
                                          printf("Search successful\n");
                                        else
                                          printf("Unsuccesful search\n");
                                }
                                break;
                        default :
                                exit(0);
                }
        }
}
```

program 2: Stack implementation of Linked List

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
struct node
{
  int info;
  struct node *link;
};
typedef struct node *NODE;


NODE getnode()
{
        NODE x;
        x = (NODE) malloc(sizeof(struct node));
        if (x == NULL)
        {
                printf("Out of Memory");
                exit(0);
        }
        return x;
}

void freenode (NODE x)
{
        free(x);
}

NODE insert_front(int item, NODE first)
```

```
{
        NODE temp;
        temp = getnode();
        temp->info = item;
        temp->link = first;
        return temp;
}

void display(NODE first)
{
        NODE temp;
        if (first == NULL)
        {
                printf("List is empty\n");
                return;
        }
        printf("The contents of linked list\n");
        temp=first;
        while(temp != NULL)
        {
                printf("%d ", temp->info);
                temp = temp->link;
        }
        printf("\n");
}
NODE delete_front(NODE first)
{
        NODE temp;
        if (first == NULL)
        {
                printf("List is empty cannot be deleted\n");
                return first;
        }
        temp = first;
        first = first->link;

        printf("The item deleted is %d\n", temp->info);
        freenode(temp);
        return first;
}

void main()
{
  NODE first = NULL;
  int choice, item;
  clrscr();
```

```
 for (;;)
 {
        printf("1:Insert Front\n2:Display\n");
        printf("3.Delete\n");
        printf("4:Quit\n");
        printf("Enter the choice\n");
        scanf("%d", &choice);
        switch(choice)
        {
                case 1 :
                        printf("Enter the item to be inserted\n");
                        scanf("%d", &item);
                        first = insert_front(item, first);
                        break;
                case 2 :
                        display(first);
                        break;
                case 3:
                        first = delete_front(first);
                        break;
                default :
                        exit(0);
        }
 }
}
```

```
O/P:- 1:Insert Front
2:Display
3.Delete
4:Quit
Enter the choice
1
Enter the item to be inserted
7
1:Insert Front
2:Display
3.Delete
4:Quit
Enter the choice
2
The contents of linked list
7 8 9
1:Insert Front
2:Display
3.Delete
4:Quit
Enter the choice
```

```
3
The item deleted is 7
1:Insert Front
2:Display
3.Delete
4:Quit
Enter the choice
3
The item deleted is 8
1:Insert Front
2:Display
3.Delete
4:Quit
Enter the choice
3
The item deleted is 9
1:Insert Front
2:Display
3.Delete
4:Quit
Enter the choice
3
List is empty cannot be deleted
1:Insert Front
2:Display
3.Delete
4:Quit
Enter the choice
```

Program 3: Program to maintain a heap

```
/* CH8PR9.C: Program to maintain a heap. */

#include <stdio.h>
#include <conio.h>

void restoreup ( int, int * ) ;
void restoredown ( int, int *, int ) ;
void makeheap ( int *, int ) ;
void add ( int, int *, int * ) ;
int replace ( int, int *, int ) ;
int del ( int *, int * ) ;

void main( )
{
```

```
        int arr [20] = { 1000, 7, 10, 25, 17, 23, 27, 16,
                                        19, 37, 42, 4, 33, 1, 5, 11 } ;
        int i, n = 15 ;

        clrscr( ) ;
        makeheap ( arr, n ) ;

        printf ( "Heap:\n" ) ;
        for ( i = 1 ; i <= n ; i++ )
                printf ( "%d\t", arr [i] ) ;

        i = 24 ;
        add ( i, arr, &n ) ;

        printf ( "\n\nElement added %d.\n", i ) ;
        printf ( "\nHeap after addition of an element:\n" ) ;
        for ( i = 1 ; i <= n ; i++ )
                printf ( "%d\t", arr [i] ) ;

        i = replace ( 2, arr, n ) ;
        printf ( "\n\nElement replaced %d.\n", i ) ;
        printf ( "\nHeap after replacement of an element:\n" ) ;
        for ( i = 1 ; i <= n ; i++ )
                printf ( "%d\t", arr [i] ) ;

        i = del ( arr, &n ) ;
        printf ( "\n\nElement deleted %d.\n", i ) ;
        printf ( "\nHeap after deletion of an element:\n" ) ;
        for ( i = 1 ; i <= n ; i++ )
                printf ( "%d\t", arr [i] ) ;

        getch( ) ;
}

void restoreup ( int i, int *arr )
{
        int val ;
        val = arr [i] ;
        while ( arr [i / 2] <= val )
        {
                arr [i] = arr [i / 2] ;
                i = i / 2 ;
        }
        arr [i] = val ;
}
```

```
void restoredown ( int pos, int *arr, int n )
{
        int i, val ;
        val = arr [pos] ;
        while ( pos <= n / 2 )
        {
                i = 2 * pos ;
                if ( ( i < n ) && ( arr [i] < arr [i + 1] ) )
                        i++ ;
                if ( val >= arr [i] )
                        break ;
                arr [pos] = arr [i] ;
                pos = i ;
        }
        arr [pos] = val ;
}

void makeheap ( int *arr, int n )
{
        int i ;
        for ( i = n / 2 ; i >= 1 ; i-- )
                restoredown ( i, arr, n ) ;
}

void add ( int val, int *arr, int *n )
{
        ( *n ) ++ ;
        arr [*n] = val ;
        restoreup ( *n, arr ) ;
}
int replace ( int i, int *arr, int n )
{
        int r = arr [1] ;
        arr [1] = i ;
        for ( i = n / 2 ; i >= 1 ; i-- )
                restoredown ( i, arr, n ) ;
        return r ;
}

int del ( int *arr, int *n )
{
        int val ;
        val = arr [1] ;
        arr [1] = arr [*n] ;
        ( *n ) -- ;
        restoredown ( 1, arr, *n ) ;
```

```
        return val ;
}
```

O/P:-

```
Heap:
42    37    33    19    23    27    16    7    17    10
4     25    1     5     11
```

Element added 24.

```
Heap after addition of an element:
42    37    33    24    23    27    16    19    17    10
4     25    1     5     11    7
```

Element replaced 42.

```
Heap after replacement of an element:
37    24    33    19    23    27    16    7    17    10
4     25    1     5     11    2
```

Element deleted 37.

```
Heap after deletion of an element:
33    24    27    19    23    25    16    7    17    10
4     2     1     5     11
```

Program 4: Program which maintains a B-tree of order 5

```
/* CH8PR8.C: Program which maintains a B-tree of order 5. */

#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <alloc.h>

#define MAX 4
#define MIN 2

struct btnode
{
        int count ;
        int value[MAX + 1] ;
        struct btnode *child[MAX + 1] ;
} ;

struct btnode * insert ( int, struct btnode * ) ;
```

```
int setval ( int, struct btnode *, int *, struct btnode ** ) ;
struct btnode * search ( int, struct btnode *, int * ) ;
int searchnode ( int, struct btnode *, int * ) ;
void fillnode ( int, struct btnode *, struct btnode *, int ) ;
void split ( int, struct btnode *, struct btnode *,
                                    int, int *, struct btnode ** ) ;
struct btnode * delete ( int, struct btnode * ) ;
int delhelp ( int, struct btnode * ) ;
void clear ( struct btnode *, int ) ;
void copysucc ( struct btnode *, int ) ;
void restore ( struct btnode *, int ) ;
void rightshift ( struct btnode *, int ) ;
void leftshift ( struct btnode *, int ) ;
void merge ( struct btnode *, int ) ;
void display ( struct btnode * ) ;

void main( )
{
        struct node *root ;
        root = NULL ;

        clrscr( ) ;

        root = insert ( 27, root ) ;
        root = insert ( 42, root ) ;
        root = insert ( 22, root ) ;
        root = insert ( 47, root ) ;
        root = insert ( 32, root ) ;
        root = insert ( 2, root ) ;
        root = insert ( 51, root ) ;
        root = insert ( 40, root ) ;
        root = insert ( 13, root ) ;

        printf ( "B-tree of order 5:\n" ) ;
        display ( root ) ;

        root = delete ( 22, root ) ;
        root = delete ( 11, root ) ;

        printf ( "\n\nAfter deletion of values:\n" ) ;
        display ( root ) ;

        getch( ) ;
}

/* inserts a value in the B-tree*/
```

```
struct btnode * insert ( int val, struct btnode *root )
{
        int i ;
        struct btnode *c, *n ;
        int flag ;

        flag = setval ( val, root, &i, &c ) ;
        if ( flag )
        {
                n = ( struct btnode * ) malloc ( sizeof ( struct btnode ) ) ;
                n -> count = 1 ;
                n -> value [1] = i ;
                n -> child [0] = root ;
                n -> child [1] = c ;
                return n ;
        }
        return root ;
}

/* sets the value in the node */
int setval ( int val, struct btnode *n, int *p, struct btnode **c )
{
        int k ;
        if ( n == NULL )
        {
                *p = val ;
                *c = NULL ;
                return 1 ;
        }
        else
        {
                if ( searchnode ( val, n, &k ) )
                        printf ( "\nKey value already exists.\n" ) ;
                if ( setval ( val, n -> child [k], p, c ) )
                {
                        if ( n -> count < MAX )
                        {
                                fillnode ( *p, *c, n, k ) ;
                                return 0 ;
                        }
                        else
                        {
                                split ( *p, *c, n, k, p, c ) ;
                                return 1 ;
                        }
                }
```

```
                return 0 ;
        }
}

/* searches value in the node */
struct btnode * search ( int val, struct btnode *root, int *pos )
{
        if ( root == NULL )
                return NULL ;
        else
        {
                if ( searchnode ( val, root, pos ) )
                        return root ;
                else
                        return search ( val, root -> child [*pos], pos ) ;
        }
}

/* searches for the node */
int searchnode ( int val, struct btnode *n, int *pos )
{
        if ( val < n -> value [1] )
        {
                *pos = 0 ;
                return 0 ;
        }
        else
        {
                *pos = n -> count ;
                while ( ( val < n -> value [*pos] ) && *pos > 1 )
                        ( *pos )-- ;
                if ( val == n -> value [*pos] )
                        return 1 ;
                else
                        return 0 ;
        }
}


/* adjusts the value of the node */
void fillnode ( int val, struct btnode *c, struct btnode *n, int k )
{
        int i ;
        for ( i = n -> count ; i > k ; i-- )
        {
                n -> value [i + 1] = n -> value [i] ;
```

```
                n -> child [i + 1] = n -> child [i] ;
        }
        n -> value [k + 1] = val ;
        n -> child [k + 1] = c ;
        n -> count++ ;
}

/* splits the node */
void split ( int val, struct btnode *c, struct btnode *n,
                                int k, int *y, struct btnode **newnode )
{
        int i, mid ;

        if ( k <= MIN )
                mid = MIN ;
        else
                mid = MIN + 1 ;

        *newnode = ( struct btnode * ) malloc ( sizeof ( struct btnode ) ) ;

        for ( i = mid + 1 ; i <= MAX ; i++ )
        {
                ( *newnode ) -> value [i - mid] = n -> value [i] ;
                ( *newnode ) -> child [i - mid] = n -> child [i] ;
        }

        ( *newnode ) -> count = MAX - mid ;
        n -> count = mid ;

        if ( k <= MIN )
                fillnode ( val, c, n, k ) ;
        else
                fillnode ( val, c, *newnode, k - mid ) ;

        *y = n -> value [n -> count] ;
        ( *newnode ) -> child [0] = n -> child [n -> count] ;
        n -> count-- ;
}

/* deletes value from the node */
struct btnode * delete ( int val, struct btnode *root )
{
        struct btnode * temp ;
        if ( ! delhelp ( val, root ) )
                printf ( "\nValue %d not found.", val ) ;
        else
```

```
        {
                if ( root -> count == 0 )
                {
                        temp = root ;
                        root = root -> child [0] ;
                        free ( temp ) ;
                }
        }
        return root ;
}

/* helper function for delete( ) */
int delhelp ( int val, struct btnode *root )
{
        int i ;
        int flag ;
        if ( root == NULL )
                return 0 ;
        else
        {
                flag = searchnode ( val, root, &i ) ;
                if ( flag )
                {
                        if ( root -> child [i - 1] )
                        {
                                copysucc ( root, i ) ;
                                flag = delhelp ( root -> value [i], root -> child [i] ) ;
                                if ( !flag )
                                        printf ( "\nValue %d not found.", val ) ;
                        }
                        else
                                clear ( root, i ) ;
                }
                else
                        flag = delhelp ( val, root -> child [i] ) ;

                if ( root -> child [i] != NULL )
                {
                        if ( root -> child [i] -> count < MIN )
                                restore ( root, i ) ;
                }
                return flag ;
        }
}

/* removes the value from the node and adjusts the values */
```

```
void clear ( struct btnode *node, int k )
{
        int i ;
        for ( i = k + 1 ; i <= node -> count ; i++ )
        {
                node -> value [i - 1] = node -> value [i] ;
                node -> child [i - 1] = node -> child [i] ;
        }
        node -> count-- ;
}

/* copies the successor of the value that is to be deleted */
void copysucc ( struct btnode *node, int i )
{
        struct btnode *temp ;

        temp = node -> child [i] ;

        while ( temp -> child[0] )
                temp = temp -> child [0] ;

        node -> value [i] = temp -> value [1] ;
}

/* adjusts the node */
void restore ( struct btnode *node, int i )
{
        if ( i == 0 )
        {
                if ( node -> child [1] -> count > MIN )
                        leftshift ( node, 1 ) ;
                else
                        merge ( node, 1 ) ;
        }
        else
        {
                if ( i == node -> count )
                {
                        if ( node -> child [i - 1] -> count > MIN )
                                rightshift ( node, i ) ;
                        else
                                merge ( node, i ) ;
                }
                else
                {
                        if ( node -> child [i - 1] -> count > MIN )
```

```
                        rightshift ( node, i ) ;
                else
                {
                        if ( node -> child [i + 1] -> count > MIN )
                                leftshift ( node, i + 1 ) ;
                        else
                                merge ( node, i ) ;
                }
        }
    }
}

/* adjusts the values and children while shifting the value from parent to right
   child */
void rightshift ( struct btnode *node, int k )
{
        int i ;
        struct btnode *temp ;

        temp = node -> child [k] ;

        for ( i = temp -> count ; i > 0 ; i-- )
        {
                temp -> value [i + 1] = temp -> value [i] ;
                temp -> child [i + 1] = temp -> child [i] ;
        }

        temp -> child [1] = temp -> child [0] ;
        temp -> count++ ;
        temp -> value [1] = node -> value [k] ;

        temp = node -> child [k - 1] ;
        node -> value [k] = temp -> value [temp -> count] ;
        node -> child [k] -> child [0] = temp -> child [temp -> count] ;
        temp -> count-- ;
}

/* adjusts the values and children while shifting the value from parent to left
   child */
void leftshift ( struct btnode *node, int k )
{
        int i ;
        struct btnode *temp ;

        temp = node -> child [k - 1] ;
        temp -> count++ ;
```

```
        temp -> value [temp -> count] = node -> value [k] ;
        temp -> child [temp -> count] = node -> child [k] -> child [0] ;

        temp = node -> child [k] ;
        node -> value [k] = temp -> value [1] ;
        temp -> child [0] = temp -> child [1] ;
        temp -> count-- ;

        for ( i = 1 ; i <= temp -> count ; i++ )
        {
                temp -> value [i] = temp -> value [i + 1] ;
                temp -> child [i] = temp -> child [i + 1] ;
        }
}

/* merges two nodes */
void merge ( struct btnode *node, int k )
{
        int i ;
        struct btnode *temp1, *temp2 ;

        temp1 = node -> child [k] ;
        temp2 = node -> child [k - 1] ;
        temp2 -> count++ ;
        temp2 -> value [temp2 -> count] = node -> value [k] ;
        temp2 -> child [temp2 -> count] = node -> child [0] ;

        for ( i = 1 ; i <= temp1 -> count ; i++ )
        {
                temp2 -> count++ ;
                temp2 -> value [temp2 -> count] = temp1 -> value [i] ;
                temp2 -> child [temp2 -> count] = temp1 -> child [i] ;
        }
        for ( i = k ; i < node -> count ; i++ )
        {
                node -> value [i] = node -> value [i + 1] ;
                node -> child [i] = node -> child [i + 1] ;
        }
        node -> count-- ;
        free ( temp1 ) ;
}

/* displays the B-tree */
void display ( struct btnode *root )
{
        int i ;
```

```
        if ( root != NULL )
        {
                for ( i = 0 ; i < root -> count ; i++ )
                {
                        display ( root -> child [i] ) ;
                        printf ( "%d\t", root -> value [i + 1] ) ;
                }
                display ( root -> child [i] ) ;
        }
}
```

O/P:- B-tree of order 5:

```
2     13     22     27     32     40     42     47     51
Value 11 not found.

After deletion of values:
2     13     27     32     40     42     47     51
```

Program 5: Program to maintain an AVL tree

```
/* CH8PR7.C: Program to maintain an AVL tree. */

#include <stdio.h>
#include <conio.h>
#include <alloc.h>

#define FALSE 0
#define TRUE 1

struct AVLNode
{
        int data ;
        int balfact ;
        struct AVLNode *left ;
        struct AVLNode *right ;
} ;

struct AVLNode * buildtree ( struct AVLNode *, int, int * ) ;
struct AVLNode * deldata ( struct AVLNode *, int, int * ) ;
struct AVLNode * del ( struct AVLNode *, struct AVLNode *, int * ) ;
struct AVLNode * balright ( struct AVLNode *, int * ) ;
struct AVLNode * balleft ( struct AVLNode *, int * ) ;
void display ( struct AVLNode * ) ;
void deltree ( struct AVLNode * ) ;
```

```
void main( )
{
        struct AVLNode *avl = NULL ;
        int h ;

        clrscr( ) ;

        avl = buildtree ( avl, 20, &h ) ;
        avl = buildtree ( avl, 6, &h ) ;
        avl = buildtree ( avl, 29, &h ) ;
        avl = buildtree ( avl, 5, &h ) ;
        avl = buildtree ( avl, 12, &h ) ;
        avl = buildtree ( avl, 25, &h ) ;
        avl = buildtree ( avl, 32, &h ) ;
        avl = buildtree ( avl, 10, &h ) ;
        avl = buildtree ( avl, 15, &h ) ;
        avl = buildtree ( avl, 27, &h ) ;
        avl = buildtree ( avl, 13, &h ) ;

        printf ( "\nAVL tree:\n" ) ;
        display ( avl ) ;

        avl = deldata ( avl, 20, &h ) ;
        avl = deldata ( avl, 12, &h ) ;

        printf ( "\nAVL tree after deletion of a node:\n" ) ;
        display ( avl ) ;

        deltree ( avl ) ;

        getch( ) ;
}

/* inserts an element into tree */
struct AVLNode * buildtree ( struct AVLNode *root, int data, int *h )
{
        struct AVLNode *node1, *node2 ;

        if ( !root )
        {
                root = ( struct AVLNode * ) malloc ( sizeof ( struct AVLNode ) ) ;
                root -> data = data ;
                root -> left = NULL ;
                root -> right = NULL ;
                root -> balfact = 0 ;
                *h = TRUE ;
```

```
                return ( root ) ;
        }

        if ( data < root -> data )
        {
                root -> left = buildtree ( root -> left, data, h ) ;
                /* If left subtree is higher */
                if ( *h )
                {
                        switch ( root -> balfact )
                        {
                                case 1:
                                        node1 = root -> left ;
                                        if ( node1 -> balfact == 1 )
                                        {
                                                printf ( "\nRight rotation along %d.", root ->
data ) ;
                                                root -> left = node1 -> right ;
                                                node1 -> right = root ;
                                                root -> balfact = 0 ;
                                                root = node1 ;
                                        }
                                        else
                                        {
                                                printf ( "\nDouble rotation, left along %d",
                                       node1 -> data ) ;
                                                node2 = node1 -> right ;
                                                node1 -> right = node2 -> left ;
                                                printf ( " then right along %d.\n", root -> data
) ;
                                                node2 -> left = node1 ;
                                                root -> left = node2 -> right ;
                                                node2 -> right = root ;
                                                if ( node2 -> balfact == 1 )
                                                        root -> balfact = -1 ;
                                                else
                                                        root -> balfact = 0 ;
                                                if ( node2 -> balfact == -1 )
                                                        node1 -> balfact = 1 ;
                                                else
                                                        node1 -> balfact = 0 ;
                                                root = node2 ;
                                        }
                                        root -> balfact = 0 ;
                                        *h = FALSE ;
                                        break ;
```

```
                        case 0:
                                root -> balfact = 1 ;
                                break ;

                        case -1:
                                root -> balfact = 0 ;
                                *h = FALSE ;
                        }
                }
        }

        if ( data > root -> data )
        {
                root -> right = buildtree ( root -> right, data, h ) ;
                /* If the right subtree is higher */
                if ( *h )
                {
                        switch ( root -> balfact )
                        {
                                case 1:
                                        root -> balfact = 0 ;
                                        *h = FALSE ;
                                        break ;

                                case 0:
                                        root -> balfact = -1 ;
                                        break;

                                case -1:
                                        node1 = root -> right ;
                                        if ( node1 -> balfact == -1 )
                                        {
                                                printf ( "\nLeft rotation along %d.", root ->
data ) ;

                                                root -> right = node1 -> left ;
                                                node1 -> left = root ;
                                                root -> balfact = 0 ;
                                                root = node1 ;
                                        }
                                        else
                                        {
                                                printf ( "\nDouble rotation, right along %d",
                                               node1 -> data ) ;
                                                node2 = node1 -> left ;
                                                node1 -> left = node2 -> right ;
```

```
                                        node2 -> right = node1 ;
                                        printf ( " then left along %d.\n", root -> data )
;
                                        root -> right = node2 -> left ;
                                        node2 -> left = root ;

                                        if ( node2 -> balfact == -1 )
                                                root -> balfact = 1 ;
                                        else
                                                root -> balfact = 0 ;
                                        if ( node2 -> balfact == 1 )
                                                node1 -> balfact = -1 ;
                                        else
                                                node1 -> balfact = 0 ;
                                        root = node2 ;
                                }
                                root -> balfact = 0 ;
                                *h = FALSE ;
                        }
                }
        }
        return ( root ) ;
}

/* deletes an item from the tree */
struct AVLNode * deldata ( struct AVLNode *root, int data, int *h )
{
        struct AVLNode *node ;

        if ( !root )
        {
                printf ( "\nNo such data." ) ;
                return ( root ) ;
        }
        else
        {
                if ( data < root -> data )
                {
                        root -> left = deldata ( root -> left, data, h ) ;
                        if ( *h )
                                root = balright ( root, h ) ;
                }
                else
                {
                        if ( data > root -> data )
                        {
```

```
                        root -> right = deldata ( root -> right, data, h ) ;
                        if ( *h )
                                root = balleft ( root, h ) ;
                }
                else
                {
                        node = root ;
                        if ( node -> right == NULL )
                        {
                                root = node -> left ;
                                *h = TRUE ;
                                free ( node ) ;
                        }
                        else
                        {
                                if ( node -> left == NULL )
                                {
                                        root = node -> right ;
                                        *h = TRUE ;
                                        free ( node ) ;
                                }
                                else
                                {
                                        node -> right = del ( node -> right, node, h ) ;
                                        if ( *h )
                                                root = balleft ( root, h ) ;
                                }
                        }
                }
        }
        return ( root ) ;
}

struct AVLNode * del ( struct AVLNode *succ, struct AVLNode *node, int *h )
{
        struct AVLNode *temp = succ ;
        if ( succ -> left != NULL )
        {
                succ -> left = del ( succ -> left, node, h ) ;
                if ( *h )
                        succ = balright ( succ, h ) ;
        }
        else
        {
                temp = succ ;
```

```
                node -> data = succ -> data ;
                succ = succ -> right ;
                free ( temp ) ;
                *h = TRUE ;
        }
        return ( succ ) ;
}

/* balances the tree, if right sub-tree is higher */
struct AVLNode * balright ( struct AVLNode *root, int *h )
{
        struct AVLNode *node1, *node2 ;

        switch ( root -> balfact )
        {
                case 1:
                        root -> balfact = 0 ;
                        break;

                case 0:
                        root -> balfact = -1 ;
                        *h  = FALSE ;
                        break;

                case -1:
                        node1 = root -> right ;
                        if ( node1 -> balfact <= 0 )
                        {
                                printf ( "\nLeft rotation along %d.", root -> data ) ;
                                root -> right = node1 -> left ;
                                node1 -> left = root ;
                                if ( node1 -> balfact == 0 )
                                {
                                        root -> balfact = -1 ;
                                        node1 -> balfact = 1 ;
                                  *h = FALSE ;
                                }
                                else
                                {
                                        root -> balfact = node1 -> balfact = 0 ;
                                }
                                root = node1 ;
                        }
                        else
                        {
                                printf ( "\nDouble rotation, right along %d", node1 -> data );
```

```
                        node2 = node1 -> left ;
                        node1 -> left = node2 -> right ;
                        node2 -> right = node1 ;
                        printf ( " then left along %d.\n", root -> data );
                        root -> right = node2 -> left ;
                        node2 -> left = root ;

                        if ( node2 -> balfact == -1 )
                                root -> balfact = 1 ;
                        else
                                root -> balfact = 0 ;
                        if ( node2 -> balfact == 1 )
                                node1 -> balfact = -1 ;
                        else
                                node1 -> balfact = 0 ;
                        root = node2 ;
                        node2 -> balfact = 0 ;
                }
        }
        return ( root ) ;
}

/* balances the tree, if left sub-tree is higher */
struct AVLNode * balleft ( struct AVLNode *root, int *h )
{
        struct AVLNode *node1, *node2 ;

        switch ( root -> balfact )
        {
                case -1:
                        root -> balfact = 0 ;
                        break ;

                case 0:
                        root -> balfact = 1 ;
                        *h = FALSE ;
                        break ;

                case 1:
                        node1 = root -> left ;
                        if ( node1 -> balfact >= 0 )
                        {
                                printf ( "\nRight rotation along %d.", root -> data ) ;
                                root -> left = node1 -> right ;
                                node1 -> right = root ;
                                if ( node1 -> balfact == 0 )
```

```
                        {
                                root -> balfact = 1 ;
                                node1 -> balfact = -1 ;
                                *h = FALSE ;
                        }
                        else
                        {
                                root -> balfact = node1 -> balfact = 0 ;
                        }
                        root = node1 ;
                }
                else
                {
                        printf ( "\nDouble rotation, left along %d", node1 -> data ) ;
                        node2 = node1 -> right ;
                        node1 -> right = node2 -> left ;
                        node2 -> left = node1 ;
                        printf ( " then right along %d.\n", root -> data ) ;
                        root -> left = node2 -> right ;
                        node2 -> right = root ;

                        if ( node2 -> balfact == 1 )
                                root -> balfact = -1 ;
                        else
                                root -> balfact = 0 ;
                        if ( node2-> balfact == -1 )
                                node1 -> balfact = 1 ;
                        else
                                node1 -> balfact = 0 ;
                        root = node2 ;
                        node2 -> balfact = 0 ;
                }
        }
        return ( root ) ;
}

/* displays the tree in-order */
void display ( struct AVLNode *root )
{
        if ( root != NULL )
        {
                display ( root -> left ) ;
                printf ( "%d\t", root -> data ) ;
                display ( root -> right ) ;
        }
}
```

```
/* deletes the tree */
void deltree ( struct AVLNode *root )
{
        if ( root != NULL )
        {
                deltree ( root -> left ) ;
                deltree ( root -> right ) ;
        }
        free ( root ) ;
}
```

O/P:- Left rotation along 6.

```
AVL tree:
5      6      10     12     13     15     20     25     27     29
32
AVL tree after deletion of a node:
5      6      10     13     15     25     27     29     32
```

Program 6: Program to reconstruct a binary search tree and traversals

```
/* CH8PR6.C: Program to reconstruct a binary search tree. */

#include <stdio.h>
#include <conio.h>
#include <alloc.h>

#define MAX 101

struct node
{
        struct node *left ;
        int data ;
        struct node *right ;
} ;

void insert ( struct node **, int ) ;
void preorder ( struct node * ) ;
void postorder ( struct node * ) ;
void inorder ( struct node * ) ;
struct node * recons ( int *, int *, int ) ;
void deltree ( struct node * ) ;

int in[MAX], pre[MAX], x ;
```

```
void main( )
{
        struct node *t, *p, *q ;
        int req, i, num ;

        t = NULL ;  /* empty tree */

        clrscr( ) ;
        printf ( "Specify the number of items to be inserted: " ) ;
        while ( 1 )
        {
                scanf ( "%d", &req ) ;
                if ( req >= MAX || req <= 0 )
                        printf ( "\nEnter number between 1 to 100.\n" ) ;
                else
                        break ;
        }

        for ( i = 0 ; i < req ; i++ )
        {
                printf ( "Enter the data: " ) ;
                scanf ( "%d", &num ) ;
                insert ( &t, num ) ;
        }

        printf ( "\nIn-order   Traversal:\n" ) ;
        x = 0 ;
        inorder ( t ) ;

        printf ( "\nPre-order  Traversal:\n" ) ;
        x = 0 ;
        preorder ( t ) ;

        printf ( "\nPost-order Traversal:\n" ) ;
        x = 0 ;
        postorder ( t ) ;

        deltree ( t ) ;
        t = NULL ;
        t = recons ( in, pre, req ) ;

        printf ( "\n\nAfter reconstruction of the binary tree.\n" ) ;

        x = 0 ;
        printf ( "\nIn-order   Traversal:\n" ) ;
        inorder ( t ) ;
```

```
        x = 0 ;
        printf ( "\nPre-order  Traversal:\n" ) ;
        preorder ( t ) ;
        x = 0 ;
        printf ( "\nPost-order Traversal:\n" ) ;
        postorder ( t ) ;

        deltree ( t ) ;
        getch( ) ;
}

/* inserts a new node in a binary search tree */
void insert ( struct node **sr, int num )
{
        if ( *sr == NULL )
        {
                *sr = ( struct node * ) malloc ( sizeof ( struct node ) ) ;

                ( *sr ) -> left = NULL ;
                ( *sr ) -> data = num ;
                ( *sr ) -> right = NULL ;
                return ;
        }
        else  /* search the node to which new node will be attached */
        {
                /* if new data is less, traverse to left */
                if ( num < ( *sr ) -> data )
                        insert ( &( ( *sr ) -> left ), num ) ;
                else
                        /* else traverse to right */
                        insert ( &( ( *sr ) -> right ), num ) ;
        }
}

void preorder ( struct node *t )
{
        if ( t != NULL )
        {
                printf ( "%d\t", pre[x++]= t -> data ) ;
                preorder ( t -> left ) ;
                preorder ( t -> right ) ;
        }
}

void postorder ( struct node *t )
```

```
{
        if ( t != NULL )
        {
                postorder ( t -> left ) ;
                postorder ( t -> right ) ;
                printf ( "%d\t", t -> data ) ;
        }
}

void inorder ( struct node *t )
{
        if ( t != NULL )
        {
                inorder ( t -> left ) ;
                printf ( "%d\t", in[x++]= t -> data ) ;
                inorder ( t -> right ) ;
        }
}

struct node * recons ( int *inorder, int *preorder, int noofnodes )
{
        struct node *temp, *left, *right ;
        int tempin[100], temppre[100], i, j ;

        if ( noofnodes == 0 )
                return NULL ;

        temp = ( struct node * ) malloc ( sizeof ( struct node ) ) ;
        temp -> data = preorder[0] ;
        temp -> left = NULL ;
        temp -> right = NULL ;

        if ( noofnodes == 1 )
                return temp ;
        for ( i = 0 ; inorder[i] != preorder[0] ; )
                i++ ;

        if ( i > 0 )
        {
                for ( j = 0 ; j <= i  ; j++ )
                        tempin[j] = inorder[j] ;

                for ( j = 0 ; j < i  ; j++ )
                        temppre[j] = preorder[j + 1] ;
        }
```

```
        left = recons ( tempin, temppre, i ) ;
        temp -> left = left ;

        if ( i < noofnodes - 1 )
        {
                for ( j = i ; j < noofnodes - 1 ; j++ )
                {
                        tempin[j - i] = inorder[j + 1] ;
                        temppre[j - i] = preorder[j + 1] ;
                }
        }

        right = recons ( tempin, temppre, noofnodes - i - 1 ) ;
        temp -> right = right ;

        return temp ;
}

void deltree ( struct node *t )
{
        if ( t != NULL )
        {
                deltree ( t -> left ) ;
                deltree ( t -> right ) ;
        }
        free ( t ) ;
}
```

O/P:- Specify the number of items to be inserted: 10
Enter the data: 5
Enter the data: 6
Enter the data: 7
Enter the data: 8
Enter the data: 34
Enter the data: 2
Enter the data: 4
Enter the data: 6
Enter the data: 9
Enter the data: 0

In-order Traversal:
0 2 4 5 6 6 7 8 9 34

Pre-order Traversal:
5 2 0 4 6 7 6 8 34 9

Post-order Traversal:

```
0     4     2     6     9     34    8     7     6     5
```

After reconstruction of the binary tree.

```
In-order   Traversal:
0     2     4     5     6     6     7     8     9     34

Pre-order  Traversal:
5     2     0     4     6     7     6     8     34    9

Post-order Traversal:
0     4     2     6     9     34    8     7     6     5
```

program 7: Program to maintain a threaded binary tree

```
/* CH8PR5.C: Program to maintain a threaded binary tree. */

#include <stdio.h>
#include <conio.h>
#include <alloc.h>

enum boolean
{
        false = 0,
        true = 1
} ;

struct thtree
{
        enum boolean isleft ;
        struct thtree *left ;
        int data ;
        struct thtree *right ;
        enum boolen isright ;
} ;

void insert ( struct thtree **, int ) ;
void delete ( struct thtree **, int ) ;
void search ( struct thtree **, int, struct thtree **,
                                    struct thtree **, int * ) ;
void inorder ( struct thtree * ) ;
void deltree ( struct thtree ** ) ;
```

```
void main( )
{
        struct thtree *th_head ;

        th_head = NULL ;  /* empty tree */

        insert ( &th_head, 11 ) ;
        insert ( &th_head, 9 ) ;
        insert ( &th_head, 13 ) ;
        insert ( &th_head, 8 ) ;
        insert ( &th_head, 10 ) ;
        insert ( &th_head, 12 ) ;
        insert ( &th_head, 14 ) ;
        insert ( &th_head, 15 ) ;
        insert ( &th_head, 7 ) ;

        clrscr( ) ;
        printf ( "Threaded binary tree before deletion:\n" ) ;
        inorder ( th_head ) ;

        delete ( &th_head, 10 ) ;
        printf ( "\nThreaded binary tree after deletion:\n" ) ;
        inorder ( th_head ) ;

        delete ( &th_head, 14 ) ;
        printf ( "\nThreaded binary tree after deletion:\n" ) ;
        inorder ( th_head ) ;

        delete ( &th_head, 8 ) ;
        printf ( "\nThreaded binary tree after deletion:\n" ) ;
        inorder ( th_head ) ;

        delete ( &th_head, 13 ) ;
        printf ( "\nThreaded binary tree after deletion:\n" ) ;
        inorder ( th_head ) ;

        deltree ( &th_head ) ;

        getch( ) ;
}

/* inserts a node in a threaded binary tree */
void insert ( struct thtree **s, int num )
{
        struct thtree *p, *z, *head = *s ;
```

```
/* allocating a new node */
z = malloc ( sizeof ( struct thtree ) ) ;

z -> isleft = true ;  /* indicates a thread */
z -> data = num ;  /* assign new data */
z -> isright = true ;  /* indicates a thread */

/* if tree is empty */
if ( *s == NULL )
{
        head = malloc ( sizeof ( struct thtree ) ) ;

        /* the entire tree is treated as a left sub-tree of the head node */
        head -> isleft = false ;
        head -> left = z ;  /* z becomes leftchild of the head node */
        head -> data = -9999 ;  /* no data */
        head -> right = head ;  /* right link will always be pointing
                                                                to itself */
        head -> isright = false ;

        *s = head ;

        z -> left = head ;  /* left thread to head */
        z -> right = head ;  /* right thread to head */
}
else  /* if tree is non-empty */
{
        p = head -> left ;

        /* traverse till the thread is found attached to the head */
        while ( p != head )
        {
                if ( p -> data > num )
                {
                        if ( p -> isleft != true )  /* checking for a thread */
                                p = p -> left ;
                        else
                        {
                                z -> left = p -> left ;
                                p -> left = z ;
                                p -> isleft = false ;  /* indicates a link */
                                z -> isright = true ;
                                z -> right = p ;
                                return ;
                        }
```

```
                        }
                        else
                        {
                                if ( p -> data < num )
                                {
                                        if ( p -> isright != true )
                                                p = p -> right ;
                                        else
                                        {
                                                z -> right = p -> right ;
                                                p -> right = z ;
                                                p -> isright = false ;  /* indicates a link */
                                                z -> isleft = true ;
                                                z -> left = p ;
                                                return ;
                                        }
                                }
                        }
                }
        }
}

/* deletes a node from the binary search tree */
void delete ( struct thtree **root, int num )
{
        int found ;
        struct thtree *parent, *x, *xsucc ;

        /* if tree is empty */
        if ( *root == NULL )
        {
                printf ( "\nTree is empty" ) ;
                return ;
        }

        parent = x = NULL ;

        /* call to search function to find the node to be deleted */
        search ( root, num, &parent, &x, &found ) ;

        /* if the node to deleted is not found */
        if ( found == false )
        {
                printf ( "\nData to be deleted, not found" ) ;
                return ;
        }
```

```
/* if the node to be deleted has two children */
if ( x -> isleft == false && x -> isright == false )
{
        parent = x ;
        xsucc = x -> right ;

        while ( xsucc -> isleft == false )
        {
                parent = xsucc ;
                xsucc = xsucc -> left ;
        }

        x -> data = xsucc -> data ;
        x = xsucc ;
}

/* if the node to be deleted has no child */
if ( x -> isleft == true && x -> isright == true )
{
        /* if node to be deleted is a root node */
        if ( parent == NULL )
        {
                ( *root ) -> left = *root ;
                ( *root ) -> isleft = true ;

                free ( x ) ;
                return ;
        }

        if ( parent -> right == x )
        {
                parent -> isright = true ;
                parent -> right = x -> right ;
        }
        else
        {
                parent -> isleft = true ;
                parent -> left = x -> left ;
        }

        free ( x ) ;
        return ;
}

/* if the node to be deleted has only rightchild */
```

```
if ( x -> isleft == true && x -> isright == false )
{
        /* node to be deleted is a root node */
        if ( parent == NULL )
        {
                ( *root ) -> left = x -> right ;
                free ( x ) ;
                return ;
        }

        if ( parent -> left == x )
        {
                parent -> left = x -> right ;
                x -> right -> left = x -> left ;
        }
        else
        {
                parent -> right = x -> right ;
                x -> right -> left = parent ;
        }

        free ( x ) ;
        return ;
}

/* if the node to be deleted has only left child */
if ( x -> isleft == false && x -> isright == true )
{
        /* the node to be deleted is a root node */
        if ( parent == NULL )
        {
                parent = x ;
                xsucc = x -> left ;

                while ( xsucc -> right == false )
                xsucc = xsucc -> right ;

                xsucc -> right = *root ;

                ( *root ) -> left = x -> left ;

                free ( x ) ;
                return ;
        }

        if ( parent -> left == x )
```

```
                {
                        parent -> left = x -> left ;
                        x -> left -> right = parent ;
                }
                else
                {
                        parent -> right = x -> left ;
                        x -> left -> right = x -> right ;
                }

                free ( x ) ;
                return ;
        }
}

/* returns the address of the node to be deleted, address of its parent and
        whether the node is found or not */
void search ( struct thtree **root, int num, struct thtree **par,
                                struct thtree **x, int *found )
{
        struct thtree *q ;

        q = ( *root ) -> left ;
        *found = false ;
        *par = NULL ;

        while ( q != *root )
        {
                /* if the node to be deleted is found */
                if ( q -> data == num )
                {
                        *found = true ;
                        *x = q ;
                        return ;
                }

                *par = q ;

                if ( q -> data > num )
                {
                        if ( q -> isleft == true )
                        {
                                *found = false ;
                                x = NULL ;
                                return ;
                        }
```

```
                        q = q -> left ;
                }
                else
                {
                        if ( q -> isright == true )
                        {
                                *found = false ;
                                *x = NULL ;
                                return ;
                        }
                        q = q -> right ;
                }
        }
}

/* traverses the threaded binary tree in inorder */
void inorder ( struct thtree *root )
{
        struct thtree *p ;

        p = root -> left ;

        while ( p != root )
        {
                while ( p -> isleft == false )
                        p = p -> left ;

                printf ( "%d\t", p -> data ) ;

                while ( p -> isright == true )
                {
                        p = p -> right ;

                        if ( p == root )
                                break ;

                        printf ( "%d\t", p -> data ) ;

                }
                p = p -> right ;
        }
}

void deltree ( struct thtree **root )
{
        while ( ( *root ) -> left != *root )
```

```
                delete ( root, ( *root ) -> left -> data ) ;
}
```

O/P:- Threaded binary tree before deletion:

```
7       8       9       10      11      12      13      14      15
Threaded binary tree after deletion:
7       8       9       11      12      13      14      15
Threaded binary tree after deletion:
7       8       9       11      12      13      15
Threaded binary tree after deletion:
7       9       11      12      13      15
Threaded binary tree after deletion:
7       9       11      12      15
```

Program 8: Program to insert and delete a node from the binary search tree

```
/* CH8PR4.C: Program to insert and delete a node
                        from the binary search tree. */

#include <stdio.h>
#include <conio.h>
#include <alloc.h>

#define TRUE 1
#define FALSE 0

struct btreenode
{
        struct btreenode *leftchild ;
        int data ;
        struct btreenode *rightchild ;
} ;

void insert ( struct btreenode **, int ) ;
void delete ( struct btreenode **, int ) ;
void search ( struct btreenode **, int, struct btreenode **,
                                struct btreenode **, int * ) ;
void inorder ( struct btreenode * ) ;

void main( )
{
        struct btreenode *bt ;
        int req, i = 0, num, a[ ] = { 11, 9, 13, 8, 10, 12, 14, 15, 7 } ;

        bt = NULL ;  /* empty tree */
```

```
        clrscr( ) ;

        while ( i <= 8 )
        {
                insert ( &bt, a[i] ) ;
                i++ ;
        }
        clrscr( ) ;
        printf ( "Binary tree before deletion:\n" ) ;
        inorder ( bt ) ;

        delete ( &bt, 10 ) ;
        printf ( "\nBinary tree after deletion:\n" ) ;
        inorder ( bt ) ;

        delete ( &bt, 14 ) ;
        printf ( "\nBinary tree after deletion:\n" ) ;
        inorder ( bt ) ;

        delete ( &bt, 8 ) ;
        printf ( "\nBinary tree after deletion:\n" ) ;
        inorder ( bt ) ;

        delete ( &bt, 13 ) ;
        printf ( "\nBinary tree after deletion:\n" ) ;
        inorder ( bt ) ;
}

/* inserts a new node in a binary search tree */
void insert ( struct btreenode **sr, int num )
{
        if ( *sr == NULL )
        {
                *sr = malloc ( sizeof ( struct btreenode ) ) ;

                ( *sr ) -> leftchild = NULL ;
                ( *sr ) -> data = num ;
                ( *sr ) -> rightchild = NULL ;
        }
        else  /* search the node to which new node will be attached */
        {
                /* if new data is less, traverse to left */
                if ( num < ( *sr ) -> data )
                        insert ( &( ( *sr ) -> leftchild ), num ) ;
                else
```

```
                        /* else traverse to right */
                        insert ( &( ( *sr ) -> rightchild ), num ) ;
        }
}

/* deletes a node from the binary search tree */
void delete ( struct btreenode **root, int num )
{
        int found ;
        struct btreenode *parent, *x, *xsucc ;

        /* if tree is empty */
        if ( *root == NULL )
        {
                printf ( "\nTree is empty" ) ;
                return ;
        }

        parent = x = NULL ;

        /* call to search function to find the node to be deleted */
        search ( root, num, &parent, &x, &found ) ;

        /* if the node to deleted is not found */
        if ( found == FALSE )
        {
                printf ( "\nData to be deleted, not found" ) ;
                return ;
        }

        /* if the node to be deleted has two children */
        if ( x -> leftchild != NULL && x -> rightchild != NULL )
        {
                parent = x ;
                xsucc = x -> rightchild ;

                while ( xsucc -> leftchild != NULL )
                {
                        parent = xsucc ;
                        xsucc = xsucc -> leftchild ;
                }

                x -> data = xsucc -> data ;
                x = xsucc ;
        }
```

```
        /* if the node to be deleted has no child */
        if ( x -> leftchild == NULL && x -> rightchild == NULL )
        {
                if ( parent -> rightchild == x )
                        parent -> rightchild = NULL ;
                else
                        parent -> leftchild = NULL ;

                free ( x ) ;
                return ;
        }

        /* if the node to be deleted has only rightchild */
        if ( x -> leftchild == NULL && x -> rightchild != NULL )
        {
                if ( parent -> leftchild == x )
                        parent -> leftchild = x -> rightchild ;
                else
                        parent -> rightchild = x -> rightchild ;

                free ( x ) ;
                return ;
        }

        /* if the node to be deleted has only left child */
        if ( x -> leftchild != NULL && x -> rightchild == NULL )
        {
                if ( parent -> leftchild == x )
                        parent -> leftchild = x -> leftchild ;
                else
                        parent -> rightchild = x -> leftchild ;

                free ( x ) ;
                return ;
        }
}

/*returns the address of the node to be deleted, address of its parent and
  whether the node is found or not */
void search ( struct btreenode **root, int num, struct btreenode **par, struct
                btreenode **x, int *found )
{
        struct btreenode *q ;

        q = *root ;
        *found = FALSE ;
```

```
        *par = NULL ;

        while ( q != NULL )
        {
                /* if the node to be deleted is found */
                if ( q -> data == num )
                {
                        *found = TRUE ;
                        *x = q ;
                        return ;
                }

                *par = q ;

                if ( q -> data > num )
                        q = q -> leftchild ;
                else
                        q = q -> rightchild ;
        }
}

/* traverse a binary search tree in a LDR (Left-Data-Right) fashion */
void inorder ( struct btreenode *sr )
{
        if ( sr != NULL )
        {
                inorder ( sr -> leftchild ) ;

                /* print the data of the node whose leftchild is NULL or the path  has
                        already been traversed */
                printf ( "%d\t", sr -> data ) ;

                inorder ( sr -> rightchild ) ;
        }
}
```

O/P:- Binary tree before deletion:

```
7       8       9       10      11      12      13      14      15
Binary tree after deletion:
7       8       9       11      12      13      14      15
Binary tree after deletion:
7       8       9       11      12      13      15
Binary tree after deletion:
7       9       11      12      13      15
Binary tree after deletion:
7       9       11      12      15
```

Program 9: Program to implement a binary search tree.

```
/* CH8PR3.C: Program to implement a binary search tree. */

#include <stdio.h>
#include <conio.h>
#include <alloc.h>
struct btreenode
{
        struct btreenode *leftchild ;
        int data ;
        struct btreenode *rightchild ;
} ;

void insert ( struct btreenode **, int ) ;
void inorder ( struct btreenode * ) ;
void preorder ( struct btreenode * ) ;
void postorder ( struct btreenode * ) ;

void main( )
{
        struct btreenode *bt ;
        int req, i = 1, num ;

        bt = NULL ;  /* empty tree */

        clrscr( ) ;
        printf ( "Specify the number of items to be inserted: " ) ;
        scanf ( "%d", &req ) ;

        while ( i++ <= req )
        {
                printf ( "Enter the data: " ) ;
                scanf ( "%d", &num ) ;
                insert ( &bt, num ) ;
        }

        printf ( "\nIn-order   Traversal: " ) ;
        inorder ( bt ) ;

        printf ( "\nPre-order  Traversal: " ) ;
        preorder ( bt ) ;
```

```
        printf ( "\nPost-order Traversal: " ) ;
        postorder ( bt ) ;
}

/* inserts a new node in a binary search tree */
void insert ( struct btreenode **sr, int num )
{
        if ( *sr == NULL )
        {
                *sr = malloc ( sizeof ( struct btreenode ) ) ;

                (*sr)->leftchild = NULL ;
                (*sr)->data = num ;
                (*sr)->rightchild = NULL ;
                return ;
        }
        else  /* search the node to which new node will be attached */
        {
                /* if new data is less, traverse to left */
                if ( num < (*sr) -> data )
                        insert ( &( (*sr)-> leftchild ), num ) ;
                else
                        /* else traverse to right */
                        insert ( &( (*sr)-> rightchild ), num ) ;
        }
        return ;
}

/* traverse a binary search tree in a LDR (Left-Data-Right) fashion */
void inorder ( struct btreenode *sr )
{
        if ( sr != NULL )
        {
                inorder ( sr -> leftchild ) ;

                /* print the data of the node whose leftchild is NULL or the path
                   has already been traversed */
                printf ( "\t%d", sr -> data ) ;

                inorder ( sr -> rightchild ) ;
        }
        else
                return ;
}

/* traverse a binary search tree in a DLR (Data-Left-right) fashion */
```

```
void preorder ( struct btreenode *sr )
{
        if ( sr != NULL )
        {
                /* print the data of a node */
                printf ( "\t%d", sr -> data ) ;
                /* traverse till leftchild is not NULL */
                preorder ( sr -> leftchild ) ;
                /* traverse till rightchild is not NULL */
                preorder ( sr -> rightchild ) ;
        }
        else
                return ;
}


/* traverse a binary search tree in LRD (Left-Right-Data) fashion */
void postorder ( struct btreenode *sr )
{
        if ( sr != NULL )
        {
                postorder ( sr -> leftchild ) ;
                postorder ( sr -> rightchild ) ;
                printf ( "\t%d", sr -> data ) ;
        }
        else
                return ;
}

/* commented out
struct x
{
        int data;
} ;

void init(struct x **);

void main()
{
        struct x *q;
        init(&q);
}
void init (struct x **q)
{
        (*q)->data = 10;
        clrscr();
```

```
        printf("%d ", (*q)->data);
}
*/
```

```
O/P Specify the number of items to be inserted: 5
Enter the data: 1
Enter the data: 2
Enter the data: 3
Enter the data: 4
Enter the data: 5

In-order   Traversal:  1     2     3     4     5
Pre-order  Traversal:  1     2     3     4     5
Post-order Traversal:  5     4     3     2     1
```

Program 10: Program to build a binary search tree from an array

```
/* CH8PR2.C: Program to build a binary search tree from an array. */

#include <stdio.h>
#include <conio.h>
#include <alloc.h>

struct node
{
        struct node *left ;
        char data ;
        struct node *right ;
} ;

struct node * buildtree ( int ) ;
void inorder ( struct node * ) ;

char a[ ] = {
                        'A', 'B', 'C', 'D', 'E', 'F', 'G', '\0', '\0', 'H', '\0',
                        '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0'
                } ;

void main( )
{
        struct node *root ;

        clrscr( ) ;
```

```
        root = buildtree ( 0 ) ;
        printf ( "In-order Traversal:\n" ) ;
        inorder ( root ) ;

        getch( ) ;
}
struct node * buildtree ( int n )
{
        struct node *temp = NULL ;
        if ( a[n] != '\0' )
        {
                temp = ( struct node * ) malloc ( sizeof ( struct node ) ) ;
                temp -> left = buildtree ( 2 * n + 1 ) ;
                temp -> data = a[n] ;
                temp -> right = buildtree ( 2 * n + 2 ) ;
        }
        return temp ;
}

void inorder ( struct node *root )
{
        if ( root != NULL )
        {
                inorder ( root -> left ) ;
                printf ( "%c\t", root -> data ) ;
                inorder ( root -> right ) ;
        }
}
```

O/P:- In-order Traversal:

D B H E A F C G

Program 11: build a binary search tree from arrays

```
/* CH8PR1.C: Program to build a binary search tree from arrays. */

#include <stdio.h>
#include <conio.h>
#include <alloc.h>

struct node
{
        struct node *left ;
        char data ;
```

```
        struct node *right ;
} ;

struct node * buildtree ( int ) ;
void inorder ( struct node * ) ;

char arr[ ] = { 'A', 'B', 'C', 'D', 'E', 'F', 'G', '\0', '\0', 'H' } ;
int  lc[ ] = {  1,  3,  5,  -1,  9, -1, -1,  -1,  -1, -1 } ;
int  rc[ ] = {  2,  4,  6,  -1, -1, -1, -1,  -1,  -1, -1 } ;

void main( )
{
        struct node *root ;

        clrscr( ) ;

        root = buildtree ( 0 ) ;
        printf ( "In-order Traversal:\n" ) ;
        inorder ( root ) ;

        getch( ) ;
}

struct node * buildtree ( int index )
{
        struct node *temp = NULL ;
        if ( index != -1 )
        {
                temp = ( struct node * ) malloc ( sizeof ( struct node ) ) ;
                temp -> left = buildtree ( lc[index] ) ;
                temp -> data = arr[index] ;
                temp -> right = buildtree ( rc[index] ) ;
        }
        return temp ;
}

void inorder ( struct node *root )
{
        if ( root != NULL )
        {
                inorder ( root -> left ) ;
                printf ( "%c\t", root -> data ) ;
                inorder ( root -> right ) ;
        }
}
```

O/P:- In-order Traversal:

D B H E A F C G